

Extending Intuitionistic Logic to a Probability Calculus (in Lean 4)

Anonymous Author(s)

Abstract

Kolmogorov's probability fixes the event algebra to be Boolean, so every event is decidable. This is a poor fit for the semidecidable properties, termination, reachability, that verification cares about. Following the Cox–Jaynes interpretation of probability as extended logic, we define a construct for extending a logic (treated as a parameter in our approach) to a calculus of probability, and specify criteria for when this works: (1) when the resulting calculus can reconstruct the original logic in the limiting case of certain (0– or 1– probability) events, and (2) when the construction, evaluated at classical logic, agrees with classical probability theory. Instantiating the construction at intuitionistic logic gives valuations on frames, complete Heyting algebras of open sets, the algebras of verifiable propositions. In this setting, we define slack, $1 - v(A) - v(\neg A)$, the classical complement rule's deficit and boundary mass in the measure model. Slack pinpoints exactly when Bayesian conditioning, which treats the evidence as fully decided, gives an incorrect update, disagreeing with Dempster's rule. A separate obstruction, a five-element counterexample, is why Cox's sum rule cannot be derived and must instead be posited. Classical probability credits the slack of a semidecidable property to its failure, an event no finite observation can confirm. The whole development is machine-checked in Lean 4 over mathlib, in roughly 5,000 lines with no sorry.

Keywords: intuitionistic logic, probability, Cox's theorem, belief functions, locales, Lean, mathlib, formalized mathematics

1 Introduction

Kolmogorov's probability [Kolmogorov 1933] assigns numerical values representing uncertainty to events. The event logic is classical, forming a Boolean σ -algebra. The assignments satisfy non-negativity, normalization, and σ -additivity. This commits the framework to the axioms of classical logic, including the law of excluded middle. Many properties of programs (e.g. "does this computation halt?", "does this process reach a given state?") are only *semidecidable*: a procedure confirms the property when it holds, and no procedure, in general, confirms its failure. The mismatch that results is not a contradiction. It is a failure of discrimination. A probability calculus built on a particular logic inherits that logic's identifications, so it cannot discriminate events the logic proves equal.

In the case of classical logic, this identification is the *law of excluded middle*: $\text{halts} \sqcup \neg \text{halts} = \top$, so the Boolean event algebra contains $\neg \text{halts}$. The complement rule $P(A^c) = 1 - P(A)$ then forces the remainder $1 - P(\text{halts})$ onto $\neg \text{halts}$, an event no finite observation can confirm. The events "not yet confirmed" and "refuted" are identified. For a program whose non-termination is independent of ZFC [Yedidia and Aaronson 2016], the classical value $P(\text{halts}) = 0$ is correct, yet neither running the program nor any ZFC proof ever establishes it. Uniformly over programs, computing these sharp values is exactly as hard as the halting problem.

Another failure of discrimination appears in *linear logic*, whose tensor $\otimes$ distinguishes a from $a \otimes a$, one use of a resource from two. Modularity with $\otimes$ in place of $\sqcup$, $v(a) + v(b) = v(a \sqcup b) + v(a \otimes b)$, inherits idempotence of disjunction, $a \sqcup a = a$, and at $b = a$ forces $v(a \otimes a) = v(a)$. So, no assignment satisfying this axiom distinguishes one use of a from two.

Probability is, as the Cox–Jaynes tradition has it, *extended logic* [Cox 1946; Jaynes 2003], and different logics can be extended through the same construction. The logic of events, in its algebraic form the event algebra, is a *parameter* of the construction, and the choice of event algebra determines the probability calculus. An event algebra consists of a set of events together with operations interpreting the logic's connectives and structural rules, while a probability calculus is differentiated from an arbitrary grading of events by satisfying certain axioms. We specify how to construct such extensions, and propose two criteria for when the extension of a logic is a probability calculus:

- (1) the valuations taking only the values 0 and 1 (the two degrees of complete certainty) must reconstruct the underlying event logic. This is what we take "extended logic" to mean.
- (2) the construction evaluated at classical logic must return the classical theory, ensuring it is a genuine generalization of classical probability.

We propose two fixes for the identifications above. Both extend a logic to a probability calculus, and on finite frames the second reduces to the first. The first fix we propose is one that accepts the standard axioms imposed on a grading to make it a probability calculus, while choosing an event algebra that makes the required distinctions. The extension of the event algebra to a probability calculus is then given by the *valuations*, i.e. the maps from events to numbers satisfying monotonicity, normalization, and modularity. The

resulting extension does not guarantee defining a probability calculus, and we specify the conditions under which it does: the $\{0, 1\}$ -valued valuations are the points of the event algebra, and they reconstruct its order when it is distributive. Intuitionistic logic, in particular, is a case for which this approach yields an extension, and we work it out in detail. Its event algebras are the Heyting algebras, and we work with frames (the complete Heyting algebras). The open sets of any topological space form a frame. On a frame the pseudocomplement $\neg a$ need not satisfy $a \sqcup \neg a = \top$, so the complement rule weakens to $v(a) + v(\neg a) \leq 1$. The gap $\text{slack}(a) = 1 - v(a) - v(\neg a)$ is the mass of the part of a neither confirmed nor refuted. For the valuation that a measure μ induces on the open sets of a space, the slack of an open set is the μ -measure of its boundary.

The second fix also applies to logics whose connectives are not lattice operations, such as linear logic. It generalizes the axioms of the probability calculus in addition to varying the event algebra. It applies to any logic presented by a set of *certain states*, its $\{0, 1\}$ -valued assignments. The calculus is the set of finite mixtures of certain states. A mixture that takes only the values 0 and 1 is one of the certain states it mixes, because every certain state with positive weight must agree with it at every event. So the $\{0, 1\}$ -valued elements of the calculus are exactly the certain states, which present the logic, and criterion (1) holds. For finitely many events, the axioms of the calculus are exactly the affine laws that hold on every certain state. On finite frames and finite Boolean algebras these laws coincide with monotonicity, normalization, and modularity, and the calculus is the set of valuations.

For linear logic, the event algebra is a quantale, a complete lattice with an associative tensor $\otimes$ that need not be idempotent. The first fix fails there because of the probability axioms, not the event algebra: the modularity axiom identifies a with $a \otimes a$. Probabilistic coherence spaces [Danos and Ehrhard 2011] avoid this by assigning weights to the points of each formula's web instead of to formulas, with $a \otimes a$ interpreted on a product web. Their elements are not valuations in the sense of the modularity axiom. For every multiplicative-additive formula over finite data types, the calculus of the second fix is contained in the probabilistic coherence space of the formula, and the certain elements of that space are cliques of Girard's coherence space [Girard 1987].

Our extension and coherence spaced coincide at first-order linear logic. At higher order they can differ: for a program with two functional arguments the calculus is the set of randomized sequential programs, a strict subset of the probabilistic coherence space (Section 11). The probabilistic coherence space, the standard model of probabilistic programs at this type, therefore admits behaviours that no sequential program flipping coins can produce in a linear probabilistic language for this type. Reasoning in it can certify only weaker guarantees than the programs satisfy. The gap has

the same numerical form as the gap between classical and non-classical correlations in quantum foundations [Cabello et al. 2014]. At !Bool the calculus is also a strict subset.

Quantum logic, whose event algebras are orthomodular lattices [Birkhoff and von Neumann 1936], is a logic to which neither fix applies. Both rely on $\{0, 1\}$ -valued assignments: the first on $\{0, 1\}$ -valued valuations, which do not exist on non-distributive lattices, and the second relies on having certain states. A certain state of quantum logic would assign 1 to exactly one ray of every orthogonal basis, and by the Kochen–Specker theorem [Kochen and Specker 1967] there is none in dimension at least 3. We prove this for an 18-ray configuration in $\mathbb{R}^4$ [Cabello et al. 1996], so the calculus of the second fix is empty there. Quantum states, which by Gleason's theorem [Gleason 1957] are given by density matrices, nevertheless exist, and they fail criterion (1).

Contributions.

1. **Weak excluded middle** (Sections 2 and 7). Global weak excluded middle on the opens of X is exactly extremal disconnectedness of X , for compact Hausdorff X exactly the Stonean spaces.
2. **Two characterizations** (Sections 2, 3 and 5). The complement rule holds for *every* valuation iff the frame satisfies excluded middle, the exact content of Van Horn's classical negation axiom R3 [Horn 2003]. Dempster's rule and Bayesian conditioning, classically identical, agree iff the evidence has zero slack. The conditional $v(a \sqcap b)/v(b)$, the product rule, and Bayes symmetry hold unchanged. The complement rule weakens to the interval $[v(A), 1 - v(\neg A)]$ of width $\text{slack}(A)$, and the law of total probability to a lower bound whose deficit is a conditional slack.
3. **Structure theorems** (Sections 4 and 5). On finite frames a valuation is a classical mass function on points. In general it splits into atomic and diffuse parts, and on its Boolean core it is a belief function.
4. **Cox's theorem, dissected** (Section 6, Appendix C). Cox's theorem and its repairs [Cox 1946; Halpern 1999; Horn 2003] derive classical probability from qualitative requirements on plausibility. The product-rule half is independent of the logic, proved by an Aczél/Hölder generator built from scratch. The sum-rule half, a theorem classically, admits no derivation here: on a five-element frame a disjunction's value is not a function of its disjuncts' values, so modularity must be posited. Weak excluded middle does not restore it, since the counterexample frame satisfies weak excluded middle.
5. **Two grounding models** (Section 7). Any classical measure, read on the opens, is a valuation whose slack is boundary mass. A sharp readout of halting is uncomputable, and on Cantor space the value of the halting event is Chaitin's Ω . A classical posterior overstates

refutation by exactly the conditional boundary mass $\mu(\partial A \cap B)/\mu(B)$ (Theorem 33), so the classical and intuitionistic readings differ only where boundary mass is positive. Slack is submodular yet does not determine the valuation.

6. **The monad dichotomy** (Section 8). Product valuations are unique on $\text{Low}(\mathbb{N} \times \mathbb{N})$ and provably non-unique on $\text{Set}(\mathbb{N} \times \mathbb{N})$, where Fubini fails. The countably-presented fragment composes associatively and commutatively.
7. **Beyond frames** (Section 9). The certain valuations reconstruct the event order exactly on distributive lattices, and completeness is not needed. On the non-distributive diamond M_3 no certain valuation exists, so the construction does not extend to quantum event lattices unchanged. Grading linear logic's $\otimes$ by modularity forces $v(a \otimes a) = v(a)$, so such a grading cannot count resources.
8. **Probability calculi from certain states** (Section 10). The calculus of a logic presented by its $\{0, 1\}$ -valued certain states is the set of their finite mixtures. Its certain elements are exactly the certain states, and its axioms are exactly the affine laws valid on the certain states. On finite frames this returns the valuations, and on finite Boolean algebras Kolmogorov's calculus. For arbitrary sets of events, the functions satisfying every finitely supported law valid on the certain states are exactly the barycenters of probability measures concentrated on the certain states.
9. **Linear logic** (Sections 10.2 and 10.3). Every multiplicative-additive formula over finite data types denotes a probabilistic coherence space, whose certain elements are cliques of Girard's coherence space, and the calculus of the formula is contained in it. At implications between data types the two coincide. At $!\text{Bool}$ the calculus is a strict subset.
10. **Second-order linear types** (Section 11). For a program with two functional arguments over finite data types, the certain elements of the probabilistic coherence space are the deterministic sequential programs, and the calculus is exactly the set of randomized sequential programs, by both directions of Kuhn's theorem [Kuhn 1953]. The calculus is a strict subset of the probabilistic coherence space. The space is the clique-constrained polytope of an exclusivity graph whose stable-set polytope is the calculus, so the two can agree only if the graph has no odd hole or anti-hole. One tensor further, a clique of Girard's coherence space, Shannon's stable set of $C_5 \boxtimes C_5$, is not an element of the probabilistic coherence space.
11. **Quantum event logic** (Section 12). For the 18 rays and 9 orthogonal bases of Cabello's Kochen–Specker set in $\mathbb{R}^4$ [Cabello et al. 1996], no $\{0, 1\}$ -valued assignment gives exactly one ray of every basis the value

1, so the calculus is empty, while every Born-rule assignment of a unit vector sums to 1 on every basis. Quantum probability on this event logic fails criterion (1).

12. **A first formal treatment.** Everything above is machine-checked in Lean 4 over mathlib, to our knowledge the first mechanization of non-additive probability over intuitionistic logic.

Mechanization. The development is carried out in Lean 4 [de Moura and Ullrich 2021] over mathlib [mathlib Community 2020], in roughly 5,000 lines with no sorry, and is included in full as anonymous supplementary material.¹ Two points are specific to a proof assistant. We separate *object-level* classicality, excluded middle in the frame Ω , from the *meta-level* classicality mathlib inherits through `Classical.choice`, identify the four places where the meta-level dependence is essential (Section 3), and give choice-free proofs of the finite separation results. The product-rule half of Cox's theorem needs an Aczél/Hölder regreduation absent from mathlib, which we build from scratch.

2 Valuations on Frames

2.1 The Event Logic

Definition 1 (frame). A *frame* is a complete lattice Ω in which finite meets distribute over arbitrary joins,

$$a \sqcap \left(\bigsqcup_{i \in I} b_i \right) = \bigsqcup_{i \in I} (a \sqcap b_i)$$

for every $a \in \Omega$ and every family $\{b_i\}_{i \in I} \subseteq \Omega$. Equivalently, it is a complete Heyting algebra. By $x \Rightarrow y$ we denote the Heyting implication in a frame, and $\neg a := a \Rightarrow \perp$ is the pseudocomplement of $a \in \Omega$. Also, a^c denotes a complement of a , if it exists, i.e. an element such that $a \sqcap a^c = \perp$ and $a \sqcup a^c = \top$.

Frames are the algebraic semantics of intuitionistic logic and the point-free abstraction of topological spaces. The same objects with morphisms reversed form the category of *locales*, so the distinction is invisible at the level of a single algebra. We say *frame* throughout and reserve *locale* for the spatial and categorical statements (points of a locale, classifying locales) where that word is the standard one. The motivating example throughout is $\Omega = \mathcal{O}(X)$, the opens of a space, with $\sqcap, \sqcup$ as intersection and union and the pseudocomplement $\neg a$ of an open given by the *interior* of its set complement. In mathlib this is the typeclass `Order.Frame`, and the Heyting operations come with the key asymmetry already in place: $a \sqcap \neg a = \perp$ holds in any Heyting algebra (`inf_compl_eq_bot`), while $a \sqcup \neg a = \top$ is available only in

¹Claude (Anthropic) was used to assist with drafting and debugging the Lean formalization.

Boolean algebras. An element with $a \sqcup \neg a = \top$ is *complemented*: it satisfies its own instance of excluded middle.

2.2 The Valuation Structure

The central object assigns a degree of certainty to each element of the frame, with inclusion–exclusion as the only interaction axiom:

Definition 2 (valuation^{Lean}). A *valuation* on a frame Ω is a *monotone* function $v : \Omega \rightarrow [0, \infty]$ with $v(\perp) = 0$, $v(\top) = 1$ (*normalization*), and, for all $a, b \in \Omega$, *modularity*:

$$v(a) + v(b) = v(a \sqcup b) + v(a \sqcap b).$$

```

structure Valuation (Ω : Type*) [Order.
  Frame Ω] where
  toFun : Ω → ℝ≥0∞
  map_bot' : toFun ⊥ = 0
  map_top' : toFun ⊤ = 1
  mono' : Monotone toFun
  modular' : ∀ a b, toFun a + toFun b
    = toFun (a ⊔ b) + toFun (a ⊓ b)

```

This axiom set is not ours: Weatherson’s axioms for intuitionistic probability are exactly these [Weatherson 2003], and the valuations of constructive measure theory [Coquand and Spitters 2009; Vickers 2008] are their Scott-continuous instances (Section 13).

Values live in the extended non-negative reals $[0, \infty]$, and normalization confines them to $[0, 1]^{\text{Lean}}$. The conventions this choice carries, unconditional summation and truncated subtraction, are an artifact of the mechanization and are recorded in Appendix A.

2.3 Slack

Modularity applied to the disjoint pair $(a, \neg a)$ collapses to $v(a) + v(\neg a) = v(a \sqcup \neg a)^{\text{Lean}}$. So, the join need not be $\top$:

Theorem 3 (the complement bound^{Lean}). *For every valuation v on a frame Ω and every $a \in \Omega$, $v(a) + v(\neg a) \leq 1$.*

```

theorem Valuation.add_compl_le_one (v :
  Valuation Ω) (a : Ω) :
  v a + v ac ≤ 1

```

This is a *derived* Dempster–Shafer sub-additivity inequality. The deficit

$$\text{slack}(a) = 1 - (v(a) + v(\neg a)) = 1 - v(a \sqcup \neg a)$$

is the valuation of the undecided region. It decomposes canonically. The pair $(\neg a, \neg a)$, double negation against negation, is always disjoint. However, it is a partition only under *weak* excluded middle ($\neg a \sqcup \neg \neg a = \top$). Applying modularity to this disjoint pair splits the slack into two logically independent gaps:

Definition 4 (regular element). An element $a \in \Omega$ is *regular* (or $\neg\neg$ -stable) if $a = \neg\neg a$.

Theorem 5 (slack decomposition^{Lean}). *For every valuation v on a frame Ω and every $a \in \Omega$, $\text{slack}(a) = \text{dnGap}(a) + \text{deMorganGap}(a)$, where $\text{dnGap}(a) = v(\neg\neg a) - v(a)$ is 0 if a is $\neg\neg$ -stable (regular), and $\text{deMorganGap}(a) = 1 - v(\neg\neg a \sqcup \neg a)$ is 0 if weak excluded middle holds at a .*

The single Dempster–Shafer “ignorance” number is thus resolved into a regularity defect and a De Morgan defect: the first measures failure of double-negation stability, the second corresponds to a named intermediate logic (weak excluded middle). Constructively, the contrapositive of a universal hypothesis returns only its double-negated form, so for open sets, this specification of probability does not fall victim to Hempel’s raven paradox [Hempel 1945]: evidence of the white sock indoors confirms at most $\neg\neg a$, not a itself, and dnGap is exactly the gap between confirming $\neg\neg a$ and confirming a itself.

Because dnGap and deMorganGap are themselves defined from v , the apparent three unknowns $v(a)$, $v(\neg a)$, and $v(\neg\neg a)$ are not independent: they are part of a three-constraint system, yielding the following result:

Theorem 6 (slack in closed form^{Lean}). *For every valuation v on a frame Ω and every $a \in \Omega$,*

$$\text{slack}(a) = (v(\neg\neg a) - v(a)) + (1 - v(\neg\neg a) - v(\neg a)),$$

where $v(a) \leq v(\neg\neg a)$.

The inequality follows from monotonicity applied to $a \leq \neg\neg a$, a fact of every Heyting algebra, and shows the first summand is exactly the difference, not the floor produced by $[0, \infty]$ ’s truncated subtraction. The split into a regularity defect and a De Morgan defect follows from monotonicity and modularity alone, with no further axiom about ignorance required. This contrasts with the usual Dempster–Shafer picture, where the mass assigned to non-singleton, ambiguous sets is an extra primitive.

Theorem 6 expresses slack pointwise in terms of v . The converse, whether slack in turn determines v , is more delicate. One constraint is automatic:

Theorem 7 (slack is submodular^{Lean}). *For every valuation v on a frame Ω and every $a, b \in \Omega$,*

$$\text{slack}(a \sqcup b) + \text{slack}(a \sqcap b) \leq \text{slack}(a) + \text{slack}(b).$$

The proof runs v ’s own modularity twice, once at (a, b) and once at $(\neg a, \neg b)$, and combines the two results with the De Morgan laws: the equality $\neg(a \sqcup b) = \neg a \sqcap \neg b$, which holds in every Heyting algebra, and the inequality $\neg a \sqcup \neg b \leq \neg(a \sqcap b)$, monotone under v but not in general an equality. Any function proposed as the slack of some valuation must therefore be submodular.

Remark 8 (non-recovery^{Lean}). Submodularity is necessary, not sufficient: Section 7.2 exhibits two valuations on the same frame with identical slack functions.

Remark 9 (slack under weak excluded middle^{Lean}). The two gaps, dnGap and deMorganGap , computed at a single element, are logically independent. However, if weak excluded middle is imposed on the frame for every a ($\neg\neg a \sqcup \neg a = \top^{\text{Lean}}$), then deMorganGap vanishes identically and $\text{slack}(a) = \text{dnGap}(a)$ for every valuation and every a . This sits strictly between plain intuitionistic logic, where both gaps can be positive, and full excluded middle, where both vanish^{Lean}. A concrete instance appears in Section 7.

2.4 What Survives without Excluded Middle

Three classical pillars survive intact, and one fails in a precisely delimited way. *Disjoint additivity* is unconditional: $a \sqcap b = \perp$ implies $v(a \sqcup b) = v(a) + v(b)$. *Conditioning* is native: for $v(b) \neq 0$ the posterior $v(a \mid b) = v(a \sqcap b) / v(b)$ is again a valuation (the proof is frame distributivity), and the product rule $v(a \mid b) \cdot v(b) = v(a \sqcap b)$ and Bayes symmetry (and therefore Bayesian updating) $v(a \mid b) v(b) = v(b \mid a) v(a)$ hold^{Lean}. *Total probability* holds over any genuine partition, meaning $a \sqcap a' = \perp$ and $a \sqcup a' = \top$: $v(b) = v(a \sqcap b) + v(a' \sqcap b)$. What fails is total probability over $\{a, \neg a\}$. That family need not join to $\top$, and the conditional masses fall short of $v(b)$ by a conditional slack^{Lean}.

The classical fragment. We characterize the elements of Ω on which the classical complement rule holds. The $\neg\neg$ -stable (*regular*) elements, which form a Boolean algebra (by Glivenko's theorem), are a natural candidate, but they do not suffice. Modularity constrains the frame join $\sqcup$, whereas the Boolean join of regular elements is the regularized $\neg\neg(a \sqcup b)$, for which only the inequality $v(a) + v(b) \leq v(\neg\neg(a \sqcup b)) + v(a \sqcap b)$ survives (i.e. the 2-monotonicity of Section 5). A regular but uncomplemented element, which is an open ray $(-\infty, 0)$ in the opens of $\mathbb{R}$, under an atomic measure at 0, still carries slack. The correct classical fragment is the *complemented* elements, where $v(a) + v(a^c) = 1$ holds exactly^{Lean}. On a connected locale that fragment can be as small as $\{\perp, \top\}$. In the Boolean limit, every element is complemented, the slack vanishes identically, and classical Kolmogorov axioms are recovered^{Lean}.

Mixtures. Valuations are closed under finite convex combination^{Lean}, with every axiom inherited termwise, because modularity is linear. This innocuous fact becomes a characterization in Section 4.

3 Classicality in Theory and Metatheory

Classicality enters this work twice: in the metatheory, the logic of the proof assistant itself, and in the object theory, the frame's own logic.

The metatheory. Lean's kernel is constructive, and its core library declares `Classical.choice` as an axiom, from which excluded middle follows. A theorem is constructive when its proof does not use this axiom, which `#print` axioms reports. `mathlib` uses the axiom throughout. The order on $\mathbb{R}$ and $[0, \infty]$ has no computable Decidable instance, so every library lemma that case-splits on $x \leq y$ for such numbers uses `Classical.propDecidable`, and so do the library's finite sums and finite sets. A theorem that never invokes excluded middle for frame elements $a \in \Omega$ can therefore still depend on the axiom through the library. Almost every theorem of this development does. This dependence is incidental: it comes from the library, not from the argument. Table ?? lists the classical principles the arguments themselves use, and what a constructive version would require.

The finite combinatorial results (M_3 , the pentagon, the perfect-graph criterion, the Kochen–Specker set, and the multiplicative–additive formulas) need none of these principles. The Cox model of Section 6 is given an explicit computable Decidable instance.

Object-level classicality: the R3 axiom. Van Horn's repaired Cox theorem [Horn 2003] axiomatizes plausibility over classical propositions. Its negation axiom, called *R3*, posits that the plausibility of $\neg A$ is a fixed non-increasing function S of the plausibility of A . Consistency forces the involution $S \circ S = \text{id}$, and therefore the complement rule $p \mapsto 1 - p$. We show that the involutive negation is double-negation elimination, so *R3* is where classical logic enters every known Cox-style derivation. To do this, we first isolate the exact content of *R3*'s conclusion as a predicate on valuations, then prove equivalence to excluded middle.

That predicate is *classical negation*: a valuation v has classical negation if $v(a) + v(\neg a) = 1$ for every $a \in \Omega$.

Theorem 10 (the *R3* characterization^{Lean}). *For every frame Ω , every valuation on Ω has a classical negation iff $a \sqcup \neg a = \top$ for every $a \in \Omega$.*

The forward direction is a calculation. The content is the converse: wherever excluded middle fails, a valuation with genuine slack must be *constructed*. Suppose $a \sqcup \neg a \neq \top$.

Definition 11 (ideal, prime ideal). An *ideal* $J \subseteq \Omega$ is a downward closed set which is also closed under finite joins. An ideal J is *prime* if $\top \notin J$ and $a \sqcap b \in J$ implies $a \in J$ or $b \in J$.

The prime ideal separation theorem (`mathlib's` `DistribLattice.prime_ideal_of_disjoint_filter_ideal`) yields a prime ideal J containing $a \sqcup \neg a$ but not $\top$. The indicator of J 's complement,

$$v(x) = \begin{cases} 0 & x \in J \\ 1 & x \notin J, \end{cases}$$

is a valuation. Modularity in the only nontrivial case ($x, y \notin J$) is *exactly* primeness, $x \sqcap y \in J \Rightarrow x \in J \vee y \in J$. And this valuation assigns $v(a) + v(\neg a) = 0 \neq 1$.

Sharp valuations are the points of the locale. The construction above is one half of an exact correspondence. Call v *sharp* if it is $\{0, 1\}$ -valued, a state of full certainty. Sharpness characterizes exactly the prime-ideal indicators:

Theorem 12 (sharp valuations are points^{Lean}). *For every valuation v on a frame Ω , v is sharp iff v is the complement-indicator of a prime ideal.*

The correspondence is given by two mutually inverse maps: $v \mapsto Z(v) = \{x \in \Omega : v(x) = 0\}$, which computes the zero-set of a sharp valuation, and the map taking J to its complement-indicator (the valuation of Theorem 10's construction). $Z(v)$ is prime exactly because v is sharp, and the complement-indicator of $Z(v)$ recovers v itself. In locale-theoretic terms, sharp valuations recover the *points* of the locale. A prime ideal is *completely prime* when it is also closed under arbitrary (not just finite) suprema of its members. The finitely prime points of Theorem 12 sharpen to the completely prime points exactly where Scott continuity is added, recovering points of the locale in the sense used by [Coquand and Spitters 2009; Vickers 2008].

Definition 13 (directed set, Scott continuity). A subset $D \subseteq \Omega$ is *directed* if, for any finite subset $F \subseteq D$, there exists an element $d \in D$ that is an upper bound of F (taking $F = \emptyset$ shows a directed set is nonempty). A valuation v is *Scott-continuous* if $v(\bigsqcup D) = \sup_{a \in D} v(a)$ for every directed D .

Theorem 14 (Scott-continuous sharp valuations^{Lean}). *For every valuation v on a frame Ω , v is sharp and Scott-continuous iff v is the complement-indicator of a completely prime ideal.*

The characterization and the two correspondences together locate the object-level classicality precisely: whenever excluded middle fails at some a , the failure occurs at a point of the locale.

Points and the adjunction with spaces. Write $[1] = \{0 < 1\}$, the initial frame. For a lattice P , monotone maps $P \rightarrow [1]$ correspond to upper sets (the preimage of 1), bounded lattice homomorphisms to prime filters, and frame homomorphisms to completely prime filters, those F with $\bigsqcup S \in F$ only if $S \cap F \neq \emptyset$. The complements are the lower sets, prime ideals, and completely prime ideals of Theorem 12 and the Scott-continuous characterization above. The points of a frame P , $\text{pt}(P) = \text{Frm}(P, [1])$, carry the topology of the sets $\Sigma_a = \{f \mid f(a) = 1\}$, and Σ preserves finite meets and arbitrary joins. With $\mathcal{O}$ sending a space to its frame of opens, this gives the adjunction $\mathcal{O} \dashv \text{pt}$ between spaces and locales [Johnstone 1982], in mathlib as `Locale.adjunctionTopToLocalePT`. Its units send a point x to $U \mapsto [x \in U]$ and an element a to Σ_a . A prime filter need

not be completely prime: in $[0, \infty]$, the filter $\{\infty\}$ is prime but contains $\bigsqcup_n n = \infty$ and no n . The corresponding sharp valuation is not Scott-continuous, as for the $\top$ -indicator on $\text{Low}(\mathbb{N})$ of Section 4^{Lean}.

Finite frames without choice. The prime ideal theorem is a choice principle. Without it, a frame can have no points at all [Johnstone 1982], and then it has no sharp valuations, so the converse of Theorem 10 and the statement that sharp valuations reconstruct the logic (the first criterion of Section 1) hold only in a classical metatheory. On a finite distributive lattice with decidable order the needed prime filter is found by search: an element below a and not below b that is minimal for the size of its downset is join-prime, and the indicator of its principal filter is a sharp valuation. This proves the finite separation theorem and the finite form of the converse of Theorem 10 without `Classical.choice`^{LeanLean}, for $\mathbb{N}$ -valued valuations. In a constructive metatheory the first criterion therefore holds for finite event algebras and for frames with enough points, and not in general.

A development in Rocq. Rocq's kernel, like Lean's, does not assume excluded middle or choice, so the two differ in their libraries more than in their logic. The differences would be the following.

- **Finite results.** The Mathematical Components library [Mahboubi and Tassi 2021] works with types with decidable equality and finite types, and avoids classical axioms in its algebraic core. The finite theorems of this paper (the finite representation, the frame recipe, M_3 , the pentagon, the perfect-graph criterion, and the Kochen–Specker set) would be proved there without classical axioms and could be extracted to programs.
- **Values.** $[0, \infty]$ is a classical object. A constructive development would use rational values for finite results and lower reals, approximable from below, for valuations in general [Coquand and Spitters 2009; Vickers 2008]. The value of the halting event, Chaitin's Ω , is a lower real of this kind: it is approximated from below by the halting programs found so far, and no better.
- **Analysis.** MathComp-Analysis [Affeldt et al. 2018] assumes classical axioms, as mathlib does. The completeness theorem for rational points is the Fourier–Motzkin argument above, which carries over directly. Completeness over real points would require the constructive analysis of Bishop's school, with locatedness hypotheses for hyperplane separation. The barycenter theorem for infinitely many events requires compactness of $\{0, 1\}^{\mathbb{N}}$, which constructively is the fan theorem, a principle that would have to be added as an axiom.
- **Computability.** Without excluded middle the sharp classical readout cannot be defined. The

uncomputability statement would be phrased synthetically [Forster et al. 2019]: no function from codes to Booleans decides halting, which requires an axiom such as Church's thesis to state as a negative result about all functions.

- *Points.* The separation results would be stated for frames with enough points, or for finite lattices as above.

Lean was chosen for mathlib's measure theory, order theory and topology, which the infinite results use. The cost is that constructive content is a property to be checked theorem by theorem with `#print axioms`, rather than a default.

A natural direction for future work is to develop the extension of logics to probability calculi in a constructive metatheory throughout. This would give algorithms in place of existence proofs, for instance the weights of a convex combination and the separating laws of Theorem 67, and would match the semidecidable reading of events with value types such as lower reals. It would also restrict the results that depend on points existing to frames with enough points.

4 Valuation Representation

Every valuation on a finite frame is a probability mass function on points. On infinite frames, this can fail, and Scott continuity marks the boundary between where it still holds and where it does not.

4.1 Finite Frames Representation

Theorem 15 (Birkhoff duality [Birkhoff 1937]). *Every finite distributive lattice is isomorphic to the lattice of lower sets of a finite poset: its sup-irreducible elements form a poset P whose lower sets reconstruct the lattice, and every finite poset arises this way from its own lattice of lower sets.*

Let P be the finite poset of sup-irreducible elements of a finite frame. Then, by Birkhoff duality (`OrderIso.lowerSetSupIrred`), every finite frame is the lattice of lower sets of P . We fix the following notation, used throughout the rest of the paper. A lower set $L \subseteq P$ satisfies $p \in L, q \leq p \Rightarrow q \in L$, and an upper set is defined dually. Lower sets are always denoted L , and upper sets U (opens of a topological space are still U, V, W , as in traditional usage). The principal lower set at p is $\downarrow p = \{q : q \leq p\}$, the strict principal lower set at p is $\Downarrow p = \{q : q < p\}$, and the principal upper set at p is $\uparrow p = \{q : q \geq p\}$. We prove the finite representation theorem below on the lower sets of P . Transporting it across the isomorphism above then gives the theorem for every finite frame, not just lower sets^{Lean}. Let us define the mass of a point as the discrete derivative of v^{Lean} :

$$\text{mass}(p) = v(\downarrow p) - v(\Downarrow p).$$

The masses recover v and sum to 1:

Theorem 16 (finite representation^{Lean}). *For a finite P , any valuation v on $\text{Low}(P)$, and any $L \in \text{Low}(P)$, $v(L) = \sum_{p \in L} \text{mass}(p)$, and $\sum_p \text{mass}(p) = 1$.*

The proof works by induction on $|L|$: repeatedly remove a maximal point p from L , and use modularity applied to $(L \setminus \uparrow p, \downarrow p)$ to show each removal changes v by exactly $\text{mass}(p)$. The mass function is a classical probability mass function on P^{Lean} . And since lower sets are exactly the opens of the Alexandrov topology, where interior is the identity, the theorem also reads $v = \mu \circ \text{int}$, i.e. every valuation on a finite frame is the classical measure μ applied after taking the interior. In the finite case, this is the converse of the measure-theoretic bridge of Section 7. The intuitionistic mass also agrees with the classical notion of the mass of a point: reading $v = \mu \circ \text{int}$,

$$\text{mass}(p) = \mu(\downarrow p) - \mu(\Downarrow p) = \mu(\{p\}),$$

since $\downarrow p$ is the disjoint union of $\Downarrow p$ and $\{p\}$ and μ is defined on all subsets of P .

Theorem 16 represented v as a sum of point masses. The dual view represents v as a mixture of sharp point-valuations. For each $p \in P$, let δ_p be the valuation with $\delta_p(L) = 1$ if $p \in L$, and $\delta_p(L) = 0$ otherwise:

Theorem 17 (mixture characterization^{Lean}). *On a finite frame, every valuation v satisfies $v = \sum_p \text{mass}(p) \cdot \delta_p$.*

Together with the fact that any mixture of valuations is again a valuation^{Lean}, this theorem characterizes the valuations on a finite frame exactly: they are the finite convex mixtures of point-valuations, closed under mixing because modularity is linear. Section 6 shows modularity cannot instead be derived from more primitive axioms. It must be posited outright, and doing so is exactly what makes every valuation a mixture of sharp, all-or-nothing certainties.

4.2 Infinite Boundary Representation via Scott Continuity

Both halves of the finite theorem fail for infinite frames:

Example 18 (obstruction^{Lean}). On $\text{Low}(\mathbb{N})$, the indicator v of $\top$ is a valuation with $\text{mass}(p) = 0$ for every point p , and $v(\top) = 1$.

No point carries any of the mass. The failure is one of approximation: $v(\top)$ is not the supremum of $v(\downarrow n)$ over the finite stages. The $\top$ -indicator of Example 18 is not Scott-continuous^{Lean}, and Scott continuity is exactly what restores the representation:

Example 19 (recovery^{Lean}). Let v be a valuation on $\text{Low}(\mathbb{N})$. If $v(\top) = \sup_n v(\downarrow n)$ then $v(\top) = \sum_n \text{mass}(n)$.

In general, on any poset all of whose principal lower sets are finite, Scott continuity implies zero diffuse part without additional preconditions. However, subtraction in $[0, \infty]$ is truncated at 0, so if $\sum_p \text{mass}(p)$ were greater than $v(\top)$,

the expression $v(\top) - \sum_p \text{mass}(p)$ would still evaluate to 0. Theorem 20 rules out $\sum_p \text{mass}(p) > v(\top)$ unconditionally, so the subtraction below is never truncated:

Theorem 20 (general decomposition^{Lean}). *For every valuation v on $\text{Low}(P)$ and an arbitrary poset P , $\sum_p \text{mass}(p) \leq v(\top)$.*

By Theorem 20 the remainder $v(\top) - \sum_p \text{mass}(p)$ is a genuine difference. Every valuation thus splits unconditionally into an *atomic* part carried by points ($\sum_p \text{mass}(p)$, which is always a sub-probability), and a *diffuse* (nonnegative) remainder $v(\top) - \sum_p \text{mass}(p)$. This is a constructive Lebesgue-style decomposition. The finite and Scott cases are the purely atomic extremes, and the $\top$ -indicator is the purely diffuse one. The proof of Theorem 20 re-runs the maximal-point induction on each finite subset of P , bounding its partial sum by $v(\top)$, then takes the supremum over all finite subsets to bound $\sum_p \text{mass}(p)$ itself. Its proof does not directly invoke an excluded-middle case split. However, it relies on the (classical) decidability of the ordering of the reals (a default in mathlib). Representing the diffuse part on non-spatial frames remains the open frontier. The obstruction there is again that the frame has *too few points*, the same one as behind the undecidability of Section 7.

5 The Dempster–Shafer Reading

Sections 2–4 independently arrive at the same structure as Dempster–Shafer (DS) theory [Dempster 1967; Shafer 1976], including sub-additive complements, a belief/plausibility interval, and mass functions. This section makes the identification exact in both statics and dynamics. In each case, it locates the precise point where excluded middle makes a difference.

5.1 Total Monotonicity via Constructive Inclusion–Exclusion

The $\neg\neg$ -stable (*regular*) elements of a frame form a Boolean algebra with meet $\sqcap$ and join $\neg\neg(a \sqcup b)$ (Glivenko). Let us call the resulting algebra the *Booleanization* of this set.

Definition 21 (belief function, plausibility function^{Lean}). Let B be a Boolean algebra. A DS *belief function* on B is a monotone map $\text{Bel} : B \rightarrow [0, 1]$ with $\text{Bel}(\perp) = 0$ and $\text{Bel}(\top) = 1$ that is *n-monotone* for every $n \geq 2$: for all $A_1, \dots, A_n \in B$,

$$\text{Bel}\left(\bigvee_{i=1}^n A_i\right) \geq \sum_{\emptyset \neq T \subseteq \{1, \dots, n\}} (-1)^{|T|+1} \text{Bel}\left(\bigwedge_{i \in T} A_i\right).$$

Its *plausibility function* is the map $\text{Pl} : B \rightarrow [0, 1]$ given by $\text{Pl}(A) = 1 - \text{Bel}(\neg A)$.

We show v , restricted to its Booleanization, is such a belief function. The 2-monotone case is immediate from modularity^{Lean}. The full tower follows from the following result:

Theorem 22 (inclusion–exclusion^{Lean}). *For every valuation on every frame Ω , every finite family $a : \iota \rightarrow \Omega$, and every finite $s \subseteq \iota$, inclusion–exclusion holds with equality for the frame join:*

$$v\left(\bigsqcup_{i \in s} a_i\right) = \sum_{\emptyset \neq T \subseteq s} (-1)^{|T|+1} v\left(\bigsqcap_{i \in T} a_i\right).$$

We state this in Lean additively, with the odd-cardinality half of the signed sum set against the even-cardinality half, so that no $[0, \infty]$ subtraction occurs. The proof is by induction with modularity and frame distributivity.

Replacing the frame join $\bigsqcup_i a_i$ on the equality’s other side with its double negation $\neg\neg(\bigsqcup_i a_i)$, the Booleanization’s own join, yields the following inequality:

Theorem 23 (total monotonicity^{Lean}). *For every valuation on every frame Ω , every finite family $a : \iota \rightarrow \Omega$, and every finite $s \subseteq \iota$:*

$$\sum_{\emptyset \neq T \subseteq s} (-1)^{|T|+1} v\left(\bigsqcap_{i \in T} a_i\right) \leq v\left(\neg\neg\left(\bigsqcup_{i \in s} a_i\right)\right).$$

The Booleanization’s join is given by $\neg\neg \circ \sqcup$, and its meets of regular elements are again regular. Interpreted in this way, Theorem 23 states exactly the ∞ -monotone tower: v restricted to its Booleanization is a belief function.

The proof of Theorem 23 from Theorem 22 follows from the monotonicity along $a \leq \neg\neg a$. Here, inclusion–exclusion does *not* fail constructively. It is exact for the intuitionistic disjunction. Applying the double-negation nucleus (a closure operator preserving finite meets) $\neg\neg$ to the join on the equality’s right-hand side can only enlarge it. This follows from the fact that $a \leq \neg\neg a$ and that v is monotone, which turns the equality into the inequality of Theorem 23. The amount by which the right-hand side is enlarged is exactly the double-negation gap of Section 2.

Definition 21 is the classical notion, stated on an abstract Boolean algebra. In our setting each of its ingredients is already present: the Boolean algebra is the Booleanization, the belief function is v itself, restricted to the regular elements and evaluated against the Boolean join $\neg\neg \circ \sqcup$, and the plausibility function is $\text{Pl}(a) = 1 - v(\neg a)$. Additive probability does not become a belief function by weakening an axiom, since modularity for v is never relaxed. What changes is only the join used to state the tower. The belief–plausibility interval is governed by the slack:

Theorem 24 (interval width^{Lean}). *For every valuation v on a frame Ω and every $a \in \Omega$, $\text{Pl}(a) - v(a) = \text{slack}(a)$.*

5.2 The Conditioning Characterization

DS theory carries two update rules that coincide classically. *Geometric* conditioning renormalizes belief, and under the DS dictionary it is the localic posterior $v(a \sqcap b)/v(b)$ of Section 2.

Definition 25 (Dempster’s rule of conditioning^{Lean}). For a valuation v on a frame Ω , elements $a, b \in \Omega$, and evidence of nonzero plausibility, $v(b) \neq 0$, *Dempster’s rule of conditioning* [Shafer 1976] states that

$$v(a \parallel b) = \frac{v(a \sqcup \neg b) - v(\neg b)}{1 - v(\neg b)}.$$

The hypothesis $v(b) \neq 0$ makes the denominator nonzero, since $v(b) + v(\neg b) \leq 1$ gives $v(b) \leq 1 - v(\neg b)$.

That is, the DS conditioning rule discards the mass incompatible with the evidence and renormalizes by its plausibility. Both the geometric posterior of Section 2 and Dempster’s rule of Definition 25 are again valuations. Their modularity is the frame distributivity $(a \sqcup \neg b) \sqcap (a' \sqcup \neg b) = (a \sqcap a') \sqcup \neg b$. Their disagreement is given by the *excluded-middle gap* $\text{emGap}(a, b) = v(a) - v(a \sqcap (b \sqcup \neg b))$, i.e. the mass of a outside b ’s instance of excluded middle:

Lemma 26 (the join–complement identity^{Lean}). *For every valuation v on every frame Ω and every $a, b \in \Omega$, $v(a \sqcup \neg b) = v(\neg b) + v(a \sqcap b) + \text{emGap}(a, b)$.*

Dempster’s numerator therefore exceeds Bayes’s by exactly $\text{emGap}(a, b)$. This gap depends on a , but whether it vanishes at every a depends only on the slack at the evidence b :

Theorem 27 (the conditioning characterization^{Lean}). *For every valuation v on every frame Ω and every $b \in \Omega$ with $v(b) \neq 0$, $(v(a \parallel b) = v(a \sqcap b)/v(b) \text{ for every } a \in \Omega) \text{ iff } \text{slack}(b) = 0$.*

The hypothesis $\text{slack}(b) = 0$ eliminates the gap $\text{emGap}(a, b)$ for every a and makes the two normalizers coincide, since $1 - v(\neg b) = v(b)$. Conversely, testing at $a := b$, geometric conditioning gives certainty 1, while Dempster gives $v(b)/(1 - v(\neg b))$, which is 1 only if the slack vanishes. Theorem 27 is the valuation-relative analogue of Theorem 10: both are iff statements about the same proposition, $b \sqcup \neg b = \top$.

We call the frame-level fact *static* and the valuation-level fact *dynamic*: the former is an equation in the frame itself, between $b \sqcup \neg b$ and $\top$, with no valuation involved, while the latter tests the same proposition through v , since $\text{slack}(b) = 0$ is exactly $v(b \sqcup \neg b) = v(\top)$. Dempster and Bayes therefore coincide at b whenever v cannot distinguish $b \sqcup \neg b$ from $\top$, even if the two differ in the frame itself. In the measure model, Dempster and Bayes disagree precisely when the evidence’s boundary carries mass.

6 Cox’s Theorem

Cox’s theorem [Cox 1946; Jaynes 2003] derives the probability calculus from conditions on plausibility. After Halpern’s counterexample [Halpern 1999], its repaired forms [Arnborg and Sjödin 2000; Horn 2003] fix the calculus up to monotone

regratuation. Every known derivation is classical, and Section 3 showed why: the negation axiom R3 is excluded middle. Dropping R3 leaves the product rule unaffected, since it is logic-free, and requires the sum rule to become an axiom rather than a theorem. Separately, the naive constructive statement, demanding a regratuation strictly monotone on all of $\mathbb{R}$, is unsatisfiable (Section 6.3).

6.1 A Cox Structure without Negation

Definition 28 (Cox model^{Lean}). A *Cox model* on a frame Ω is a plausibility assignment $\text{pl} : \Omega \times \Omega \rightarrow \mathbb{R}$, written $\text{pl}(a, b)$ for the plausibility of a given b , together with a *conjunction functional* $F : \mathbb{R} \times \mathbb{R} \rightarrow \mathbb{R}$, such that $\text{pl}(\cdot, c)$ is monotone for every c , the boundary conditions $\text{pl}(\perp, c) = 0$ and $\text{pl}(\top, c) = 1$ hold for every c , the product rule $\text{pl}(a \sqcap b, c) = F(\text{pl}(a, b \sqcap c), \text{pl}(b, c))$ holds, F is associative, and $x \mapsto F(x, y)$ is strictly monotone for every $y > 0$.

The missing R3 axiom can be observed in the omission of any constraint on the relation between $\text{pl}(\neg a, b)$ and $\text{pl}(a, b)$. The definition restricts the monotonicity of F to $0 < y$ to avoid a contradiction with the fact that the boundary axioms force $F(x, 0) = 0$ for all x . An explicit model on the non-classical chain $[0, \infty]$ satisfies this constraint^{Lean}, as $a = \top$ is provably decidable.

6.2 The Sum Rule Is Irreducible

Classically, additivity is *derived*: R3 turns disjunction into negated conjunction via De Morgan, and the product rule does the rest. Constructively that route is closed, since $a \sqcup b \neq \neg(\neg a \sqcap \neg b)$ in general:

Example 29 (the sum rule is irreducible^{Lean}). Let Ω be the five-element frame of lower sets of the poset $o < a, o < b$ with a, b incomparable. Then there is a monotone, normalized assignment q on Ω that is additive on disjoint joins yet not modular, and no binary operation S on values satisfies $q(x \sqcup y) = S(q(x), q(y))$ for all x, y . Moreover, the pairs $(\downarrow a, \downarrow a)$ and $(\downarrow a, \downarrow b)$ have identical value-pairs $(\frac{1}{2}, \frac{1}{2})$ but joins valued $\frac{1}{2}$ and 1.

Unlike for a conjunction, a disjunction’s plausibility cannot generally be expressed as its disjuncts’ plausibilities via conditioning. So, a constructive Cox theorem *must* posit the sum-rule half. Modularity, which says that the calculus is closed under mixing certainties, is what posits this, justified by Theorem 17. The failure is not repaired by strengthening the logic short of Boolean: the counterexample frame satisfies weak excluded middle^{Lean}, since every nonempty lower set contains the bottom point o , so the irreducibility persists under that intermediate assumption.

6.3 Cox Model Theorem

A regratuation is a map $g : \mathbb{R} \rightarrow [0, \infty]$ with $g(0) = 0$. No such map is strictly monotone on all of $\mathbb{R}$, since strictness

would force $g(-1) < g(0) = 0$, impossible in $[0, \infty]$. The repair (strictness only on $[0, 1]$, where plausibilities live) is exactly the same domain restriction that Halpern's critique of the classical theorem turns on. The corrected statement is:

Theorem 30 (the constructive Cox theorem^{Lean}). *For every frame Ω , every Cox model on Ω whose unconditional plausibility is modular regraduates to a valuation: there exist a valuation v on Ω and a strictly monotone function g on $[0, 1]$ with $g(0) = 0$, $g(1) = 1$, and for all $a \in \Omega$, $v(a) = g(\text{pl}(a, \top))$.*

The regraduation for the sum-rule half is straightforward ($g = \text{ENNReal.ofReal}$). The product-rule half, showing F regraduates to multiplication, rests on Aczél's associativity theorem [Aczél 1966], a statement of pure analysis whose type never mentions Ω . `mathlib` has no Aczél theorem, no Hölder-style embedding for ordered semigroups on intervals, and no Cauchy functional equation under monotonicity, so we build the analytic core from scratch, obtaining a normalized, strictly monotone multiplicative generator on the positive cone^{Lean}. The construction also weakens Hölder's classical hypotheses [Hölder 1901]: the Archimedean property is *derived* from continuity and positivity rather than assumed, and the interval form of divisibility follows from the intermediate value theorem^{Lean} applied to a plain divisibility hypothesis^{Lean}. The constructive/classical divide is therefore *entirely* in the sum rule: nothing in the product-rule apparatus mentions Ω . Appendix C gives the construction. The residual analytic gap is discussed in Section 14.

7 Two Grounding Models

Two concrete models demonstrate that our abstract theory is non-degenerate. One shows every classical measure *contains* an intuitionistic probability. The other shows the intuitionistic theory cannot collapse back without solving the halting problem.

7.1 Measures through the Interior Operator

Gödel–McKinsey–Tarski interpret intuitionistic logic in modal S4: propositions are *open* sets, and the modal necessity operator $\Box$ of S4 is interpreted as the topological interior, written $\text{int}(\cdot)$ below [Gödel 1933; McKinsey and Tarski 1944]. Our valuations quantify that same translation:

Theorem 31 (measures as valuations^{Lean}). *Let μ be a probability measure on a space X , $A \subseteq X$, and define $v(A) = \mu(\text{int } A)$. Since $\text{int } U = U$ for open U , v restricts on the frame $O(X)$ to $U \mapsto \mu(U)$, and this restriction is a valuation.*

Modularity is $\mu(U \cup V) + \mu(U \cap V) = \mu(U) + \mu(V)$. The slack of an open U is then $1 - \mu(U) - \mu(\text{int } U^c)$, which is the measure of the boundary ∂U . We can now prove that, in general, slack is exactly the boundary mass:

Theorem 32 (slack is boundary mass^{Lean}). *For any probability measure μ on a topological space X and any open U , $\text{slack}(U) = \mu(\partial U)$.*

The Heyting complement of U in $O(X)$ is the interior of its set complement. So, the undecided region, the complement of $U \sqcup \neg U$, is exactly $\overline{U} \setminus U = \partial U$. The general sub-additivity inequality of Section 2 still holds here^{Lean}. This identification of slack with boundary mass also exhibits the modal identification $\text{Bel} = P(\Box \cdot)$ of the DS literature [Ruspini 1986] arising from the logic itself. Theorem 16 is the converse in the finite case (in the general case, the converse is the open problem, see Section 4).

The same decomposition locates a failure of classical probability calculus in conditioning, that is, in updating a belief about an open A on evidence given by an open B . The Boolean complement of A splits into its Heyting complement and its boundary, $A^c = \text{int } A^c \sqcup \partial A$, so a classical posterior that treats A as confirmed-or-refuted overstates the refutation by the boundary mass the evidence retains:

Theorem 33 (classical overconfidence in refutation^{Lean}). *Let μ be a probability measure on a topological space X , let v be its restriction to $O(X)$, and let A, B be opens with $\mu(B) \neq 0$. Then*

$$\frac{\mu(A \cap B)}{\mu(B)} = v(A \mid B) \quad \text{and} \quad \frac{\mu(A^c \cap B)}{\mu(B)} = v(\neg A \mid B) + \frac{\mu(\partial A \cap B)}{\mu(B)}.$$

The excess term is the conditional slack of A given B , the mass that the evidence leaves neither confirmed nor refuted. The classical posterior does not discard this mass. It credits all of it to A^c , an event no finite observation confirms. For the halting event of Section 7.3 with $B = \top$, the classical credence in non-termination is $1 - p$ while the constructive one is 0, and the difference $1 - p$ is exactly the slack.

To characterize weak excluded middle spaces, we give the (standard) definitions required:

Definition 34 (extremally disconnected). A topological space X is *extremally disconnected* if the closure of every open set is again open.

Definition 35 (Stone space, Stonean space). The *Stone space* of a Boolean algebra B is the space of ultrafilters on B , topologized by the clopen basis $\{U : b \in U\}$ for $b \in B$. *Stone duality* [Johnstone 1982] identifies Boolean algebras with exactly the compact, Hausdorff, totally disconnected spaces arising this way. A *Stonean space* is a compact Hausdorff space that is also extremally disconnected. Equivalently, it is the Stone space of a *complete* Boolean algebra.

Remark 36 (weak excluded middle on opens^{Lean}). Section 2 noted that imposing weak excluded middle on the whole frame collapses slack to pure double-negation instability. For $\Omega = O(X)$, this global condition holds exactly when X is extremally disconnected.

An immediate corollary gives the characterization:

Corollary 37 (Stonean characterization^{Lean}). *Given a compact Hausdorff space X , weak excluded middle on $O(X)$ holds exactly when X is a Stonean space.*

7.2 Slack Does Not Determine the Valuation

Submodularity (Theorem 7) is a necessary condition on slack. It is not sufficient: distinct valuations, each satisfying every axiom a valuation must satisfy, can share a slack function.

Example 38 (boundaries in the three-point space^{Lean}). Let $\text{ThreePt} = \{p_A, p_B, p_Z\}$ carry the topology generated by the two singletons $\{p_A\}$ and $\{p_B\}$, so its opens are exactly $\emptyset$, $\{p_A\}$, $\{p_B\}$, $\{p_A, p_B\}$, and the whole space. For every open U , $\partial U \subseteq \{p_Z\}$.

This is a fact about the topology alone, independent of any measure: p_Z has no proper open neighbourhood, so it is the only point any open's frontier can contain.

Example 39 (non-recovery^{Lean}). There exist valuations $v \neq v'$ on $O(\text{ThreePt})$ whose slack functions are identical: $\text{slack}_v = \text{slack}_{v'}$ as functions.

For weights $a, b, c \geq 0$ with $a + b + c = 1$, the point measure $\mu_{a,b,c} = a \delta_{p_A} + b \delta_{p_B} + c \delta_{p_Z}$, read through Section 7.1's bridge, gives a valuation $v_{a,b,c}$ on $O(\text{ThreePt})$. By Example 38 and Theorem 32, its slack at an open U is $\mu_{a,b,c}(\partial U)$, equal to c when $p_Z \in \partial U$ and to 0 otherwise: a quantity depending only on c , not on how the remaining mass splits between a and b . So $v_{1/4,1/4,1/2}$ and $v_{1/8,3/8,1/2}$ have identical slack functions while disagreeing at $\{p_A\}$, where they take the values $1/4$ and $1/8$. Both are fully modular, monotone, and normalized: nothing in Theorem 7 or its proof distinguishes them.

Weak excluded middle does not rescue recoverability. If anything, the fully classical case is the worst one for it. When X is extremally disconnected, the De Morgan gap vanishes (Section 7.1) and slack degenerates to the double-negation gap alone, but on a space where excluded middle holds outright that gap vanishes too, so slack is identically 0 for every valuation. The zero slack function is then shared by every probability measure on X , which is the most extreme failure of recovery available.

7.3 The Computability Guard

The theory developed here is non-degenerate only if there are spaces whose slack does not vanish identically. Computability supplies them:

Definition 40 (Sierpiński topology, semidecidable proposition). The *Sierpiński space* $\mathbb{S}$ is the type Prop of propositions carrying the topology with exactly three opens: $\emptyset$, the set $\text{halts} = \{q \mid q\}$ of true propositions, and $\mathbb{S}$ itself [Escardó 2004; Rosolini 1986]. A proposition is *semidecidable* if it is an open in this topology.

The topology encodes verifiability. A point of $\mathbb{S}$ is a proposition, say “this computation halts.” The one nontrivial observation available is membership in halts , established by

producing a proof (running the computation to termination). No open captures the complementary observation, and on the frame $O(\mathbb{S}) = \{\perp, \text{halts}, \top\}$ that Σ_1 asymmetry is a theorem:

Theorem 41 (the Σ_1 asymmetry^{Lean}). $\neg \text{halts} = \perp$.

Every nonempty open of $\mathbb{S}$ contains True , so $\perp$ is the only open disjoint from halts . Thus, no nonempty open of $\mathbb{S}$ is evidence that the computation does not halt.

We now instantiate the measure-to-valuation construction of Theorem 31 at $X = \mathbb{S}$, allowing us to compute the slack:

Example 42 (halting slack^{Lean}). Let $\mu = p \cdot \delta_{\text{True}} + (1-p) \cdot \delta_{\text{False}}$, and let v be the resulting valuation. Then

$$\text{slack}(\text{halts}) = 1 - p = \mu(\partial \{q \mid q\}) = \mu\{\text{False}\}.$$

Since halts is already open, $v(\text{halts}) = \mu(\text{halts}) = p$. The DS “ignorance” of Section 5 is here the mass of the single point False , the one outcome no finite observation can ever resolve. This probability is a fact about the model, not about anyone's ignorance.

Chaitin's Ω . The same bridge scales to algorithmic information theory. Fix a universal prefix-free machine U , *prefix-free* in that no halting program of U is a proper prefix of another, and *universal* in that it simulates every other prefix-free machine. Give 2^ω , the space of infinite bit strings, the fair-coin measure μ . The event H of having a halting program of U as a prefix is open, since halting is established by a finite prefix (Definition 40), and prefix-freeness makes H a disjoint union of cylinders, so $v(H) = \mu(H) = \sum_{p \text{ halts}} 2^{-|p|}$: exactly Chaitin's halting probability Ω [Chaitin 1975]. Its slack $\mu(\partial H)$ is the mass of strings approximated arbitrarily well by halting strings while never having a halting program as a prefix, and Ω 's classical uncomputability is the aggregate form of the pointwise obstruction above. The measure-theoretic core is mechanized on Cantor space with mathlib 's infinite product measure: basic cylinders are clopen with measure $2^{-|w|}$ ^{Lean}, and a *finite* prefix-free set of words has pairwise disjoint cylinders of total measure $\sum_w 2^{-|w|}$ ^{Lean}, the Kraft-inequality content underlying Ω . The universal machine itself, and the countable extension of the finite Kraft sum, are computability theory beyond our scope.

Definition 43 (halting problem). For a code c and input n , the *halting problem* is the predicate $H(c, n) = \text{“the machine with code } c \text{ halts on input } n\text{”}$ (ComputablePred).

Slack cannot be eliminated by simply declaring a classical answer for each code. The family of valuations defined below attempts to do this:

Definition 44 (sharp classical readout^{Lean}). For a given n , the *sharp classical readout* is the family of $\{0, 1\}$ -valued valuations $\rho_n(c) \in \text{Valuation}(O(\mathbb{S}))$, with $\rho_n(c)(\text{halts}) = 1$ if $H(c, n)$ holds, and $\rho_n(c)(\text{halts}) = 0$ otherwise.

The readout is sharp but not slack-free. Since $\neg \text{halts} = \perp$ (Theorem 41), a code that does not halt receives slack $1-0 = 1$, the maximum. Sharp here means that the classical answer is known, not that the complement rule holds on $\mathcal{O}(\mathbb{S})$.

However, this family is provably not computable:

Theorem 45 (the sharp readout is uncomputable^{Lean}). *For every n , the predicate $c \mapsto (\rho_n(c)(\text{halts}) = 1)$ is not computable.*

The proof identifies the predicate with $H(\cdot, n)$ and applies the undecidability of the halting problem (ComputablePred.halting_problem).

No route back to a classical, decidable treatment survives contact with computability. Excluded middle guarantees that $H(c, n)$ is true or false for any fixed c , but that guarantee is free: it names no algorithm and exhibits no witness for which disjunct holds. Demanding an actual witness, uniformly in c , is exactly ρ_n 's sharp readout, and Theorem 45 shows no algorithm computes it. Aggregating over the whole family of halting programs, rather than reading off one instance, does not help either: Ω is a perfectly well-defined classical probability, yet Chaitin showed it uncomputable by an independent argument turning on algorithmic randomness [Chaitin 1975]. Neither the pointwise nor the aggregate route gives classical probability a way to compute a solution to the halting problem, and each is blocked by its own uncomputability result.

We can conclude that slack is not a defect of the constructive theory. On $\mathcal{O}(\mathbb{S})$, forcing the complement rule at halts leaves the single value $v(\text{halts}) = 1$, so every code is assigned certain termination. That assignment is computable but false of every non-halting code. Any assignment that avoids this error, giving $v(\text{halts}) < 1$ to each non-halting code, has positive slack at every such code, and the one that is moreover sharp is the uncomputable readout of Theorem 45. For the sum rule, this guard plays the role the explicit Cox-model instance plays for the product rule (Section 6): each axiom set in the paper comes with non-trivial models satisfying it.

8 Product Valuations and Kernel Composition

The structure through which classical probability supports sequential statistical reasoning is the Giry monad:

Definition 46 (Giry monad [Giry 1982]). The *Giry monad* G sends a measurable space X to the space $G(X)$ of probability measures on X . Its unit sends a point $x \in X$ to the point mass at x , and its multiplication sends a measure M on $G(X)$ to the measure $A \mapsto \int m(A) dM(m)$.

Probabilistic reasoning then composes in stages, and, when the monad is commutative, independent stages compose in either order. The question is whether the intuitionistic calculus supports the same structure.

A *frame homomorphism* is a map preserving finite meets and arbitrary joins. Pushforward along a frame homomorphism $f : \Omega \rightarrow \Omega'$ sends a valuation v on Ω' to $a \mapsto v(f(a))$ on Ω , and this preserves the valuation axioms, making $\text{Val}(-)$ a functor on locales, the localic counterpart of the functor part of the Giry monad [Giry 1982]. A frame homomorphism is the opposite of a locale map, so the functor laws are checked against identity and composition of frame homomorphisms^{Lean}.

Definition 47 (valued frames^{Lean}). The *valuation functor* $\text{Val} : \text{Frm}^{\text{op}} \rightarrow \text{Set}$ sends a frame to its set of valuations and a frame homomorphism $\varphi : P \rightarrow Q$ to $w \mapsto w \circ \varphi$. The category SpVal of *valued frames* is its category of elements: objects are pairs (P, v) with v a valuation on P , and a morphism $(P, v) \rightarrow (Q, w)$ is a frame homomorphism $\varphi : P \rightarrow Q$ with $w \circ \varphi = v$.

A morphism of valued frames is a measure-preserving map of locales in the opposite direction. The point valuations δ_p (Section 4) provide the candidate (monad) unit. Theorem 16 shows that, on finite frames, Val agrees (on objects) with the classical finite-distribution monad on points. Flattening a valuation of valuations is integration, and integration in two variables requires product valuations. A *kernel* is a map assigning a valuation to each point of a locale, giving a conditional distribution. By Kock's identification of commutative monads with Fubini theorems [Kock 2012], kernels compose independently of order exactly when Val is a commutative monad.

A valuation u on a product frame is a *product* of valuations v and w when $u(A \times B) = v(A) \cdot w(B)$ for all rectangles $A \times B$. Let $\text{Low}(\mathbb{N} \times \mathbb{N})$ be the frame of lower sets of the grid $\mathbb{N} \times \mathbb{N}$, ordered component-wise. On this frame, products are unique:

Theorem 48 (product uniqueness^{Lean}). *Let v, w be valuations on $\text{Low}(\mathbb{N})$. Any two valuations on $\text{Low}(\mathbb{N} \times \mathbb{N})$ that are products of v and w are equal.*

By the inclusion-exclusion equality, two valuations agreeing on a $\sqcap$ -closed family agree on all finite joins of its members^{Lean}. The rectangles are closed under $\sqcap$, and in $\text{Low}(\mathbb{N} \times \mathbb{N})$ every lower set is a *finite* union of rectangles^{Lean}, so agreement on rectangles is agreement everywhere. Valuations carry no continuity axiom (Section 4), so the σ -additivity driving the classical uniqueness argument for product measures has no analogue here. The order structure of the frame does that work instead.

Theorem 49 (product non-uniqueness^{Lean}). *On the powerset frame $\text{Set}(\mathbb{N} \times \mathbb{N})$ there exist valuations v, w and two distinct products of v and w .*

An *ultrafilter* on $\mathbb{N}$ is a collection of subsets, closed under intersection and superset, containing exactly one of S and its complement for every $S \subseteq \mathbb{N}$. It is *free* when it contains no finite set. The map assigning 1 to the members of an ultrafilter and 0 to every other subset is a valuation^{Lean}. A rectangle

$S \times T$ belongs to an ultrafilter on the product exactly when S and T belong to its images under the two coordinate projections^{Lean}. A free ultrafilter $\mathcal{U}$ on $\mathbb{N}$ yields two ultrafilters on $\mathbb{N} \times \mathbb{N}$. A set belongs to the former when, for $\mathcal{U}$ -most first coordinates, its slice contains $\mathcal{U}$ -most second coordinates. A set belongs to the latter under the same test with the coordinates in the opposite order.

The two ultrafilters' valuations agree on *every* rectangle yet differ on the upper triangle $\{(m, m') \mid m < m'\}$ ^{Lean}. Integrating the same marginal data in the two orders thus disagrees. Fubini fails for valuations, as it does for non-additive integration [Denneberg 1994; Ghirardato 1997]. By Kock's correspondence, no canonical commutative multiplication exists for unrestricted valuations. This half of the dichotomy is not specific to intuitionism, since powerset-frame valuations are exactly the classical finitely additive probability measures [Rao and Rao 1983]. Dropping continuity is what admits the non-uniqueness. A monad therefore requires restricting either the frames or the valuations.

A first step in the second direction is to restrict Val to the valuations given as countable mixtures. For these, we can demonstrate that the kernels compose independently of order, the property that Theorem 49 refutes for unrestricted valuations:

Proposition 50 (countable kernel composition^{Lean}). 1. *A countable mixture of valuations is a valuation^{Lean}.*
 2. *A mixture of mixtures flattens to a mixture with product weights, which is discrete Chapman–Kolmogorov^{Lean}.*
 3. *The one-point mixture is the identity^{Lean}.*
 4. *Iterated countable mixing is order-independent^{Lean}.*

Restricted to countable-mixture presentations, the data of a monad are all in place: the category of locales, Val restricted to countable mixtures as the functor, the point valuations as the unit, and countable mixing as the multiplication. Proposition 50 verifies exactly the laws this needs: item 1 keeps the multiplication inside the fragment, item 2 is its associativity, item 3 its unit law, and item 4 its commutativity. What is not yet settled is whether this descends to valuations themselves, since a valuation can admit more than one countable-mixture presentation, and the diffuse part of Section 4 is not captured by any countable presentation at all.

9 Valuations Beyond Frames

Definition 1 fixes the event algebra to be a frame: a complete lattice in which finite meets distribute over arbitrary joins $\bigsqcup_{i \in I} a_i$, the operation Heyting implication $\Rightarrow$ (the right adjoint to $a \sqcap (-)$) is built from. Definition 2's four axioms, normalization at $\perp$ and $\top$, monotonicity, and modularity $v(a) + v(b) = v(a \sqcup b) + v(a \sqcap b)$, use only the *binary* join $\sqcup$; none of the four mentions the *arbitrary* join $\bigsqcup_{i \in I}$. Completeness is used only where later sections evaluate a valuation on an infinite join, as in the Scott-continuity results of Section 4, or build $\Rightarrow$ and $\neg$ from it, as in Section 3's slack.

Section 14 leaves open how much of the construction is tied to frames specifically, and whether some ambient category sends an event algebra to a probability calculus in general. This section settles both questions for one fragment of the construction, the sharp/point correspondence of Section 3: it isolates the exact algebraic hypothesis that correspondence needs and shows the hypothesis cannot be dropped. It does not recover $\Rightarrow$ or $\neg$ without completeness: the adjoint defining $\Rightarrow$ is built from an arbitrary join of witnesses, so it is unavailable once completeness goes, even if only finite joins are missing. The valuations studied below are negation-free by construction, not frame valuations with completeness quietly removed.

The functor without completeness. A *bounded lattice* is a lattice with a least element $\perp$ and a greatest element $\top$; a *bounded lattice homomorphism* preserves $\perp$, $\top$, $\sqcup$, and $\sqcap$. Definition 2's axioms make sense over any bounded lattice Ω , complete or not:

Definition 51 (lattice valuation^{Lean}). A *lattice valuation* on a bounded lattice Ω is a monotone function $v : \Omega \rightarrow [0, \infty]$ with $v(\perp) = 0$, $v(\top) = 1$, and, for all $a, b \in \Omega$, $v(a) + v(b) = v(a \sqcup b) + v(a \sqcap b)$.

```
structure LValuation (Ω : Type*) [Lattice
  Ω] [BoundedOrder Ω] where
  toFun : Ω → ℝ≥0∞
  map_bot' : toFun ⊥ = 0
  map_top' : toFun ⊤ = 1
  mono' : Monotone toFun
  modular' : ∀ a b, toFun a + toFun b
    = toFun (a ⊔ b) + toFun (a ⊓ b)
```

A lattice valuation on a frame is exactly a valuation in the sense of Definition 2; the two notions differ only in how much structure the ambient Ω carries.

Section 8 pulls a valuation back along a frame homomorphism and checks the assignment is functorial (identity, composition). That argument pattern-matches on $\sqcup$, $\sqcap$, $\perp$, $\top$ alone and never touches $\bigsqcup$:

Proposition 52 (pullback without completeness^{Lean}). *For bounded lattices Γ, Ω , a bounded lattice homomorphism $f : \Gamma \rightarrow \Omega$, and a lattice valuation v on Ω , $v \circ f$ is a lattice valuation on Γ . This assignment sends the identity homomorphism to the identity and composition of homomorphisms to composition of the induced maps.*

Val($-$) is therefore a functor on bounded lattices with bounded lattice homomorphisms, of which the frame-level functor of Section 8 is the restriction to the subcategory of frames and frame homomorphisms. Completeness contributes nothing to functoriality.

The valuation monoid. The modularity axiom can be moved from the valuation into its domain.

Definition 53 (valuation monoid^{Lean}). For a lattice P with least element $\perp$, $V(P)$ is the free commutative monoid on P modulo the congruence generated by $[a] + [b] \sim [a \sqcup b] + [a \sqcap b]$ for all a, b and $[\perp] \sim 0$, where $[a]$ is the class of the generator a .

Theorem 54 (universal property^{Lean}). For every commutative monoid M , the maps $f : P \rightarrow M$ with $f(\perp) = 0$ and $f(a) + f(b) = f(a \sqcup b) + f(a \sqcap b)$ are exactly the composites of $a \mapsto [a]$ with monoid homomorphisms $V(P) \rightarrow M$, and the homomorphism is unique. On a bounded lattice, the lattice valuations are therefore the monoid homomorphisms $h : V(P) \rightarrow [0, \infty]$ with $h([\top]) = 1$ and $a \mapsto h([a])$ monotone.

So Val factors through commutative monoids, and the remaining conditions, normalization and monotonicity, are conditions on the homomorphism.

Theorem 55 (flags^{Lean}). In every lattice with a least element, every element of $V(P)$ equals $[\alpha_1] + \dots + [\alpha_n]$ for some chain $\alpha_1 \leq \dots \leq \alpha_n$.

The proof inserts one generator at a time into a chain $c_1 \leq \dots \leq c_n$: $[x] + [c_1] = [x \sqcap c_1] + [x \sqcup c_1]$, and $x \sqcup c_1$ is inserted into $c_2 \leq \dots \leq c_n$. Every common lower bound of x and the chain lies below every element of the result, which keeps the result a chain. Distributivity is not used. On a finite distributive lattice $\text{Low}(Q)$ the chain is unique up to copies of $\perp$, and $V(P)$ is cancellative: evaluating against the point valuations δ_q gives the number $k(q)$ of elements of the chain containing q , the j -th largest element of the chain is $\{q \mid k(q) \geq j\}$, and $x \mapsto k$ is an injective homomorphism into $\mathbb{N}^Q$. This argument is not formalized. For general lattices both questions are open.

Separation needs only distributivity. Theorem 12 characterizes what a sharp valuation is, a complement-indicator of a prime ideal, but not whether there are enough of them to distinguish elements of Ω . That is a separate fact:

Theorem 56 (sharp valuations separate points^{Lean}). For every frame Ω and $a, b \in \Omega$ with $a \not\leq b$, there is a sharp valuation v with $v(a) = 1$ and $v(b) = 0$.

The proof runs the same prime-ideal separation theorem that built the counterexample of Theorem 10 (`DistribLattice.prime_ideal_of_disjoint_filter_ideal`), on the principal filter of a and the principal ideal of b rather than on $a \sqcup \neg a$ and $\top$. As a corollary, any two distinct $a, b \in \Omega$ are distinguished by some sharp valuation: Ω need not be linear, so $a \neq b$ alone does not hand back $a \leq b$ or $b \leq a$, but it forces at least one failure of $\leq$, and Theorem 56 applies in whichever direction fails. This is the cash-out of the first qualifying condition of Section 14: the certain

valuations do not merely characterize a single point of the locale in isolation, together they reconstruct the order on Ω . Reconstructing the order is not the same as reconstructing the full Heyting structure: $\neg$ and $\Rightarrow$, as just noted, are adjoints built from an arbitrary join and carry no meaning over a plain distributive lattice. What this theorem recovers is the order-theoretic core of the first qualifying condition, not the negation fragment of intuitionistic logic, which stays frame-specific.

Tracing the proof of Theorem 56 shows it never invokes `⊔`; the only place frame structure enters is `DistribLattice.prime_ideal_of_disjoint_filter_ideal` itself, which needs distributivity and nothing beyond it:

Theorem 57 (separation over distributive lattices^{Lean}). For every distributive bounded lattice Ω and $a, b \in \Omega$ with $a \not\leq b$, there is a sharp lattice valuation v on Ω with $v(a) = 1$ and $v(b) = 0$.

Theorem 57 reproves Theorem 56 with the hypothesis `Order.Frame` weakened to `DistribLattice`, and nothing else about the argument changes. The condition doing the work in the sharp/point correspondence is distributivity; completeness is not load-bearing for either half of it.

Distributivity is not a proof artifact. The natural question is whether Theorem 57's hypothesis is a genuine dividing line or an artifact of routing the argument through a mathlib lemma named for distributive lattices. It is genuine, and the failure it guards against is total, not merely a failure of separation:

Definition 58 (the diamond^{Lean}). M_3 is the five-element lattice with a bottom z , a top o , and three pairwise-incomparable atoms a, b, c , with $x \sqcup y = o$ and $x \sqcap y = z$ for every two distinct atoms x, y .

M_3 is the smallest lattice that is not distributive: $a \sqcap (b \sqcup c) = a \sqcap o = a$, while $(a \sqcap b) \sqcup (a \sqcap c) = z \sqcup z = z$. It is modular, so this failure is specific to distributivity rather than some coarser defect of the order.

Theorem 59 (no sharp valuation on the diamond^{Lean}). No lattice valuation on M_3 is sharp.

Modularity applied to each of the three pairs of distinct atoms forces $v(x) + v(y) = 1$, since $x \sqcup y = \top$ and $x \sqcap y = \perp$. If v were sharp, $v(a), v(b), v(c) \in \{0, 1\}$ would have to satisfy all three pairwise sum-to-one equations at once, and a two-element codomain cannot support that: fixing any one value propagates through the other two equations to a contradiction with the third. The collapse on M_3 is therefore not a failure of separation, points that happen to coincide, but a failure of existence: there is no all-or-nothing valuation on M_3 to separate anything with, and the first qualifying condition of Section 14 is unsatisfiable outright.

Where this leaves the boundary. Frames are complete distributive lattices. Theorem 57 shows completeness is not needed for the sharp/point correspondence's separation half, and Theorem 59 shows distributivity cannot be given up in exchange. The event algebras on which sharp valuations reconstruct the order, the order-theoretic core of the first qualifying condition of Section 14, are exactly the distributive bounded lattices, no more and no less. The second qualifying condition and the negation-dependent machinery of Sections 3–7, slack, the complement bound, the De Morgan gap, are untouched by this section and remain tied to frames: nothing here shows, or claims, that $\neg$ or $\Rightarrow$ survives dropping completeness. This bears on the orthomodular event lattices of quantum probability discussed in Section 13: giving up distributivity, as those lattices do, does not merely change which valuations exist, it can remove sharp valuations altogether, so the construction of this paper does not, without further modification, have anything to say about that setting even at the order-theoretic level examined here. Whether some weaker notion of certainty survives on non-distributive lattices, in place of the $\{0, 1\}$ -valued sharp valuations examined here, is not addressed by this section.

A second boundary: grading a non-idempotent tensor. Section 13 notes that linear logic's quantale tensor $\otimes$ is not the lattice meet $\sqcap$ this section's valuations are built from, so Definition 51's modularity axiom does not even parse for $\otimes$ without first deciding what to write in $\sqcap$'s place. Granting the substitution provisionally, to see what it costs, shows the obstruction runs deeper than a notational mismatch:

Definition 60 (tensor valuation^{Lean}). Fix a join-semilattice Ω and a binary operation $\otimes$ on Ω . A *tensor valuation* for $\otimes$ is a function $v : \Omega \rightarrow [0, \infty]$ such that, for all $a, b \in \Omega$, $v(a) + v(b) = v(a \sqcup b) + v(a \otimes b)$.

Theorem 61 (the transplant collapses^{Lean}). For every join-semilattice Ω , binary operation $\otimes$ on Ω , and tensor valuation v for $\otimes$, if $v(a) < \infty$ then $v(a \otimes a) = v(a)$.

Setting $b := a$ in the defining identity and using only $a \sqcup a = a$, join-idempotence, already gives $v(a) + v(a) = v(a) + v(a \otimes a)$, hence the claim by cancellation. No property of $\otimes$ is used: not associativity, not a unit, not distributivity over $\sqcup$. This is a sharper failure than Theorem 59's: M_3 shows meet-based modularity can fail to admit a sharp valuation on one uncooperative lattice, while Theorem 61 shows the $\otimes$ -based transplant admits no valuation separating a from $a \otimes a$ on any Ω and for any $\otimes$, idempotent or not. A non-idempotent $\otimes$, linear logic's whole point, is exactly what such a valuation could never distinguish from $\sqcap$'s behavior.

Example 62 (a concrete witness^{Lean}). Take $\Omega = \mathbb{N}$ with $\sqcup = \max$ and $\otimes = +$: addition is associative and commutative but not idempotent ($1 + 1 = 2 \neq 1$), so the hypothesis of Theorem 61 is not vacuous. Substituting $a = 0$, where the

join's unit and the tensor's unit coincide, forces $v(n) = v(0)$ for every n : every tensor valuation for $+$ is constant.

This is not a toy chosen merely by analogy with linear logic. Every quantale, the event algebra Section 13 attaches to linear logic, is by definition a complete lattice equipped with an associative $\otimes$ that need not be idempotent, exactly the data $(\Omega, \sqcup, \otimes)$ Theorem 61 quantifies over, with the quantale's own join playing the role of $\sqcup$. Example 62 is a quantale itself ($+$ distributes over $\max$), kept only because it is small enough to check by hand: it fails idempotence for the same reason $\otimes$ does, no algebraic law identifies two copies of a resource with one.

Corollary 63 (grading and discrimination are exclusive^{Lean}). For every join-semilattice Ω , binary operation $\otimes$ on Ω , and $a \in \Omega$ with $v(a) < \infty$: no v is both a tensor valuation for $\otimes$ and satisfies $v(a \otimes a) \neq v(a)$.

This restates Theorem 61 as the dichotomy it really is. A candidate probabilistic semantics for a non-idempotent $\otimes$ must choose between being graded by Definition 51's compositional law and discriminating the one thing a non-idempotent $\otimes$ exists to discriminate, a from $a \otimes a$; it cannot keep both. The probabilistic semantics that does exist for linear logic, probabilistic coherence spaces [Danos and Ehrhard 2011], keeps discrimination: two independent copies of a resource are not scored as interchangeable with one. Corollary 63 shows the price of that choice is forced, not chosen: probabilistic coherence spaces cannot also be graded by this paper's modularity law, on pain of losing exactly the discrimination they exist to keep, so they instead evaluate proofs with a classical, Kolmogorov-style structure over $\otimes$'s type-level behavior, never a valuation of propositions in Definition 51's sense. Nothing here is a missed opportunity on either side; the dichotomy forecloses the combination before any further axiom of Definition 51 is even reached. Section 10 replaces these axioms by a construction that applies to linear logic.

10 Probability Calculi from Certain States

Corollary 63 shows that linear logic admits no grading by Definition 51's axioms that distinguishes a from $a \otimes a$. The obstruction is in the modularity axiom, which uses the lattice operations of the event algebra. This section replaces the three axioms of Definition 2 by a construction that uses no lattice operations. It applies to any logic presented by a set of certain states. On finite frames, and in particular on finite Boolean algebras, it returns exactly the valuations of Definition 2. For every formula of multiplicative-additive linear logic over finite data types, its output is contained in the probabilistic coherence space (Definition 70) of the formula, and the certain elements of that space are cliques of Girard's coherence space. At implications between data

types it returns the probabilistic coherence space. At the exponential !Bool it returns a strict subset.

10.1 The Construction

Definition 64 (presentation by certain states, mixtures^{Lean}). A *presentation* is a set E of events together with a set S of $\{0, 1\}$ -valued functions $E \rightarrow [0, \infty]$, the *certain states*. Its *calculus* $\text{Mix}(S)$ is the set of finite convex combinations $\sum_i w_i s_i$ with $s_i \in S$, $w_i \in [0, \infty]$, and $\sum_i w_i = 1$.

For classical logic the certain states are the two-valued models, for intuitionistic logic on a frame they are the sharp valuations (Theorem 12), and for a type of linear logic they are the deterministic programs of that type (Section 10.2). Two properties hold for every presentation.

Theorem 65 (certain elements and mixing^{LeanLean}). *For every presentation (E, S) : (i) a $\{0, 1\}$ -valued x lies in $\text{Mix}(S)$ iff $x \in S$, and (ii) $\text{Mix}(\text{Mix}(S)) = \text{Mix}(S)$.*

Part (i) is the first condition of Section 1, that the $\{0, 1\}$ -valued elements of the calculus reconstruct the logic. In a presentation the logic is given by S , so the condition says that mixing creates no new $\{0, 1\}$ -valued elements. The proof: if $x = \sum_i w_i s_i$ is $\{0, 1\}$ -valued and $w_j \neq 0$, then at each e with $x_e = 0$ every s_i with $w_i \neq 0$ has $s_i(e) = 0$, and at each e with $x_e = 1$ every such s_i has $s_i(e) = 1$, since $\sum_i w_i = 1$ and $s_i(e) \leq 1$. So $s_j = x$, and $x \in S$. Part (ii) states that a convex combination of elements of $\text{Mix}(S)$ is again in $\text{Mix}(S)$. For valuations on a frame this is the closure of valuations under mixing of Section 4^{Lean}. The second condition of Section 1, that the classical case returns the classical theory, is Theorem 68 below.

The three axioms of Definition 2 are instances of one form:

Definition 66 (affine law^{Lean}). For finite E , an *affine law* is given by $a, b : E \rightarrow [0, \infty]$ and $k, k' \in [0, \infty]$. A function $x : E \rightarrow [0, \infty]$ satisfies it if $\sum_e a_e x_e + k \leq \sum_e b_e x_e + k'$.

Monotonicity $x_a \leq x_b$, normalization ($x_\top \leq 1$ and $1 \leq x_\top$, $x_\perp \leq 0$), and modularity (two opposite inequalities) are affine laws.

Theorem 67 (completeness^{Lean}). *For finite E and every set S of $\{0, 1\}$ -valued functions on E , $\text{Mix}(S)$ is the set of functions satisfying every affine law that every $s \in S$ satisfies.*

The inclusion from left to right holds because affine laws are preserved by convex combinations^{Lean}. The converse separates a point outside the convex hull of S by a hyperplane, using mathlib's geometric Hahn–Banach theorem, and reads the hyperplane off as an affine law. Theorem 67 is the generalization of the three axioms: the axioms of the calculus of a presentation are the affine laws valid on its certain states, and nothing else. On frames these are the three axioms of Definition 2:

Theorem 68 (frames^{Lean}). *For a finite poset P , the valuations on the frame $\text{Low}(P)$ of lower sets of P are exactly Mix of its sharp valuations.*

The proof combines Theorem 17 with the fact that a sharp valuation on $\text{Low}(P)$ is a point valuation δ_p^{Lean} . Every finite frame is of the form $\text{Low}(P)$ (Section 4), and finite Boolean algebras are the case of a discrete P , where the valuations are the probability mass functions on P . So the construction reduces to the valuation calculus on finite intuitionistic event algebras and to Kolmogorov's calculus on finite classical ones.

For infinite E the laws are taken with finite support. A *window* is a finite set $F \subseteq E$ of events, and a *finite law* is an affine law on some window, applied to the restrictions of functions to F .

Theorem 69 (barycenters^{LeanLean}). *Let S be a set of $\{0, 1\}$ -valued functions on an arbitrary set E . For $x : E \rightarrow [0, \infty]$ the following are equivalent: (i) x satisfies every finite law valid on S ; (ii) for every window F , the restriction of x to F lies in Mix of the restrictions of S to F ; (iii) there is a Borel probability measure μ on $\{0, 1\}^E$, with the product topology, such that $x_e = \mu\{\sigma \mid \sigma_e = 1\}$ for every $e \in E$, and $\mu(\text{bad}_F) = 0$ for every window F , where bad_F is the set of σ whose restriction to F is the restriction of no $s \in S$.*

The equivalence of (i) and (ii) applies Theorem 67 to each window. For (ii) $\Rightarrow$ (iii), the mixture on a window F gives a probability measure μ_F on $\{0, 1\}^F$, a finite convex combination of point masses at certain states. The windows are directed by inclusion. The probability measures on the compact space $\{0, 1\}^E$ form a compact space in the weak topology², so the family $(\mu_F)_F$ has a cluster point μ . A *cylinder set* is a set of σ determined by the values of σ on a window. Cylinder sets are clopen, so $\nu \mapsto \nu(C)$ is continuous in the weak topology for every cylinder set C . The sets $\{\sigma \mid \sigma_e = 1\}$ and bad_F are cylinder sets, and μ_F has the required values on them once $e \in F$, so μ has them. For (iii) $\Rightarrow$ (i), a finite law valid on S holds at μ -almost every σ , and integrating it against μ gives the law at x .

$\text{Mix}(S)$ is the least convex set containing S . Criteria (1) and (2) of Section 1 do not single it out: the probabilistic coherence space !Bool satisfies (1) and strictly contains the calculus (Theorem 72). Whether a further criterion selects $\text{Mix}(S)$, and how the criteria should treat logics without certain states (Section 12), is open.

10.2 Linear Logic

Definition 70 (probabilistic coherence space [Danos and Ehrhard 2011]^{Lean}). For a set W and a set Q of functions $W \rightarrow [0, \infty]$, let $Q^\perp = \{y : W \rightarrow [0, \infty] \mid \sum_w x_w y_w \leq 1 \text{ for all } x \in Q\}$. A *probabilistic coherence space* (PCS) with *web* W is a set P of functions $W \rightarrow [0, \infty]$ such that $P = P^{\perp\perp}$,

²In mathlib this is an instance proved from Prokhorov's theorem.

every $w \in W$ has some $x \in P$ with $x_w \neq 0$ (totality), and $\sup_{x \in P} x_w < \infty$ for every $w \in W$ (boundedness).

We use finite webs. For a finite set X , the *data type* X is the PCS of $x : X \rightarrow [0, \infty]$ with $\sum_a x_a \leq 1$. The PCS $X \multimap Y$ on $X \times Y$ is the set of t with $\sum_b \sum_a x_a t(a, b) \leq 1$ for every x in X , equivalently $\sum_b t(a, b) \leq 1$ for every a^{Lean} . Both satisfy the three conditions^{Lean}. A *coherence space* [Girard 1987] is a set with a reflexive symmetric relation, its coherence, and a *clique* is a set of pairwise coherent elements. In the coherence space $X \multimap Y$ of two data types, two points (a, b) and (a', b') cohere iff they are equal or $a \neq a'$.

Probabilistic coherence spaces interpret every formula of linear logic, including the exponentials, and model probabilistic PCF [Danos and Ehrhard 2011]. The construction of Definition 64 applies to every such type, with the $\{0, 1\}$ -valued elements of its probabilistic coherence space as certain states. We compare the two at the types where we have results. At first order they agree. The exponential $! \text{Bool}$ separates them already at first order. Section 10.3 proves the facts that hold at every multiplicative-additive formula. Whether the construction agrees with the probabilistic coherence space at other types is open.

Theorem 71 (first order^{LeanLean}). $X \multimap Y = \text{Mix}\{\det f \mid f : X \rightarrow Y_{\perp}\}$, where $\det f(a, b) = 1$ if $f(a) = b$ and 0 otherwise, and $Y_{\perp}$ adds a divergent value. A $\{0, 1\}$ -valued t lies in $X \multimap Y$ iff its support is a clique of the coherence space $X \multimap Y$.

At first order the certain states are the deterministic partial programs, they are the cliques of the coherence-space model, and the calculus is the PCS.

The exponential separates the two at first order. The PCS $! \text{Bool}$ has web the finite multisets over $\{t, f\}$, and is the biorthogonal closure of the promotions $x^i(i, j) = x_t^i x_f^j$ of the elements x of Bool , where (i, j) is the multiset with i copies of t and j copies of f .

Theorem 72 (exponential^{LeanLean}). Every $\{0, 1\}$ -valued element of $! \text{Bool}$ vanishes at every (i, j) with $i, j \geq 1$. The promotion of $x = (\frac{1}{2}, \frac{1}{2})$ takes the value $\frac{1}{4}$ at $(1, 1)$, so it lies in $! \text{Bool}$ and not in Mix of the $\{0, 1\}$ -valued elements.

The first statement uses the dual element $4\delta_{(i,j)}$, which is admissible because $x_t + x_f \leq 1$ implies $4x_t x_f \leq 1$. The $\{0, 1\}$ -valued elements are therefore supported on the multisets whose underlying set is a clique of Bool . These underlying sets are the points of the web of $! \text{Bool}$ in Girard's coherence spaces. Danos and Ehrhard [Danos and Ehrhard 2011] observe that elements of the PCS $! \text{Bool}$ are nonzero on multisets whose underlying set is not a clique. Theorem 72 shows that such elements are not mixtures of certain elements.

10.3 All Multiplicative-Additive Formulas

Definition 73 (formulas and their semantics^{Lean}). The formulas are generated from the data types $\mathbf{n}$, for $n \in \mathbb{N}$, by linear

negation $A^{\perp}$, $A \otimes B$, and $A \& B$. Write $A \wp B = (A^{\perp} \otimes B^{\perp})^{\perp}$, $A \oplus B = (A^{\perp} \& B^{\perp})^{\perp}$, and $A \multimap B = (A \otimes B^{\perp})^{\perp}$. The web $|A|$ is $\{1, \dots, n\}$ for $\mathbf{n}$, $|A|$ for $A^{\perp}$, $|A| \times |B|$ for $A \otimes B$, and $|A| \sqcup |B|$ for $A \& B$. The PCS $P(A)$ is the data type for $\mathbf{n}$, $P(A)^{\perp}$ for $A^{\perp}$, $\{x \otimes y \mid x \in P(A), y \in P(B)\}^{\perp\perp}$ for $A \otimes B$, where $(x \otimes y)(a, b) = x_a y_b$, and the set of z whose restrictions to $|A|$ and $|B|$ lie in $P(A)$ and $P(B)$ for $A \& B$ [Danos and Ehrhard 2011]. The coherence $\preceq_A$ is equality on $\mathbf{n}$, and $s \preceq_{A^{\perp}} t$ iff $s = t$ or not $s \preceq_A t$. On $A \otimes B$ it is componentwise. On $A \& B$ it is $\preceq_A$ and $\preceq_B$ on the two components, and any two points from different components cohere [Girard 1987].

Theorem 74 (every formula^{LeanLeanLeanLean}). For every formula A : (i) $P(A)$ is a PCS; (ii) the indicator of every point and of every pair of distinct coherent points lies in $P(A)$, and $x_s + x_t \leq 1$ for every $x \in P(A)$ and every pair of distinct incoherent points s, t ; (iii) every $\{0, 1\}$ -valued element of $P(A)$ is the indicator of a clique of the coherence space A ; (iv) Mix of the $\{0, 1\}$ -valued elements of $P(A)$ is contained in $P(A)$, and its $\{0, 1\}$ -valued elements are exactly those of $P(A)$.

Part (ii) is proved by simultaneous induction on A . Negation exchanges the statement about coherent pairs with the statement about incoherent pairs. For $A \otimes B$, the indicator of a coherent pair is bounded by the tensor of indicators of its projections, and a pair incoherent in $A \otimes B$ is incoherent in one component, where the bound of A or B applies. Part (iii) follows from (ii): two incoherent points in the support of a $\{0, 1\}$ -valued element would give it total 2 on a pair bounded by 1. Part (iv) is Theorem 65 together with the closure of an orthogonal under mixing^{Lean}.

The semantics of Definition 73 computes the spaces of Section 10.2: $P(\mathbf{n} \multimap \mathbf{m})$ is the PCS $X \multimap Y$ of that section for $X = \{1, \dots, n\}$ and $Y = \{1, \dots, m\}$ ^{Lean}. So Theorem 71 is a statement about these formulas^{Lean}.

The exponentials, whose webs are infinite, are not covered by Definition 73. Theorem 72 treats $! \text{Bool}$ separately.

11 Second-Order Linear Types

This section uses the construction of Section 10 at the second-order type of a program with two functional arguments, where it returns a strict subset of the probabilistic coherence space. By computation, not formalized, the construction and the probabilistic coherence space agree on $(\text{Bool} \multimap \text{Bool}) \multimap \text{Bool}$ and $((\text{Bool} \multimap \text{Bool}) \multimap \text{Bool}) \multimap \text{Bool}$, the two iterated implications into Bool we checked. The type T below is the smallest we found where they differ without exponentials.

11.1 Randomized Sequential Programs

At second order, let $T = ((X \multimap Y) \otimes (X' \multimap Y'))^{\perp}$, the type of a program that receives $f : X \multimap Y$ and $g : X' \multimap Y'$, uses each once, and halts or diverges. We take T to be the set of $t : (X \times Y) \times (X' \times Y') \rightarrow [0, \infty]$ with $\sum_{p,q} y_p t(p, q) z_q \leq 1$ for all y in $X \multimap Y$ and z in $X' \multimap Y'$, the orthogonal of the

products $y \otimes z$. For $X = Y = X' = Y' = \text{Bool}$, Lean checks that T satisfies the three PCS conditions^{Lean}. A randomized sequential program queries one argument first. Querying f first, it chooses the input a with probability $\mu(a)$, after the answer b chooses the input c to g with probability $\kappa(a, b, c)$, and after the answer d halts with probability $\rho(a, b, c, d)$. Its value at $((a, b), (c, d))$ is $\mu(a)\kappa(a, b, c)\rho(a, b, c, d)$ plus the analogous term for programs that query g first, with $\sum \mu \leq 1$ over both orders, $\sum_c \kappa(a, b, c) \leq 1$, and $\rho \leq 1$. It is *deterministic* if its values are in $\{0, 1\}$.

Theorem 75 (second order^{LeanLean}). *For nonempty finite X, X' and finite Y, Y' : (i) a $\{0, 1\}$ -valued z lies in T iff its support lies in a tree $\{((a, b), (c(b), d)) \mid b \in Y, d \in Y'\}$ for some $a \in X$ and $c : Y \rightarrow X'$, or in a tree $\{((a(d), b), (c, d))\}$ that queries g first, and these are exactly the deterministic sequential programs^{Lean}; (ii) Mix of the $\{0, 1\}$ -valued elements of T is exactly the set of randomized sequential programs.*

Part (ii) contains both directions of Kuhn's theorem for this type [Kuhn 1953]: every randomized sequential program is a mixture of deterministic ones^{Lean}, and every mixture of randomized sequential programs is a randomized sequential program^{Lean}. The second direction conditions the choices at each node on the history, and nodes reached with probability 0 are given weight 0. For $X = Y = X' = Y' = \text{Bool}$ the same statements are also proved directly^{Lean}.

A language for T . Randomized sequential programs are the denotations of a small language^{Lean}. It has four syntactic stages: no function called, f called, g called, and both called. Each stage has divergence and a finite probabilistic choice $\text{choose}(w_1 P_1, \dots, w_k P_k)$ with $\sum_i w_i \leq 1$, the missing mass diverging. The first stage has call $_f a k$ and call $_g c k$ with a continuation k for each answer, the middle stages call the remaining function, and the last stage halts. The stages make each function called exactly once. A choice denotes the weighted sum of the denotations of its branches, and call $_f a k$ denotes $((a', b), (c, d)) \mapsto [a' = a] \llbracket k b \rrbracket(c, d)$.

Theorem 76 (soundness and definability^{LeanLean}). *For nonempty finite X, X' and finite Y, Y' , every program denotes a randomized sequential program, hence an element of Mix of the $\{0, 1\}$ -valued elements of T . Every $\{0, 1\}$ -valued element of T is the denotation of a program without choice. So Mix of the $\{0, 1\}$ -valued elements of T is exactly the set of denotations of programs^{Lean}.*

The language is not a full programming language: it has no recursion and no higher-order arguments beyond f and g . Relating it to probabilistic PCF at this type is not done here.

The calculus is strictly smaller than the PCS. Call two distinct web points of T *exclusive* if a single pair of deterministic partial programs (f, g) produces both, that is, if their first components cohere in $X \multimap Y$ and their second components cohere in $X' \multimap Y'$. The *exclusivity graph* of T has the web points as vertices and the exclusive pairs as edges.

Theorem 77 (strict inclusion^{LeanLean}). *Let X, Y, X', Y' have at least two elements each. There are five web points of T such that the function with value $\frac{1}{2}$ on them and 0 elsewhere lies in T and is not in Mix of the $\{0, 1\}$ -valued elements of T . For $X = Y = X' = Y' = \text{Bool}$, the total weight on the five points is at most 2 for every randomized sequential program, 2 is attained, and it is at most $\frac{5}{2}$ on T , attained by the function above.*

Over Bool , writing (a, b) for the observation $f(a) = b$, the five points are $((t, t), (t, t))$, $((t, f), (f, f))$, $((f, t), (t, f))$, $((f, t), (f, t))$, and $((f, f), (f, f))$. They induce a 5-cycle in the exclusivity graph. Lean checks the two facts the proof uses, that the induced subgraph has no triangle and no three pairwise non-adjacent vertices^{Lean}, and embeds the five points into larger types by injections from Bool .

The semantics of Definition 73 computes $T: P((\mathbf{n} \multimap \mathbf{m}) \otimes (\mathbf{n}' \multimap \mathbf{m}'))^\perp$ is T for $X = \{1, \dots, n\}$, $Y = \{1, \dots, m\}$, $X' = \{1, \dots, n'\}$, $Y' = \{1, \dots, m'\}$ ^{Lean}. So Theorem 75(ii) and the strict inclusion of Theorem 77 are statements about these formulas^{Lean}.

Related work. Full abstraction of probabilistic coherence spaces for probabilistic PCF [Ehrhard et al. 2018] is proved from the analyticity of the morphisms, not from definability of every point, and in the concurrent game model of Castellan, Clairambault, Paquet and Winskel [Castellan et al. 2018] definability fails until strategies are restricted to sequential probabilistic innocent ones. Kuhn's theorem [Kuhn 1953] is the classical form of Theorem 75(ii) for games with perfect recall. The tensor of probabilistic coherence spaces has been related to no-signalling composition [Simmons and Kissinger 2022], which is the setting of the KCBS comparison below. What is added here is an explicit point outside the calculus at a type without exponentials, the exact description of the calculus and of the space as STAB and QSTAB of one graph (Theorem 79), and the mechanization.

11.2 A Clique Outside the PCS

The converse of Theorem 74(iii) fails. Let $T = ((2 \multimap 2) \otimes (2 \multimap 2))^\perp$. On the five points of Theorem 77, two distinct points cohere in T iff they are not exclusive. Order them as $c_0, \dots, c_4$ along the 5-cycle of the exclusivity graph. The coherence graph on them is the complement of this cycle, again a 5-cycle. Two distinct points of $T \otimes T$ are incoherent iff they are incoherent in one component. A clique of $(T \otimes T)^\perp$ contained in $\{c_0, \dots, c_4\}^2$ is therefore a stable set of the strong product of the two coherence graphs, a copy of $C_5 \boxtimes C_5$. The set $\{(c_i, c_{2i}) \mid i \in \mathbb{Z}/5\}$ is such a stable set, of size 5. It is Shannon's stable set, which shows that the Shannon capacity of C_5 is at least $\sqrt{5}$ [Lovász 1979].

Theorem 78 (a clique outside the PCS^{LeanLean}). *Shannon's set is a clique of the coherence space $(T \otimes T)^\perp$, and its indicator is not an element of $P((T \otimes T)^\perp)$.*

The theorem is also proved for the web of T given directly by $\text{Bool}^{\text{Lean}}$.

The function z of Theorem 77, with value $\frac{1}{2}$ on the five points, lies in $P(T)$, so $z \otimes z$ lies in $P(T \otimes T)$. Its pairing with the indicator of Shannon's set is $5 \cdot \frac{1}{4} = \frac{5}{4} > 1$. Two choices of certain states for a formula are therefore different from $(T \otimes T)^\perp$ on: the $\{0, 1\}$ -valued elements of the PCS, for which the calculus is contained in the PCS at every formula (Theorem 74(iv)), and the cliques of the coherence space, for which it is not.

11.3 When the Calculus Equals the PCS

For a finite graph G , a *clique* is a set of pairwise adjacent vertices and a *stable set* a set of pairwise non-adjacent vertices. Let $\text{STAB}(G)$ be Mix of the indicator functions of stable sets, and $\text{QSTAB}(G)$ the set of $x : V \rightarrow [0, \infty]$ with $\sum_{v \in C} x_v \leq 1$ for every clique C [Grötschel et al. 1988].

Theorem 79 (graph polytopes^{LeanLean}). $T = \text{QSTAB}(G)$ and Mix of the $\{0, 1\}$ -valued elements of T is $\text{STAB}(G)$, for G the exclusivity graph of T .

Theorem 80 (odd holes and antiholes^{LeanLean}). If $\text{STAB}(G) = \text{QSTAB}(G)$, then G contains no induced cycle of odd length $2k + 1 \geq 5$ and no induced complement of one. In particular, if Mix of the $\{0, 1\}$ -valued elements of T equals T , the exclusivity graph of T has neither.

A graph with neither is Berge. A graph is *perfect* if every induced subgraph has chromatic number equal to its largest clique size. The converse of Theorem 80 holds: $\text{STAB}(G) = \text{QSTAB}(G)$ iff G is perfect [Chvátal 1975; Lovász 1972], and G is perfect iff it is Berge, by the strong perfect graph theorem [Chudnovsky et al. 2006]. The converse is not formalized.

The *independence number* $\alpha(G)$ is the largest size of a stable set, and the *fractional packing number* $\alpha^*(G)$ is the largest value of $\sum_v x_v$ over $\text{QSTAB}(G)$. The bounds 2 and $\frac{5}{2}$ of Theorem 77 are $\alpha(C_5)$ and $\alpha^*(C_5)$. Cabello, Severini and Winter [Cabello et al. 2014] show that in a contextuality scenario with exclusivity graph G the noncontextual bound is $\alpha(G)$, the quantum bound is the Lovász number $\vartheta(G)$ [Lovász 1979], and the bound for theories satisfying only exclusivity is $\alpha^*(G)$. For C_5 this is the KCBS inequality [Klyachko et al. 2008], with $\vartheta(C_5) = \sqrt{5}$. A calculus for T corresponding to the theta body of the exclusivity graph [Grötschel et al. 1988], which lies between STAB and QSTAB , is not constructed here.

12 Quantum Event Logic

In quantum logic the events are the closed subspaces of a Hilbert space [Birkhoff and von Neumann 1936]. We restrict to rays, the one-dimensional subspaces, of $\mathbb{R}^d$. A *context* is an orthogonal basis, a maximal set of pairwise orthogonal rays, whose events can be observed jointly. A *quantum state*

assigns to each ray a number in $[0, \infty]$ such that the numbers on every context sum to 1. For a unit vector ψ , the *Born rule* $x_v = (\psi \cdot v)^2 / (v \cdot v)$ gives such an assignment. For $d \geq 3$, Gleason's theorem [Gleason 1957] states that every quantum state on all rays comes from a density matrix, a mixture of Born-rule assignments.

The presentation of Definition 64 takes the rays as events and, as certain states, the $\{0, 1\}$ -valued quantum states, which give exactly one ray of every context the value 1. By the Kochen–Specker theorem [Kochen and Specker 1967], for $d \geq 3$ there are none. We formalize a finite instance.

Definition 81 (Cabello's set^{Lean}). Cabello, Estebaranz, and García-Alcaine [Cabello et al. 1996] give 18 vectors of $\mathbb{R}^4$ with coordinates in $\{0, \pm 1\}$, and 9 sets of 4 of them:

$\{0001, 0010, 1100, 1\bar{1}00\}$, $\{0001, 0100, 1010, 10\bar{1}0\}$, $\{1\bar{1}1\bar{1}, 1\bar{1}\bar{1}1, 1\bar{1}00, 0011\}$,
 $\{1\bar{1}1\bar{1}, 1111, 10\bar{1}0, 010\bar{1}\}$, $\{0010, 0100, 1001, 100\bar{1}\}$, $\{1\bar{1}1\bar{1}, 1111, 100\bar{1}, 01\bar{1}0\}$,
 $\{11\bar{1}1, 111\bar{1}, 1\bar{1}00, 0011\}$, $\{11\bar{1}1, \bar{1}111, 1010, 010\bar{1}\}$, $\{111\bar{1}, \bar{1}111, 100\bar{1}, 01\bar{1}0\}$

where $\bar{1}$ stands for -1 .

Theorem 82 (Kochen–Specker for Cabello's set^{LeanLean}). Each of the 9 sets is an orthogonal basis of $\mathbb{R}^4$, and each of the 18 vectors lies in exactly two of them. No function from the 18 vectors to $\{0, 1\}$ takes the value 1 at exactly one vector of every basis.

The first two statements are checked by evaluation. The third is a parity count. Summing such a function over the 9 bases gives 9, and counts every vector twice, so the sum is even.

Theorem 83 (no certain states^{LeanLean}). For the presentation whose events are the 18 vectors and whose certain states are the $\{0, 1\}$ -valued functions summing to 1 on every basis, the set of certain states and Mix of it are empty. For every $\psi \in \mathbb{R}^4$ with $\psi \cdot \psi = 1$, the Born-rule assignment sums to 1 on every basis, so it is a quantum state that is not in Mix of the certain states.

The Born-rule statement is Parseval's identity $\sum_{v \in B} (\psi \cdot v)^2 / (v \cdot v) = \psi \cdot \psi$ for each orthogonal basis B , checked basis by basis^{Lean}.

Quantum probability therefore fails criterion (1) of Section 1: the calculus has no $\{0, 1\}$ -valued elements, so they cannot reconstruct the event logic. Theorem 83 proves this for Cabello's set. For all rays of $\mathbb{R}^d$ with $d \geq 3$ it follows from the Kochen–Specker theorem, which is not formalized. The construction of Section 10 produces nothing at this event logic: with no certain states, every affine law is valid on the certain states, including $1 \leq 0$, and Theorem 67 returns the empty set. The mixtures of certain states are the noncontextual hidden-variable models of the event logic, and Theorem 82 is the statement that this event logic has none [Kochen and Specker 1967]. For lattice valuations, Theorem 59 shows the corresponding absence on the non-distributive lattice M_3 , which has no sharp valuation.

13 Related Work

Informal precedents. Pairing probability with intuitionistic logic is not a new idea, though the literature on it is scattered. Weatherston develops intuitionistic probability axiomatically and defends it with Dutch-book arguments [Weatherston 2003]. His axioms match our valuation axioms exactly, with negation derived rather than assumed, so the object itself and the bound $v(a) + v(\neg a) \leq 1$ are his. What is new here is the decomposition of that bound's slack into dnGap and deMorganGap and its determinacy from v alone (Theorem 6). Paris gives an informal theorem that any modular assignment of non-classical truth values is exactly a Dempster–Shafer belief function [Paris 1994]. We mechanize the analogous fact for our intuitionistic setting: the 2-monotone case follows directly from modularity^{Lean}, and the full tower is Theorem 23. The identification $\text{Bel} = P(\Box \cdot)$ connecting belief functions [Dempster 1967; Shafer 1976] to modal logic is established [Ruspini 1986]. Section 7 shows this identification is not a separate axiom for us but a special case of our frame-level slack, recovered automatically once the frame is a topology and $\neg$ is read as the interior of the complement^{Lean}.

On the localic side, valuations on frames are standard objects of constructive measure theory, with Vickers's work and the Coquand–Spitters development as canonical references [Coquand and Spitters 2009; Vickers 2008]. These normally build in Scott continuity and target additive, Riesz-style goals. Section 4 shows Scott continuity to be a substantive assumption (it decides point-representability on the frames we examine), so our base structure omits it. Cox, Jaynes, Halpern's counterexample, and the Van Horn and Arnborg–Sjödín repairs [Arnborg and Sjödín 2000; Cox 1946; Halpern 1999; Horn 2003; Jaynes 2003] are classical throughout, and Cox's and Van Horn's derivations both lean on double-negation elimination. Section 6 proves the sum-rule half without that move, at the price of positing modularity as an axiom rather than deriving it (Example 29). Van Horn observes that dropping axiom R3 yields an interval-valued theory, exactly the belief/plausibility pair our slack separates. To our knowledge, no prior work, informal or formal, states either characterization theorem: complement rule $\iff$ excluded middle (Theorem 10), or Dempster = Bayes $\iff$ zero slack at the evidence (Theorem 27).

Prior formalizations. Two bodies of mechanized work are adjacent, and neither overlaps. ALEA [Audebaud and Paulin-Mohring 2009] formalizes a discrete distribution monad in Coq for randomized programs: additive probability, no event logic in our sense. Closest is the HoTT/Coq line of Faissole and Spitters on *valuations* over lower reals [Faissole and Spitters 2017], done constructively with Scott continuity and aimed at integration (Riesz) and a Giry-style valuation *monad*. Their valuations differ from ours in carrying Scott continuity, and their monad is the Scott-continuous instance

of the question examined in Section 8. Our development drops the continuity their base structure builds in, and the two mechanizations do not otherwise overlap. On the domain-theoretic side, commutativity of the full valuations monad on the category of directed-complete partial orders is open. A hierarchy of commutative submonads, generated from simple valuations (finite mixtures of point valuations), has been constructed instead [Jia et al. 2021]. Our countably-presented fragment, the valuations given as countable mixtures (Proposition 50), is the frame-side analogue.

Non-Boolean probability elsewhere. Quantum probability on orthomodular lattices, with Gleason's theorem [Birkhoff and von Neumann 1936; Gleason 1957], is the long-standing precedent for re-basing probability on a non-Boolean event logic. Orthomodular lattices give up distributivity itself, while our frames stay distributive by definition (Definition 1), locating all the non-classical behavior in the failure of excluded middle alone.

Linear logic marks a different kind of departure. Its event algebras are quantales, complete lattices equipped with an associative tensor $\otimes$ that need not be idempotent [Girard 1987]. The lattice reduct of a quantale is already an instance of Section 9's generalization, but $\otimes$ itself is not $\sqcap$, and a valuation grading it cannot be phrased in Definition 51's vocabulary without first deciding what modularity even means for a non-idempotent operation. The probabilistic semantics that does exist for linear logic, Girard's probabilistic coherence spaces as developed by Danos and Ehrhard [Danos and Ehrhard 2011], grades the multiplicative structure directly rather than valuing a lattice, and is not a special case of the construction studied here.

Free probability, fuzzy measures, and imprecise probability [Walley 1991] vary other parameters (independence, additivity, single-valuedness). Where Walley's coherent lower previsions posit the belief/plausibility gap directly as a coherence primitive, our slack (Section 2) is derived, forced by modularity and negation rather than assumed. Fritz's synthetic Markov-category framework [Fritz 2020] varies the ambient structure categorically rather than order-theoretically, axiomatizing conditional independence and sufficient statistics abstractly across discrete, measure-theoretic, and Gaussian probability alike. It remains unmechanized and is orthogonal to our variation in the underlying logic. Wolpert and Kinney model mathematics itself as a stochastic process, replacing binary theoremhood with a probability distribution [Wolpert and Kinney 2024]. Theoremhood is a semidecidable event, so the computability guard of Section 7 bears directly on that program: sharp credences about such events are uncomputable. Our variation is specifically the *logic*, holding the Cox-style methodology fixed. Within DS theory, the two conditionings and their inequivalence are classical [Fagin and Halpern 1991; Shafer 1976]. What is new in Theorem 27

is the exact logical characterization of their agreement, and its mechanization.

14 Conclusion

In this work, we take seriously the Cox–Jaynes reading of probability as *extended logic*, and make the extension a construction with the logic of events as a parameter. The construction fixes an event algebra (the chosen logic in algebraic form) and takes as its assignments of degrees of certainty the valuations: the maps from events to numbers satisfying monotonicity, normalization, and modularity. We specify the two conditions that qualify the result as a probability calculus: when events are certain (0 or 1 certainty), the valuations must reconstruct the underlying event logic, and the value of the construction at classical logic must be the classical theory.

The motivation for developing this extension comes from the study of program behaviour in terms of outcome probabilities. Many properties of programs are only semidecidable. However, the Boolean event algebra underlying Kolmogorov’s axioms imposes the law of excluded middle (and therefore decidability) on the structure of logical connectives applied to events. This makes it not a good fit for reasoning about events that are not decidable, such as whether an arbitrary program halts: insisting on a sharp classical readout for such an event does not decide it, but instead produces an object we show to be uncomputable, and a non-sharp classical credence assigns its remainder to non-termination, an event no finite observation confirms. The extension process we propose, when fixing its logic parameter to be intuitionistic, does not conflict with the underlying event logic of computation by construction, and is capable of reasoning about the possibility of non-termination.

Substituting the Heyting event algebra (and therefore intuitionistic logic) for the Boolean algebra representing the classical logic case changes only the consequences that excluded middle supplied. Both qualifying conditions hold at this parameter value (Sections 2 and 3), and classicality enters only through the complement rule (Theorem 10) and its dynamic counterpart, the agreement of Dempster and Bayesian conditioning (Theorem 27). The complement rule degrades to the inequality $v(a) + v(\neg a) \leq 1$. The deficit in this inequality, for which we re-purpose the term *slack*, is a new quantity: the value of the undecided region, given by the boundary mass in the measure model. Slack is submodular yet does not determine the valuation. The weaker logic also adds the two-gap decomposition of slack, the atomic/diffuse divide, and a belief-function reading through $\neg\neg$.

Everything else holds verbatim on frames: conditioning, the product rule, Bayes symmetry, disjoint additivity, total probability over genuine partitions, and inclusion–exclusion itself. On finite frames, every valuation is a classical mass function on the frame’s points. The product-rule half of Cox’s

theorem is independent of the logic while the sum-rule half must be posited, so the direct transcription of the classical Cox statement is unsatisfiable. Every result is machine-checked in Lean 4 over mathlib.

A recurring shape. Three of this paper’s negative results share a form worth naming: two individually reasonable demands on a valuation, jointly unsatisfiable, with the proof identifying which demand is forced to give rather than merely asserting that some construction fails. On M_3 (Theorem 59), sharpness and modularity together are unsatisfiable: a 2-valued readout cannot satisfy all three pairwise sum-to-one equations at once, and distributivity is the hypothesis that has to give. Grading a non-idempotent tensor (Corollary 63) is the same shape one level up, at the axiom itself: being graded by modularity’s compositional law and discriminating a from $a \otimes a$ cannot both hold, for any Ω and any $\otimes$, and probabilistic coherence spaces [Danos and Ehrhard 2011] are the witness that keeps discrimination and gives up gradability. Demanding a sharp, uniform, *computable* readout of a semidecidable event (Theorem 45) is the same trade-off again, one level further out: bivalence and computability cannot both hold, and the classical route keeps bivalence by fiat and gives up an algorithm. None of the three results says only that a construction fails; each names the exact pair of demands responsible and shows the failure is forced, not an artifact of how the construction happened to be set up.

Toward a monad. Section 8 lays the groundwork for a categorical exploration of the construction: Val is a mechanized functor on locales, and its countably-presented fragment composes as kernels, though whether that composition is canonical on valuations rather than their presentations remains open. A commutative monad structure, where it exists, would be the intuitionistic counterpart of the Giry monad and a step toward compositional semantics for probabilistic programs with semidecidable events. One candidate formulation exists: the space of Scott-continuous valuations is the classifying locale of a geometric theory [Coquand and Spitters 2009; Vickers 2008]. Deleting its continuity axiom still leaves a geometric theory, whose points, read classically, are exactly our valuations. Its multiplication, however, must integrate against a merely modular valuation, and the diffuse part of Example 18 is exactly what such integration fails to capture. We offer the formulation as a target, not a result. Beyond intuitionistic logic, no ambient category sending a logic to its probability calculus functorially is evident from our construction, and this question remains open.

Other open problems and future work. A natural extension of our mechanization effort involves completing the Aczél development, and mechanizing the Dutch-book theorem [Weatherson 2003]. Both require substantial additional effort and were deemed out of scope for this work. To extend our mechanization effort with preliminary monad results, we

would quotient the countable presentations (Proposition 50) into a Kleisli category of countably-presented kernels, and mechanize the classifying-locale formulation (which needs classifying locales of geometric theories, infrastructure math-lib currently lacks).

Some related mathematical problems remain open as well. The first is developing a representation of the diffuse part on non-spatial frames. The second is deriving modularity from a criterion more primitive than the mixture characterization (Theorem 17), knowing it cannot come from disjunction data alone (Example 29).

The construction method applies beyond intuitionistic logic: state the classical desiderata, delete or substitute the axioms the target logic no longer licenses, and check what survives, what splits, and what becomes unsatisfiable. Linear logic was the first case tried, and Section 9 closes it in the negative rather than leaving it open: substituting the quantale tensor $\otimes$ for $\sqcap$ in modularity is not merely difficult but unsatisfiable (Corollary 63), so no valuation in this paper's sense can grade $\otimes$'s consumption of evidence without losing the very discrimination $\otimes$ exists to carry [Danos and Ehrhard 2011]. What remains open is the next question this raises, not this instance of it: whether some compositional law other than modularity transplanted verbatim could play our construction's role for logics whose primitive connectives are not idempotent.

References

János Aczél. 1966. *Lectures on Functional Equations and Their Applications*. Academic Press, New York, NY, USA.

Reynald Affeldt, Cyril Cohen, and Damien Rouhling. 2018. Formalization Techniques for Asymptotic Reasoning in Classical Analysis. *Journal of Formalized Reasoning* 11, 1 (2018), 43–76.

Stefan Arnborg and Gunnar Sjödin. 2000. Bayes Rules in Finite Models. In *European Conference on Artificial Intelligence (ECAI)*. IOS Press, Amsterdam, Netherlands, 571–575.

Philippe Audebaud and Christine Paulin-Mohring. 2009. Proofs of Randomized Algorithms in Coq. *Science of Computer Programming* 74, 8 (2009), 568–589.

Garrett Birkhoff. 1937. Rings of Sets. *Duke Mathematical Journal* 3, 3 (1937), 443–454.

Garrett Birkhoff and John von Neumann. 1936. The Logic of Quantum Mechanics. *Annals of Mathematics* 37, 4 (1936), 823–843.

Adán Cabello, José M. Estebarez, and Guillermo García-Alcaine. 1996. Bell-Kochen–Specker Theorem: A Proof with 18 Vectors. *Physics Letters A* 212, 4 (1996), 183–187.

Adán Cabello, Simone Severini, and Andreas Winter. 2014. Graph-Theoretic Approach to Quantum Correlations. *Physical Review Letters* 112, 4 (2014), 040401.

Simon Castellan, Pierre Clairambault, Hugo Paquet, and Glynn Winskel. 2018. The Concurrent Game Semantics of Probabilistic PCF. In *Proceedings of the 33rd Annual ACM/IEEE Symposium on Logic in Computer Science (LICS)*. 215–224.

Gregory J. Chaitin. 1975. A Theory of Program Size Formally Identical to Information Theory. *J. ACM* 22, 3 (1975), 329–340.

Maria Chudnovsky, Neil Robertson, Paul Seymour, and Robin Thomas. 2006. The Strong Perfect Graph Theorem. *Annals of Mathematics* 164, 1 (2006), 51–229.

Václav Chvátal. 1975. On Certain Polytopes Associated with Graphs. *Journal of Combinatorial Theory, Series B* 18, 2 (1975), 138–154.

Thierry Coquand and Bas Spitters. 2009. Integrals and Valuations. *Journal of Logic and Analysis* 1, 3 (2009), 1–22.

Richard T. Cox. 1946. Probability, Frequency and Reasonable Expectation. *American Journal of Physics* 14, 1 (1946), 1–13.

Vincent Danos and Thomas Ehrhard. 2011. Probabilistic Coherence Spaces as a Model of Higher-Order Probabilistic Computation. *Information and Computation* 209, 6 (2011), 966–991.

Leonardo de Moura and Sebastian Ullrich. 2021. The Lean 4 Theorem Prover and Programming Language. In *Conference on Automated Deduction (CADE)*. Springer, Cham, Switzerland, 625–635.

Arthur P. Dempster. 1967. Upper and Lower Probabilities Induced by a Multivalued Mapping. *Annals of Mathematical Statistics* 38, 2 (1967), 325–339.

Dieter Denneberg. 1994. *Non-Additive Measure and Integral*. Kluwer, Dordrecht, Netherlands.

Thomas Ehrhard, Michele Pagani, and Christine Tasson. 2018. Full Abstraction for Probabilistic PCF. *J. ACM* 65, 4 (2018), 23:1–23:44.

Martin H. Escardó. 2004. Synthetic Topology of Data Types and Classical Spaces. *Electronic Notes in Theoretical Computer Science* 87 (2004), 21–156.

Ronald Fagin and Joseph Y. Halpern. 1991. A New Approach to Updating Beliefs. *Uncertainty in Artificial Intelligence* 6 (1991), 347–374.

Florian Faissole and Bas Spitters. 2017. Synthetic Topology in Homotopy Type Theory for Probabilistic Programming. In *Programming Languages meets Program Verification (PPS)*. Full version with M. E. Bidlingmaier: Faissole–Bidlingmaier–Spitters, “Synthetic topology in Homotopy Type Theory for probabilistic programming,” *Math. Struct. Comput. Sci.* 31(4), 2021..

Yannick Forster, Dominik Kirst, and Gert Smolka. 2019. On Synthetic Undecidability in Coq, with an Application to the Entscheidungsproblem. In *Proceedings of the 8th ACM SIGPLAN International Conference on Certified Programs and Proofs (CPP)*. 38–51.

Tobias Fritz. 2020. A Synthetic Approach to Markov Kernels, Conditional Independence and Theorems on Sufficient Statistics. *Advances in Mathematics* 370 (2020), 107239.

Paolo Ghirardato. 1997. On Independence for Non-Additive Measures, with a Fubini Theorem. *Journal of Economic Theory* 73, 2 (1997), 261–291.

Jean-Yves Girard. 1987. Linear Logic. *Theoretical Computer Science* 50, 1 (1987), 1–101.

Michèle Giry. 1982. A Categorical Approach to Probability Theory. In *Categorical Aspects of Topology and Analysis*. Lecture Notes in Mathematics, Vol. 915. Springer, Berlin, Germany, 68–85.

Andrew M. Gleason. 1957. Measures on the Closed Subspaces of a Hilbert Space. *Journal of Mathematics and Mechanics* 6, 6 (1957), 885–893.

Kurt Gödel. 1933. Eine Interpretation des intuitionistischen Aussagenkalküls. *Ergebnisse eines mathematischen Kolloquiums* 4 (1933), 39–40.

Martin Grötschel, László Lovász, and Alexander Schrijver. 1988. *Geometric Algorithms and Combinatorial Optimization*. Springer.

Joseph Y. Halpern. 1999. A Counterexample to Theorems of Cox and Fine. *Journal of Artificial Intelligence Research* 10 (1999), 67–85.

Carl G. Hempel. 1945. Studies in the Logic of Confirmation (I.). *Mind* 54, 213 (1945), 1–26.

Otto Hölder. 1901. Die Axiome der Quantität und die Lehre vom Mass. *Berichte über die Verhandlungen der Sächsischen Gesellschaft der Wissenschaften zu Leipzig* 53 (1901), 1–64.

Kevin S. Van Horn. 2003. Constructing a Logic of Plausible Inference: A Guide to Cox’s Theorem. *International Journal of Approximate Reasoning* 34, 1 (2003), 3–24.

Edwin T. Jaynes. 2003. *Probability Theory: The Logic of Science*. Cambridge University Press, Cambridge, UK.

Xiaodong Jia, Bert Lindenhovius, Michael Mislove, and Vladimir Zamdzhiev. 2021. Commutative Monads for Probabilistic Programming Languages.

In *Logic in Computer Science (LICS)*. IEEE.

Peter T. Johnstone. 1982. *Stone Spaces*. Cambridge University Press, Cambridge, UK.

Alexander A. Klyachko, M. Ali Can, Sinem Binicioğlu, and Alexander S. Shumovsky. 2008. Simple Test for Hidden Variables in Spin-1 Systems. *Physical Review Letters* 101, 2 (2008), 020403.

Simon Kochen and Ernst P. Specker. 1967. The Problem of Hidden Variables in Quantum Mechanics. *Journal of Mathematics and Mechanics* 17, 1 (1967), 59–87.

Anders Kock. 2012. Commutative Monads as a Theory of Distributions. *Theory and Applications of Categories* 26 (2012), 97–131.

Andrey N. Kolmogorov. 1933. *Grundbegriffe der Wahrscheinlichkeitsrechnung*. Springer, Berlin, Germany.

Harold W. Kuhn. 1953. Extensive Games and the Problem of Information. In *Contributions to the Theory of Games II*. Annals of Mathematics Studies, Vol. 28. Princeton University Press, 193–216.

László Lovász. 1972. Normal Hypergraphs and the Perfect Graph Conjecture. *Discrete Mathematics* 2, 3 (1972), 253–267.

László Lovász. 1979. On the Shannon Capacity of a Graph. *IEEE Transactions on Information Theory* 25, 1 (1979), 1–7.

Assia Mahboubi and Enrico Tassi. 2021. *Mathematical Components*. Zenodo. <https://doi.org/10.5281/zenodo.4457887>

The mathlib Community. 2020. The Lean Mathematical Library. In *Certified Programs and Proofs (CPP)*. ACM, New York, NY, USA, 367–381.

J. C. C. McKinsey and Alfred Tarski. 1944. The Algebra of Topology. *Annals of Mathematics* 45, 1 (1944), 141–191.

Jeff B. Paris. 1994. *The Uncertain Reasoner’s Companion: A Mathematical Perspective*. Cambridge University Press, Cambridge, UK.

K. P. S. Bhaskara Rao and M. Bhaskara Rao. 1983. *Theory of Charges: A Study of Finitely Additive Measures*. Pure and Applied Mathematics, Vol. 109. Academic Press, New York, NY, USA.

Giuseppe Rosolini. 1986. *Continuity and Effectiveness in Topoi*. Ph.D. Dissertation. University of Oxford.

Enrique H. Ruspini. 1986. *The Logical Foundations of Evidential Reasoning*. Technical Report 408. SRI International, Menlo Park, CA, USA.

Glenn Shafer. 1976. *A Mathematical Theory of Evidence*. Princeton University Press, Princeton, NJ, USA.

Will Simmons and Aleks Kissinger. 2022. Higher-Order Causal Theories Are Models of BV-Logic. In *47th International Symposium on Mathematical Foundations of Computer Science (MFCS) (LIPIcs, Vol. 241)*. 80:1–80:14.

Steven Vickers. 2008. A Localic Theory of Lower and Upper Integrals. *Mathematical Logic Quarterly* 54, 1 (2008), 109–123.

Peter Walley. 1991. *Statistical Reasoning with Imprecise Probabilities*. Chapman and Hall, London, UK.

Brian Weatherston. 2003. From Classical to Intuitionistic Probability. *Notre Dame Journal of Formal Logic* 44, 2 (2003), 111–123.

David H. Wolpert and David B. Kinney. 2024. A Stochastic Model of Mathematics and Science. *Foundations of Physics* 54 (2024), 21.

Adam Yedidia and Scott Aaronson. 2016. A Relatively Small Turing Machine Whose Behavior Is Independent of Set Theory. *Complex Systems* 25, 4 (2016), 297–327.

A Values in the Extended Reals

Valuations take values in the extended non-negative reals $[0, \infty]$, following the measure-theoretic convention of mathlib. As a result, sums indexed by an arbitrary family are unconditionally defined, with no summability hypothesis, and they commute over dependent sums: for a function f on a sigma type $\Sigma a, \beta(a)$, one has $\sum_p f(p) = \sum_a \sum_b f(a, b)$ with no side condition. This is what lets the countable-mixture results of Section 8 hold as stated. Additionally, subtraction

is truncated at 0. This never affects the main text, since every subtraction there is applied to a pair already ordered by monotonicity, as with the mass of Section 4, so the truncation is never triggered.

B Lean Statements

B.1 Introduction

B.2 Valuations on Frames

```

structure Valuation (Ω : Type*) [Order.
  Frame Ω] where
  toFun : Ω → ℝ≥0∞
  map_bot' : toFun ⊥ = 0
  map_top' : toFun ⊤ = 1
  mono' : Monotone toFun
  modular' : ∀ a b, toFun a + toFun b =
    toFun (a ⊔ b) + toFun (a ⊓ b)

theorem Valuation.add_compl_le_one (v :
  Valuation Ω) (a : Ω) :
  v a + v ac ≤ 1

theorem Valuation.add_compl_eq_sup (v :
  Valuation Ω) (a : Ω) :
  v a + v ac = v (a ⊔ ac)

theorem Valuation.le_one (v : Valuation Ω)
  (a : Ω) : v a ≤ 1

theorem Valuation.
  slack_eq_dnGap_add_deMorganGap (v :
  Valuation Ω) (a : Ω) :
  v.slack a = v.dnGap a + v.deMorganGap
    a

theorem Valuation.slack_eq_sub_add_sub (v
  : Valuation Ω) (a : Ω) :
  v.slack a = (v acc - v a) + (1 - (v a
    cc + v ac))

theorem Valuation.le_compl_compl (v :
  Valuation Ω) (a : Ω) : v a ≤ v acc

theorem Valuation.slack_submodular (v :
  Valuation Ω) (a b : Ω) :
  v.slack (a ⊔ b) + v.slack (a ⊓ b) ≤ v.
    slack a + v.slack b

def Order.Frame.IsDeMorgan (Ω : Type*) [
  Order.Frame Ω] : Prop :=
  ∀ a : Ω, acc ⊔ ac = ⊤

theorem Valuation.
  slack_eq_dnGap_of_isDeMorgan

```

```

2531      (hΩ : Order.Frame.IsDeMorgan Ω) (v :
2532      Valuation Ω) (a : Ω) :
2533      v.slack a = v.dnGap a
2534
2535 noncomputable def Valuation.condVal (v :
2536 Valuation Ω) (b : Ω) (hb : v b ≠ 0) :
2537 Valuation Ω
2538
2539 theorem Valuation.condVal_mul (v :
2540 Valuation Ω) (b : Ω) (hb : v b ≠ 0) (a
2541 : Ω) :
2542 v.condVal b hb a * v b = v (a ⊔ b)
2543
2544 theorem Valuation.condVal_symm (v :
2545 Valuation Ω) {a b : Ω}
2546 (ha : v a ≠ 0) (hb : v b ≠ 0) :
2547 v.condVal b hb a * v b = v.condVal a
2548 ha b * v a
2549
2550 theorem Valuation.cond_add_compl_le (v :
2551 Valuation Ω) (a b : Ω) :
2552 v (a ⊔ b) + v (ac ⊔ b) ≤ v b
2553
2554 theorem Valuation.
2555 add_compl_eq_one_of_complemented (v :
2556 Valuation Ω) {a : Ω}
2557 (ha : a ⊔ ac = ⊤) : v a + v ac = 1
2558
2559 variable [CompleteBooleanAlgebra Ω]
2560
2561 theorem classical_additivity (v :
2562 Valuation Ω) (a : Ω) :
2563 v a + v ac = 1
2564
2565 variable [CompleteBooleanAlgebra Ω]
2566
2567 theorem classical_slack_zero (v :
2568 Valuation Ω) (a : Ω) :
2569 v.slack a = 0
2570
2571 noncomputable def Valuation.mix (w : ι → ℝ
2572 ≥ 0) (hw : ∑ i, w i = 1)
2573 (v : ι → Valuation Ω) : Valuation Ω
2574
2575 B.3 Where Classuality Lives
2576
2577 def Valuation.HasClassicalNegation (v :
2578 Valuation Ω) : Prop :=
2579 ∀ a, v a + v ac = 1
2580
2581 theorem hasClassicalNegation_of_em (hem :
2582 ∀ a : Ω, a ⊔ ac = ⊤)
2583 (v : Valuation Ω) : v.
2584 HasClassicalNegation
2585

```

```

theorem em_of_forall_hasClassicalNegation
2587 (h : ∀ v : Valuation Ω, v.
2588 HasClassicalNegation) : ∀ a : Ω, a
2589 ⊔ ac = ⊤
2590
2591 theorem sharp_iff_point (v : Valuation Ω)
2592 :
2593 v.IsSharp ↔ ∃ (J : Order.Ideal Ω) (hJ
2594 : J.IsPrime) (htop : ⊤ ∉ J),
2595 v = Ideal.toValuation J hJ htop
2596
2597 theorem
2598 sharp_scott_iff_completelyPrimePoint (v
2599 : Valuation Ω) :
2600 v.IsSharp ∧ v.ScottContinuous ↔
2601 ∃ (J : Order.Ideal Ω) (hJ : Ideal.
2602 IsCompletelyPrime J) (htop : ⊤ ∉
2603 J),
2604 v = Ideal.toValuation J hJ.1 htop
2605
2606 B.4 Valuation Representation
2607
2608 noncomputable def Valuation.mass (v :
2609 Valuation (LowerSet P)) (p : P) : ℝ ≥ 0
2610
2611 theorem Valuation.eq_sum_mass (v :
2612 Valuation (LowerSet P)) (U : LowerSet P
2613 ) :
2614 v U = ∑ p ∈ U.toFinset, v.mass p
2615
2616 theorem Valuation.sum_mass (v : Valuation
2617 (LowerSet P)) : ∑ p, v.mass p = 1
2618
2619 noncomputable def Valuation.toPMF (v :
2620 Valuation (LowerSet P)) : PMF P
2621
2622 theorem Valuation.exists_pmf_presentation
2623 {Θ : Type u} [Order.Frame Θ] [Finite Θ]
2624 (v : Valuation Θ) :
2625 ∃ (P : Type u) (⊂ : PartialOrder P) (⊂
2626 : Fintype P) (⊂ : DecidableEq P)
2627 (e : Θ ≈o LowerSet P),
2628 (∀ a, v a = ∑ p ∈ (e a).toFinset, (v
2629 .mapEquiv e).mass p) ∧
2630 ∑ p, (v.mapEquiv e).mass p = 1
2631
2632 noncomputable def deltaPoint (p : P) :
2633 Valuation (LowerSet P)
2634
2635 theorem Valuation.eq_mix_deltaPoint (v :
2636 Valuation (LowerSet P)) :
2637 v = Valuation.mix v.mass v.sum_mass (
2638 fun p => deltaPoint p)
2639
2640

```

```

2641 theorem
2642   exists_valuation_not_point_representable
2643   :
2644    $\exists v : \text{Valuation } (\text{LowerSet } \mathbb{N}), \sum' p, v.$ 
2645     mass p  $\neq v \top$ 
2646
2647 theorem topIndicator_mass (n :  $\mathbb{N}$ ) :
2648   topIndicator.mass n = 0
2649
2650 theorem topIndicator_not_scott :
2651   ( $\sqcup n$ , topIndicator (LowerSet.Iic n))  $\neq$ 
2652     topIndicator  $\top$ 
2653
2654 theorem eq_tsum_mass_of_scott (v :
2655   Valuation (LowerSet  $\mathbb{N}$ ))
2656   (hsc :  $v \top = \sqcup n, v (\text{LowerSet.Iic } n)$ )
2657   :  $v \top = \sum' n, v.\text{mass } n$ 
2658
2659 variable [ $\forall p : P$ , Finite (Set.Iic p)]
2660
2661 theorem Valuation.isPurelyAtomic_of_scott
2662   (v : Valuation (LowerSet P))
2663   (hsc :  $v \top = \sqcup (S : \text{Finset } P), v (S.$ 
2664     sup LowerSet.Iic)) : IsPurelyAtomic
2665     v
2666
2667 theorem tsum_mass_le (v : Valuation (
2668   LowerSet P)) :  $\sum' p, v.\text{mass } p \leq v \top$ 
2669

```

B.5 The Dempster–Shafer Reading

```

2672 theorem Valuation.inclusion_exclusion (v :
2673   Valuation  $\Omega$ ) (s : Finset  $\iota$ )
2674   (a :  $\iota \rightarrow \Omega$ ) :  $v (s.\text{sup } a) + v.\text{iesEven}$ 
2675     s a =  $v.\text{iesOdd } s a$ 
2676
2677 theorem Valuation.infty_monotone (v :
2678   Valuation  $\Omega$ ) (s : Finset  $\iota$ ) (a :  $\iota \rightarrow \Omega$ )
2679   :
2680    $v.\text{iesOdd } s a \leq v ((s.\text{sup } a)^{\text{cc}}) + v.$ 
2681     iesEven s a
2682
2683 theorem Valuation.two_monotone (v :
2684   Valuation  $\Omega$ ) (a b :  $\Omega$ ) :
2685    $v a + v b \leq v (a \sqcup b)^{\text{cc}} + v (a \sqcap b)$ 
2686
2687 noncomputable def Valuation.plausibility (
2688   v : Valuation  $\Omega$ ) (a :  $\Omega$ ) :  $\mathbb{R}_{\geq 0\infty} := 1 -$ 
2689     v ac
2690
2691 theorem Valuation.plausibility_sub_self (v
2692   : Valuation  $\Omega$ ) (a :  $\Omega$ ) :
2693   v.plausibility a - v a = v.slack a
2694

```

```

2696 theorem Valuation.sup_compl_decomp (v :
2697   Valuation  $\Omega$ ) (a b :  $\Omega$ ) :
2698    $v (a \sqcup b^c) = v b^c + (v (a \sqcap b) + v.$ 
2699     emGap a b)
2700
2701 noncomputable def Valuation.dempsterCond (
2702   v : Valuation  $\Omega$ ) (b :  $\Omega$ )
2703   (hb :  $v b \neq 0$ ) : Valuation  $\Omega$ 
2704
2705 theorem Valuation.dempsterCond_apply (v :
2706   Valuation  $\Omega$ ) (b :  $\Omega$ ) (hb :  $v b \neq 0$ )
2707   (a :  $\Omega$ ) : v.dempsterCond b hb a = (v (
2708     a  $\sqcup b^c$ ) - v bc) / (1 - v bc)
2709
2710 theorem Valuation.
2711   dempsterCond_eq_condVal_iff_slack (v :
2712   Valuation  $\Omega$ ) {b :  $\Omega$ }
2713   (hb :  $v b \neq 0$ ) :
2714   ( $\forall a, v.\text{dempsterCond } b \text{ hb } a = v.$ 
2715     condVal b hb a)  $\leftrightarrow v.\text{slack } b = 0$ 
2716

```

B.6 Cox’s Theorem

```

2719 structure CoxModel ( $\Omega : \text{Type}^*$ ) [Order.
2720   Frame  $\Omega$ ] where
2721   pl :  $\Omega \rightarrow \Omega \rightarrow \mathbb{R}$ 
2722   mono_left :  $\forall c, \text{Monotone } \text{fun } a \Rightarrow \text{pl } a$ 
2723     c
2724   pl_bot :  $\forall c, \text{pl } \perp c = 0$ 
2725   pl_top :  $\forall c, \text{pl } \top c = 1$ 
2726   F :  $\mathbb{R} \rightarrow \mathbb{R} \rightarrow \mathbb{R}$ 
2727   conj :  $\forall a b c, \text{pl } (a \sqcap b) c = F (\text{pl } a ($ 
2728     b  $\sqcap c)) (\text{pl } b c)$ 
2729   F_assoc :  $\forall x y z, F (F x y) z = F x (F$ 
2730     y z)
2731   F_strictMono_left :  $\forall y, 0 < y \rightarrow$ 
2732     StrictMono fun x  $\Rightarrow F x y$ 
2733
2734 theorem isDeMorgan_lowerSetV : Order.Frame
2735   .IsDeMorgan (LowerSet V)
2736
2737 theorem modularity_irreducible :
2738    $\exists (\Omega : \text{Type}) (\_ : \text{Order.Frame } \Omega) (q :$ 
2739      $\Omega \rightarrow \mathbb{R}_{\geq 0\infty}),$ 
2740     Monotone q  $\wedge q \perp = 0 \wedge q \top = 1 \wedge$ 
2741     ( $\forall x y : \Omega, x \sqcap y = \perp \rightarrow q (x \sqcup y) =$ 
2742       q x + q y)  $\wedge$ 
2743      $\neg (\forall x y : \Omega, q x + q y = q (x \sqcup y)$ 
2744       + q (x  $\sqcap y))$ 
2745
2746 theorem no_disjunction_functional :
2747    $\neg \exists S : \mathbb{R}_{\geq 0\infty} \rightarrow \mathbb{R}_{\geq 0\infty} \rightarrow \mathbb{R}_{\geq 0\infty},$ 
2748      $\forall x y : \text{LowerSet } V, qV (x \sqcup y) = S$ 
2749       (qV x) (qV y)
2750

```

```

2751 theorem constructive_cox (M : CoxModel  $\Omega$ )
2752   (hmod :  $\forall a b : \Omega, M.pl\ a\ \top + M.pl\ b\ \top$ 
2753     = M.pl (a  $\sqcup$  b)  $\top$  + M.pl (a  $\sqcap$  b)  $\top$ )
2754   :
2755    $\exists (v : \text{Valuation } \Omega) (g : \mathbb{R} \rightarrow \mathbb{R}_{\geq 0\infty}),$ 
2756     StrictMonoOn g (Set.Icc 0 1)  $\wedge$  g 0 =
2757     0  $\wedge$  g 1 = 1  $\wedge$ 
2758      $\forall a, v\ a = g\ (M.pl\ a\ \top)$ 
2759
2760 noncomputable def coxModelENNReal :
2761   CoxModel  $\mathbb{R}_{\geq 0\infty}$ 
2762
2763 theorem Scale.exists_mul_generator :
2764    $\exists G : \mathbb{R} \rightarrow \mathbb{R},$ 
2765     ( $\forall x\ y, S.u \leq x \rightarrow S.u \leq y \rightarrow G\ (S.F\ x\ y) = G\ x * G\ y$ )  $\wedge$ 
2766     ( $\forall x\ y, S.u \leq x \rightarrow x < y \rightarrow G\ x < G\ y$ )
2767
2768 theorem Scale.aczelStatement_cone :
2769    $\exists G : \mathbb{R} \rightarrow \mathbb{R},$ 
2770     ( $\forall x\ y, S.u \leq x \rightarrow S.u \leq y \rightarrow G\ (S.F\ x\ y) = G\ x * G\ y$ )  $\wedge$ 
2771     StrictMonoOn G (Set.Ici S.u)
2772
2773 theorem exists_diag_eq {F :  $\mathbb{R} \rightarrow \mathbb{R} \rightarrow \mathbb{R}$ } {a
2774   b c :  $\mathbb{R}$ } (hab : a  $\leq$  b)
2775   (hcont : ContinuousOn (fun x => F x x)
2776     (Set.Icc a b))
2777   (hc : c  $\in$  Set.Icc (F a a) (F b b)) :  $\exists$ 
2778     x  $\in$  Set.Icc a b, F x x = c
2779
2780 structure Scale where
2781   F :  $\mathbb{R} \rightarrow \mathbb{R} \rightarrow \mathbb{R}$ 
2782   hdiv :  $\forall c, \exists x, F\ x\ x = c$ 
2783   hassoc :  $\forall x\ y\ z, F\ (F\ x\ y)\ z = F\ x\ (F\ y\ z)$ 
2784   hpos :  $\forall c\ x, x < F\ x\ c$ 
2785   hmono :  $\forall x_1\ x_2\ y_1\ y_2 : \mathbb{R}, x_1 \leq x_2 \rightarrow y_1$ 
2786      $\leq y_2 \rightarrow F\ x_1\ y_1 \leq F\ x_2\ y_2$ 
2787   hcont : Continuous fun p :  $\mathbb{R} \times \mathbb{R} \Rightarrow F\ p$ 
2788     .1 p.2
2789   hbot :  $\forall x, \text{Filter.Tendsto (fun z => F x z) Filter.atBot (nhds x)}$ 
2790   u :  $\mathbb{R}$ 
2791
2792 B.7 Two Grounding Models
2793
2794 noncomputable def MeasureTheory.Measure.
2795   toValuationOpens
2796   ( $\mu : \text{Measure } X$ ) [IsProbabilityMeasure
2797      $\mu$ ] : Valuation (Opens X)
2798
2799 theorem toValuationOpens_slack_eq_frontier
2800   ( $\mu : \text{Measure } X$ )
2801
2802   [IsProbabilityMeasure  $\mu$ ] (U : Opens X)
2803   :
2804    $\mu.toValuationOpens.slack\ U = \mu\ ($ 
2805     frontier (U : Set X))
2806
2807 theorem classical_posterior_compl_eq ( $\mu : \text{Measure } X$ )
2808   [IsProbabilityMeasure  $\mu$ ] (a b : Opens X)
2809   (hb :  $\mu.toValuationOpens\ b \neq 0$ ) :
2810    $\mu\ ((a : \text{Set } X)^c \hat{\wedge} b) / \mu\ b$ 
2811   =  $\mu.toValuationOpens.condVal\ b\ hb\ a^c$ 
2812   +  $\mu\ (\text{frontier } (a : \text{Set } X) \hat{\wedge} b) /$ 
2813      $\mu\ b$ 
2814
2815 theorem interiorMeasure_add_compl_le ( $\mu : \text{Measure } X$ ) [IsProbabilityMeasure  $\mu$ ]
2816   (S : Set X) :  $\mu.interiorMeasure\ S + \mu.interiorMeasure\ S^c \leq 1$ 
2817
2818 theorem
2819   isDeMorgan_opens_iff_extremallyDisconnected
2820   :
2821   Order.Frame.IsDeMorgan (Opens X)  $\leftrightarrow$ 
2822     ExtremallyDisconnected X
2823
2824 theorem isDeMorgan_opens_iff_stonean [CompactSpace X] [T2Space X] :
2825   Order.Frame.IsDeMorgan (Opens X)  $\leftrightarrow$ 
2826     ExtremallyDisconnected X
2827
2828 inductive ThreePt : Type
2829   | ptA : ThreePt
2830   | ptB : ThreePt
2831   | ptZ : ThreePt
2832
2833 theorem frontier_subset_ptZ {U : Set ThreePt} (hU : IsOpen U) :
2834   frontier U  $\subseteq$  {ptZ}
2835
2836 theorem exists_ne_valuation_same_slack :
2837    $\exists v\ v' : \text{Valuation } (\text{Opens } \text{ThreePt}), v.slack = v'.slack \wedge v \neq v'$ 
2838
2839 theorem haltsOpen_compl : haltsOpenc = ( $\perp$  : Opens Prop)
2840
2841 theorem slack_eq_boundary {p :  $\mathbb{R}_{\geq 0\infty}$ } (hp1 : p  $\leq$  1) :
2842   (sierpinskiValuation hp1).slack
2843     haltsOpen
2844     = coinMeasure p (frontier {q : Prop | q})
2845
2846
2847
2848
2849
2850
2851
2852
2853
2854
2855
2856
2857
2858
2859
2860

```

```

2861 noncomputable def haltingWeight (n : ℕ) (c
2862   : Code) : ℝ≥0∞ :=
2863   if (c.eval n).Dom then 1 else 0
2864
2865 noncomputable def sharpReadout (n : ℕ) (c
2866   : Code) : Valuation (Opens Prop) :=
2867   sierpinskiValuation (
2868     haltingWeight_le_one n c)
2869
2870 theorem sharpReadout_not_computable (n : ℕ
2871   ) :
2872   ¬ComputablePred fun c : Code =>
2873     sharpReadout n c haltsOpen = 1
2874
2875 theorem isOpen_wordCylinder (w : List Bool
2876   ) : IsOpen (wordCylinder w)
2877
2878 theorem cantorMeasure_wordCylinder (w :
2879   List Bool) :
2880   cantorMeasure (wordCylinder w) = (1 /
2881     2) ^ w.length
2882
2883 theorem
2884   disjoint_wordCylinder_of_not_prefix {v
2885     w : List Bool}
2886   (hv : ¬ v <+: w) (hw : ¬ w <+: v) :
2887   Disjoint (wordCylinder v) (
2888     wordCylinder w)
2889
2890 theorem cantorMeasure_biUnion_wordCylinder
2891   {F : Finset (List Bool)}
2892   (hF : PrefixFree F) :
2893   cantorMeasure (⋃ w ∈ F, wordCylinder w
2894     ) = ∑ w ∈ F, (1 / 2 : ℝ≥0∞) ^ w.
2895     length
2896
2897 B.8 The Monad Question
2898
2899 def Valuation.comap (f : FrameHom Γ Ω) (v
2900   : Valuation Ω) : Valuation Γ
2901
2902 theorem Valuation.comap_id (v : Valuation
2903   Ω) : v.comap (FrameHom.id Ω) = v
2904
2905 theorem Valuation.comap_comp (f : FrameHom
2906   Γ Ω) (g : FrameHom Δ Γ)
2907   (v : Valuation Ω) : v.comap (f.comp g)
2908   = (v.comap f).comap g
2909
2910 theorem exists_rect_decomposition (W :
2911   LowerSet (ℕ × ℕ)) :
2912   ∃ (N : ℕ) (A B : ℕ → LowerSet ℕ),
2913     W = (Finset.range N).sup fun i => A
2914       i ×s B i
2915

```

```

theorem Valuation.eq_on_finsetSup_of_eq_on
2916   {G : Set Ω}
2917   (hG : ∀ g ∈ G, ∀ g' ∈ G, g ⊔ g' ∈ G) (
2918     u u' : Valuation Ω)
2919   (h : ∀ g ∈ G, u g = u' g) {ι : Type*}
2920   [DecidableEq ι]
2921   (s : Finset ι) (a : ι → Ω) (ha : ∀ i ∈
2922     s, a i ∈ G) :
2923     u (s.sup a) = u' (s.sup a)
2924
2925 theorem Valuation.product_unique (v w :
2926   Valuation (LowerSet ℕ))
2927   {u u' : Valuation (LowerSet (ℕ × ℕ))}
2928   (hu : ∀ A B : LowerSet ℕ, u (A ×s B) =
2929     v A * w B)
2930   (hu' : ∀ A B : LowerSet ℕ, u' (A ×s B)
2931     = v A * w B) : u = u'
2932
2933 noncomputable def ultrafilterValuation (W
2934   : Ultrafilter ι) : Valuation (Set ι)
2935
2936 theorem rect_mem_iff {α β : Type*} {W :
2937   Ultrafilter (α × β)}
2938   {S : Set α} {T : Set β} :
2939   S ×s T ∈ W ↔ S ∈ W.map Prod.fst ∧ T ∈
2940     W.map Prod.snd
2941
2942 theorem product_valuation_not_unique :
2943   ∃ u u' : Valuation (Set (ℕ × ℕ)),
2944     (∀ S T : Set ℕ, u (S ×s T) = u' (S ×
2945       s T)) ∧ u ≠ u'
2946
2947 theorem fubini_fails_for_valuations :
2948   ∃ (v w : Valuation (Set ℕ)) (u u' :
2949     Valuation (Set (ℕ × ℕ))),
2950     (∀ S T : Set ℕ, u (S ×s T) = v S * w
2951       T) ∧
2952     (∀ S T : Set ℕ, u' (S ×s T) = v S *
2953       w T) ∧ u ≠ u'
2954
2955 noncomputable def Valuation.mixCountable {
2956   ι : Type*} (w : ι → ℝ≥0∞)
2957   (hw : ∑' i, w i = 1) (v : ι →
2958     Valuation Ω) : Valuation Ω
2959
2960 theorem Valuation.
2961   mixCountable_mixCountable {ι : Type*} {
2962     κ : ι → Type*}
2963   (w : ι → ℝ≥0∞) (hw : ∑' i, w i = 1)
2964   (u : ∀ i, κ i → ℝ≥0∞) (hu : ∀ i, ∑' j,
2965     u i j = 1)
2966   (v : ∀ i, κ i → Valuation Ω) :
2967   Valuation.mixCountable w hw
2968
2969
2970

```

```

2971     (fun i => Valuation.mixCountable (
2972       u i) (hu i) (v i))
2973   = Valuation.mixCountable (fun p :  $\Sigma$ 
2974     i,  $\kappa$  i => w p.1 * u p.1 p.2)
2975     (tsum_sigma_mul_eq_one w hw u hu
2976       ) (fun p => v p.1 p.2)
2977
2978   theorem Valuation.mixCountable_unique (v :
2979     Valuation  $\Omega$ ) :
2980     Valuation.mixCountable (fun _ : PUnit
2981       => 1) (by simp) (fun _ => v) = v
2982
2983   theorem Valuation.mixCountable_comm { $\iota$   $\kappa$  :
2984     Type*}
2985     (w :  $\iota \rightarrow \mathbb{R}_{\geq 0\infty}$ ) (hw :  $\sum' i, w i = 1$ ) (u
2986       :  $\kappa \rightarrow \mathbb{R}_{\geq 0\infty}$ ) (hu :  $\sum' j, u j = 1$ )
2987     (v :  $\iota \rightarrow \kappa \rightarrow \text{Valuation } \Omega$ ) :
2988     Valuation.mixCountable w hw (fun i =>
2989       Valuation.mixCountable u hu (v i))
2990     = Valuation.mixCountable u hu
2991       (fun j => Valuation.mixCountable
2992         w hw (fun i => v i j))
2993
2994
2995
2996

```

B.9 Valuations Beyond Frames

Theorem 56.

```

2997   theorem exists_sharp_separating {a b :  $\Omega$ }
2998     (hab :  $\neg a \leq b$ ) :
2999      $\exists v : \text{Valuation } \Omega, v.\text{IsSharp} \wedge v a = 1$ 
3000        $\wedge v b = 0$ 
3001
3002

```

Definition 51.

```

3003   structure LValuation ( $\Omega : \text{Type}^*$ ) [Lattice
3004      $\Omega$ ] [BoundedOrder  $\Omega$ ] where
3005     toFun :  $\Omega \rightarrow \mathbb{R}_{\geq 0\infty}$ 
3006     map_bot' : toFun  $\perp = 0$ 
3007     map_top' : toFun  $\top = 1$ 
3008     mono' : Monotone toFun
3009     modular' :  $\forall a b, \text{toFun } a + \text{toFun } b$ 
3010       = toFun (a  $\sqcup$  b) + toFun (a  $\sqcap$ 
3011         b)
3012
3013

```

Proposition 52.

```

3014   def LValuation.comap (f :
3015     BoundedLatticeHom  $\Gamma$   $\Omega$ ) (v : LValuation
3016        $\Omega$ ) :
3017     LValuation  $\Gamma$ 
3018
3019   theorem LValuation.comap_id (v :
3020     LValuation  $\Omega$ ) :
3021     v.comap (BoundedLatticeHom.id  $\Omega$ ) = v
3022
3023   theorem LValuation.comap_comp (f :
3024     BoundedLatticeHom  $\Gamma$   $\Omega$ )
3025

```

```

3026     (g : BoundedLatticeHom  $\Delta$   $\Gamma$ ) (v :
3027       LValuation  $\Omega$ ) :
3028     v.comap (f.comp g) = (v.comap f).comap
3029       g
3030

```

Theorem 57.

```

3031   theorem LValuation.exists_sharp_separating
3032     [DistribLattice  $\Omega$ ]
3033     {a b :  $\Omega$ } (hab :  $\neg a \leq b$ ) :
3034      $\exists v : \text{LValuation } \Omega, v.\text{IsSharp} \wedge v a =$ 
3035       1  $\wedge v b = 0$ 
3036

```

Definition 58.

```

3037   inductive M3 : Type
3038     | z | a | b | c | o
3039
3040   instance : Lattice M3 where
3041     -- z bottom, o top, a b c pairwise-
3042     -- incomparable atoms;
3043     -- any two distinct atoms join to o and
3044     -- meet to z
3045
3046   instance : BoundedOrder M3 where
3047     top := o
3048     bot := z
3049

```

```

3050   theorem M3.not_distributive :
3051      $\neg \forall x y w : M3, x \sqcap (y \sqcup w) = (x \sqcap y)$ 
3052        $\sqcup (x \sqcap w)$ 
3053

```

Theorem 59.

```

3054   theorem M3.no_sharp_valuation (v :
3055     LValuation M3) :  $\neg v.\text{IsSharp}$ 
3056

```

Definition 60.

```

3057   def IsTensorValuation (v :  $\Omega \rightarrow \mathbb{R}_{\geq 0\infty}$ ) (t :
3058      $\Omega \rightarrow \Omega \rightarrow \Omega$ ) : Prop :=
3059      $\forall a b, v a + v b = v (a \sqcup b) + v (t a b)$ 
3060

```

Theorem 61.

```

3061   theorem IsTensorValuation.collapse {v :  $\Omega$ 
3062      $\rightarrow \mathbb{R}_{\geq 0\infty}$ } {t :  $\Omega \rightarrow \Omega \rightarrow \Omega$ }
3063     (h : IsTensorValuation v t) {a :  $\Omega$ } (
3064       ha : v a  $\neq \top$ ) :
3065     v (t a a) = v a
3066

```

Example 62.

```

3067   theorem forced_constant_of_add_valuation {
3068     v :  $\mathbb{N} \rightarrow \mathbb{R}_{\geq 0\infty}$ }
3069     (h : IsTensorValuation v ( $\hat{A} \cdot + \hat{A} \cdot$ )) (
3070       hfin :  $\forall n, v n \neq \top$ ) (n :  $\mathbb{N}$ ) :
3071     v n = v 0
3072

```

Corollary 63.

```

3081 theorem
3082   not_isTensorValuation_of_discriminates
3083   {v :  $\Omega \rightarrow \mathbb{R}_{\geq 0\infty}$ }
3084   {t :  $\Omega \rightarrow \Omega \rightarrow \Omega$ } {a :  $\Omega$ } (ha : v a  $\neq \top$ )
3085     (hne : v (t a a)  $\neq$  v a) :
3086      $\neg$  IsTensorValuation v t
3087
3088
3089

```

B.10 Probability Calculi from Certain States

```

3090 def IsSharpVec (x :  $E \rightarrow \mathbb{R}_{\geq 0\infty}$ ) : Prop :=  $\forall$ 
3091   e, x e = 0  $\vee$  x e = 1
3092

```

```

3093 def Mix (S : Set ( $E \rightarrow \mathbb{R}_{\geq 0\infty}$ )) : Set ( $E \rightarrow \mathbb{R}_{\geq 0\infty}$ ) :=
3094   {x |  $\exists$  (n :  $\mathbb{N}$ ) (w : Fin n  $\rightarrow \mathbb{R}_{\geq 0\infty}$ ) (s :
3095     Fin n  $\rightarrow E \rightarrow \mathbb{R}_{\geq 0\infty}$ ),
3096     ( $\forall$  i, s i  $\in$  S)  $\wedge \sum$  i, w i = 1  $\wedge$  x =  $\sum$ 
3097       i, w i  $\hat{\wedge}$  s i}
3098
3099

```

```

3100 theorem sharp_mem_Mix_iff {S : Set ( $E \rightarrow \mathbb{R}_{\geq 0\infty}$ )} (hS :  $\forall$  s  $\in$  S, IsSharpVec s)
3101   {x :  $E \rightarrow \mathbb{R}_{\geq 0\infty}$ } (hx : IsSharpVec x) : x
3102    $\in$  Mix S  $\leftrightarrow$  x  $\in$  S
3103
3104

```

```

3105 theorem Mix_Mix (S : Set ( $E \rightarrow \mathbb{R}_{\geq 0\infty}$ )) : Mix
3106   (Mix S) = Mix S
3107
3108
3109

```

```

3110 structure AffineLaw (E : Type*) [Fintype E]
3111   where
3112   a :  $E \rightarrow \mathbb{R}_{\geq 0\infty}$ 
3113   b :  $E \rightarrow \mathbb{R}_{\geq 0\infty}$ 
3114   k :  $\mathbb{R}_{\geq 0\infty}$ 
3115   k' :  $\mathbb{R}_{\geq 0\infty}$ 
3116

```

```

3117 def AffineLaw.Holds [Fintype E] (L :
3118   AffineLaw E) (x :  $E \rightarrow \mathbb{R}_{\geq 0\infty}$ ) : Prop :=
3119    $\sum$  e, L.a e * x e + L.k  $\leq \sum$  e, L.b e * x
3120     e + L.k'
3121

```

```

3122 theorem AffineLaw.holds_of_Mix [Fintype E]
3123   (L : AffineLaw E) {S : Set ( $E \rightarrow \mathbb{R}_{\geq 0\infty}$ )}
3124   (hL :  $\forall$  s  $\in$  S, L.Holds s) {x :  $E \rightarrow \mathbb{R}_{\geq 0\infty}$ }
3125   (hx : x  $\in$  Mix S) : L.Holds x
3126

```

```

3127 theorem Mix_eq_laws' {S : Set ( $E \rightarrow \mathbb{R}_{\geq 0\infty}$ )}
3128   (hS :  $\forall$  s  $\in$  S, IsSharpVec s) :
3129   Mix S = {x |  $\forall$  L : AffineLaw E, ( $\forall$  s  $\in$ 
3130     S, L.Holds s)  $\rightarrow$  L.Holds x}
3131

```

```

3132 theorem frame_recipe :
3133   Set.range (fun v : Valuation (LowerSet
3134     P) => v.toFun)
3135

```

```

= Mix {f |  $\exists$  v : Valuation (LowerSet
P), v.IsSharp  $\wedge$  f = v.toFun}

```

```

theorem Valuation.eq_deltaPoint_of_isSharp
  {v : Valuation (LowerSet P)} (hv : v.
  IsSharp) :
   $\exists$  p, v = deltaPoint p

```

```

structure FinLaw (E : Type*) where
  F : Finset E
  a :  $E \rightarrow \mathbb{R}_{\geq 0\infty}$ 
  b :  $E \rightarrow \mathbb{R}_{\geq 0\infty}$ 
  k :  $\mathbb{R}_{\geq 0\infty}$ 
  k' :  $\mathbb{R}_{\geq 0\infty}$ 

```

```

def FinLaw.Holds (L : FinLaw E) (x :  $E \rightarrow \mathbb{R}_{\geq 0\infty}$ ) : Prop :=
   $\sum$  e  $\in$  L.F, L.a e * x e + L.k  $\leq \sum$  e  $\in$  L.F
  , L.b e * x e + L.k'

```

```

def restr (F : Finset E) (x :  $E \rightarrow \mathbb{R}_{\geq 0\infty}$ ) :
   $F \rightarrow \mathbb{R}_{\geq 0\infty}$  := fun e => x e

```

```

theorem laws_iff_local_Mix {S : Set ( $E \rightarrow \mathbb{R}_{\geq 0\infty}$ )} (hS :  $\forall$  s  $\in$  S, IsSharpVec s)
  (x :  $E \rightarrow \mathbb{R}_{\geq 0\infty}$ ) :
  ( $\forall$  L : FinLaw E, ( $\forall$  s  $\in$  S, L.Holds s)
   $\rightarrow$  L.Holds x)  $\leftrightarrow$ 
   $\forall$  F : Finset E, restr F x  $\in$  Mix (
  restr F '' S)

```

```

noncomputable def ofVec (s :  $E \rightarrow \mathbb{R}_{\geq 0\infty}$ ) :
  Cfg E := fun e => decide (s e = 1)

```

```

noncomputable def testFn {U : Set (Cfg E)}
  (hU : IsClopen U) : Cfg E  $\rightarrow$  L : FinLaw E, (s S,
  L.Holds s)  $\rightarrow$  L.Holds x)
  : ProbabilityMeasure (Cfg E), (F, (: Measure (Cfg E))
  (bad S F) = 0)
  e, x e = (: Measure (Cfg E)) (atom e)

```

```

def flat (X : Type*) [Fintype X] : Set (X
   $\rightarrow \mathbb{R}_{\geq 0\infty}$ ) := {x |  $\sum$  a, x a  $\leq$  1}

```

```

noncomputable def app (t : X  $\times$  Y  $\rightarrow \mathbb{R}_{\geq 0\infty}$ ) (
  x : X  $\rightarrow \mathbb{R}_{\geq 0\infty}$ ) : Y  $\rightarrow \mathbb{R}_{\geq 0\infty}$  :=
  fun b =>  $\sum$  a, x a * t (a, b)

```

```

def lin (X Y : Type*) [Fintype X] [Fintype
  Y] : Set (X  $\times$  Y  $\rightarrow \mathbb{R}_{\geq 0\infty}$ ) :=
  {t |  $\forall$  x  $\in$  flat X, app t x  $\in$  flat Y}

```

```

theorem mem_lin_iff [DecidableEq X] (t : X
   $\times$  Y  $\rightarrow \mathbb{R}_{\geq 0\infty}$ ) :

```

```

3191      t ∈ lin X Y ↔ ∀ a, ∑ b, t (a, b) ≤ 1
3192
3193  noncomputable def pairing (x y : E → ℝ≥0∞)
3194    : ℝ≥0∞ := ∑ e, x e * y e
3195
3196  def row (a : X) : X × Y → ℝ≥0∞ := fun ab
3197    => if ab.1 = a then 1 else 0
3198
3199  def orth (S : Set (E → ℝ≥0∞)) : Set (E → ℝ
3200    ≥0∞) := {y | ∀ x ∈ S, pairing x y ≤ 1}
3201
3202  structure IsPCS (P : Set (E → ℝ≥0∞)) :
3203    Prop where
3204    closed : orth (orth P) = P
3205    total : ∀ e, ∃ x ∈ P, x e ≠ 0
3206    bounded : ∀ e, ∃ M ≠ T, ∀ x ∈ P, x e ≤ M
3207
3208  theorem isPCS_lin : IsPCS (lin X Y)
3209
3210  theorem isPCS_flat : IsPCS (flat X)
3211
3212  theorem isPCS_tensDual : IsPCS tensDual
3213
3214  theorem pcs_recipe {X Y : Type} [Fintype X
3215    ] [Fintype Y] [DecidableEq X] :
3216    lin X Y = Mix (Set.range (det : (X →
3217      Option Y) → X × Y → ℝ≥0∞))
3218
3219  def flatC (X : Type*) : X → X → Prop :=
3220    fun x y => x = y
3221
3222  def dualC {X : Type*} (c : X → X → Prop) :
3223    X → X → Prop := fun x y => x = y ∨ ¬ c
3224    x y
3225
3226  def tensC {X Y : Type*} (c : X → X → Prop)
3227    (d : Y → Y → Prop) : X × Y → X × Y →
3228    Prop :=
3229    fun p q => c p.1 q.1 ∧ d p.2 q.2
3230
3231  def arrowC {X Y : Type*} (c : X → X → Prop
3232    ) (d : Y → Y → Prop) : X × Y → X × Y →
3233    Prop :=
3234    dualC (tensC c (dualC d))
3235
3236  def IsClique {X : Type*} (c : X → X → Prop
3237    ) (B : X → Prop) : Prop := ∀ x y, B x →
3238    B y → c x y
3239
3240  def secondC : W × W → W × W → Prop :=
3241    dualC (tensC (arrowC (flatC Bool) (flatC
3242      Bool)) (arrowC (flatC Bool) (flatC
3243      Bool)))
3244
3245
3246  noncomputable def throughAll (C : Finset W
3247    ) : Bool → Option Bool := fun a =>
3248    if h : ∃ p ∈ C, p.1 = a then some (
3249      Classical.choose h).2 else none
3250
3251  theorem sharp_lin_iff_clique {X Y : Type*}
3252    [Fintype X] [Fintype Y] [DecidableEq X
3253    ] [DecidableEq Y]
3254    {t : X × Y → ℝ≥0∞} (h01 : ∀ p, t p = 0
3255      ∨ t p = 1) :
3256    t ∈ lin X Y ↔ IsClique (arrowC (flatC
3257      X) (flatC Y)) (t ⌞ = 1)
3258
3259  def cov (f : X → Option Y) (g : X' →
3260    Option Y') (pq : (X × Y) × (X' × Y')) :
3261    Prop :=
3262    f pq.1.1 = some pq.1.2 ∧ g pq.2.1 = some
3263    pq.2.2
3264
3265  def cp {X Y : Type} (p q : X × Y) : Prop
3266    := p = q ∨ p.1 ≠ q.1
3267
3268  def cp2 (s t : (X × Y) × (X' × Y')) : Prop
3269    := cp s.1 t.1 ∧ cp s.2 t.2
3270
3271  noncomputable def thru {X Y : Type} [
3272    DecidableEq X] (C : Finset (X × Y)) : X
3273    → Option Y :=
3274    fun a => if h : ∃ p ∈ C, p.1 = a then
3275      some (Classical.choose h).2 else none
3276
3277  def GTree (c : X') (af : Y' → X) (pq : (X
3278    × Y) × (X' × Y')) : Prop :=
3279    pq.2.1 = c ∧ pq.1.1 = af pq.2.2
3280
3281  def tD (X Y X' Y' : Type) [Fintype X] [
3282    Fintype Y] [Fintype X'] [Fintype Y'] :
3283    Set ((X × Y) × (X' × Y') → ℝ≥0∞) :=
3284    {t | ∀ y ∈ lin X Y, ∀ z ∈ lin X' Y', ∑ p
3285      , ∑ q, y p * t (p, q) * z q ≤ 1}
3286
3287  def FTree (a : X) (cf : Y → X') (pq : (X ×
3288    Y) × (X' × Y')) : Prop :=
3289    pq.1.1 = a ∧ pq.2.1 = cf pq.1.2
3290
3291  theorem sharp_tD_iff [Nonempty X] [
3292    Nonempty X'] {z : (X × Y) × (X' × Y') →
3293    ℝ≥0∞}
3294    (h01 : ∀ pq, z pq = 0 ∨ z pq = 1) :
3295    z ∈ tD X Y X' Y' ↔
3296    (∃ a cf, ∀ pq, z pq = 1 → FTree a cf
3297      pq) ∨ (∃ c af, ∀ pq, z pq = 1 →
3298      GTree c af pq)
3299
3300

```

```

3301 noncomputable def Core.val (K : Core A B C
3302   D) (a : A) (b : B) (c : C) (d : D) :  $\mathbb{R}_{\geq 0\infty}$  :=
3303    $\geq 0\infty$  :=
3304   K. $\mu$  a * K. $\kappa$  a b c * K. $\rho$  a b c d
3305
3306 noncomputable def Core.mass (K : Core A B
3307   C D) :  $\mathbb{R}_{\geq 0\infty}$  :=  $\sum a$ , K. $\mu$  a
3308
3309 theorem sharp_mem_sequentialG [Nonempty X]
3310   [Nonempty X'] {z : (X  $\times$  Y)  $\times$  (X'  $\times$  Y')}
3311    $\rightarrow \mathbb{R}_{\geq 0\infty}$ 
3312   (hz : z  $\in$  tD X Y X' Y') (h01 :  $\forall u$ , z
3313     u = 0  $\vee$  z u = 1) : z  $\in$  sequentialG
3314     X Y X' Y'
3315
3316 structure Core (A B C D : Type) [Fintype A
3317   ] [Fintype C] where
3318    $\mu$  : A  $\rightarrow \mathbb{R}_{\geq 0\infty}$ 
3319    $\kappa$  : A  $\rightarrow$  B  $\rightarrow$  C  $\rightarrow \mathbb{R}_{\geq 0\infty}$ 
3320    $\rho$  : A  $\rightarrow$  B  $\rightarrow$  C  $\rightarrow$  D  $\rightarrow \mathbb{R}_{\geq 0\infty}$ 
3321   h $\kappa$  :  $\forall a$  b,  $\sum c$ ,  $\kappa$  a b c  $\leq$  1
3322   h $\rho$  :  $\forall a$  b c d,  $\rho$  a b c d  $\leq$  1
3323
3324 def sequentialG (X Y X' Y' : Type) [
3325   Fintype X] [Fintype X'] : Set ((X  $\times$  Y)
3326      $\times$  (X'  $\times$  Y'))  $\rightarrow \mathbb{R}_{\geq 0\infty}$  :=
3327   {t |  $\exists$  (F : Core X Y X' Y') (G : Core X'
3328     Y' X Y), F.mass + G.mass  $\leq$  1  $\wedge$ 
3329     t = fun pq => F.val pq.1.1 pq.1.2 pq
3330       .2.1 pq.2.2 + G.val pq.2.1 pq.2.2
3331       pq.1.1 pq.1.2}
3332
3333 theorem recipe_eq_sequentialG [Nonempty X]
3334   [Nonempty X'] :
3335   Mix {z | z  $\in$  tD X Y X' Y'  $\wedge \forall u$ , z u =
3336     0  $\vee$  z u = 1} = sequentialG X Y X'
3337     Y'
3338
3339 def sharpSeqG (X Y X' Y' : Type) [Fintype
3340   X] [Fintype X'] : Set ((X  $\times$  Y)  $\times$  (X'  $\times$ 
3341     Y'))  $\rightarrow \mathbb{R}_{\geq 0\infty}$  :=
3342   {z | z  $\in$  sequentialG X Y X' Y'  $\wedge \forall u$ , z
3343     u = 0  $\vee$  z u = 1}
3344
3345 theorem sequentialG_subset_Mix :
3346   sequentialG X Y X' Y'  $\subseteq$  Mix (sharpSeqG
3347     X Y X' Y')
3348
3349 noncomputable def compl' (m : A  $\rightarrow \mathbb{R}_{\geq 0\infty}$ ) :
3350   Option A  $\rightarrow \mathbb{R}_{\geq 0\infty}$ 
3351   | none => 1 -  $\sum a$ , m a
3352   | some a => m a
3353
3354
3355
3356 abbrev ChoiceG (A B C D : Type) := (Unit  $\rightarrow$ 
3357   Option A)  $\times$  (A  $\times$  B  $\rightarrow$  Option C)  $\times$  (A  $\times$ 
3358   B  $\times$  C  $\times$  D  $\rightarrow$  Bool)
3359
3360 noncomputable def detCoreG [DecidableEq A]
3361   [DecidableEq C] ( $\sigma$  : ChoiceG A B C D)
3362   : Core A B C D
3363
3364 theorem Mix_sequentialG : Mix (sequentialG
3365   X Y X' Y') = sequentialG X Y X' Y'
3366
3367 def emb (s : W  $\times$  W) : (X  $\times$  Y)  $\times$  (X'  $\times$  Y')
3368   :=
3369   ((ex s.1.1, ey s.1.2), (ex' s.2.1, ey' s
3370     .2.2))
3371
3372 noncomputable def pentE (ex : Bool  $\rightarrow$  X) (
3373   ey : Bool  $\rightarrow$  Y) (ex' : Bool  $\rightarrow$  X') (ey' :
3374   Bool  $\rightarrow$  Y') :
3375   (X  $\times$  Y)  $\times$  (X'  $\times$  Y')  $\rightarrow \mathbb{R}_{\geq 0\infty}$  :=
3376   fun u => if u  $\in$  pentE.image (emb ex ey ex
3377     ' ey') then 2-1 else 0
3378
3379 theorem pentE_not_mix (hx : Function.
3380   Injective ex) (hy : Function.Injective
3381   ey)
3382   (hx' : Function.Injective ex') (hy' :
3383   Function.Injective ey')
3384   {n :  $\mathbb{N}$ } (w : Fin n  $\rightarrow \mathbb{R}_{\geq 0\infty}$ ) (hw :  $\sum i$ ,
3385     w i = 1) (z : Fin n  $\rightarrow$  (X  $\times$  Y)  $\times$  (X'
3386      $\times$  Y'))  $\rightarrow \mathbb{R}_{\geq 0\infty}$ 
3387   (hz :  $\forall i$ , z i  $\in$  tD X Y X' Y') (h01 :
3388      $\forall i$  u, z i u = 0  $\vee$  z i u = 1) :
3389   pentE ex ey ex' ey'  $\neq \sum i$ , w i  $\hat{\wedge}$  z i
3390
3391 theorem pentE_mem (hx : Function.Injective
3392   ex) (hy : Function.Injective ey)
3393   (hx' : Function.Injective ex') (hy' :
3394   Function.Injective ey') :
3395   pentE ex ey ex' ey'  $\in$  tD X Y X' Y'
3396
3397 noncomputable def idBool : Bool  $\times$  Bool  $\rightarrow \mathbb{R}_{\geq 0\infty}$  := fun ab => if ab.1 = ab.2 then 1
3398   else 0
3399
3400 noncomputable def det (f : X  $\rightarrow$  Option Y) :
3401   X  $\times$  Y  $\rightarrow \mathbb{R}_{\geq 0\infty}$ 
3402
3403 noncomputable def rowDist (t : X  $\times$  Y  $\rightarrow \mathbb{R}_{\geq 0}$ 
3404    $\infty$ ) (a : X) : Option Y  $\rightarrow \mathbb{R}_{\geq 0\infty}$ 
3405   | none => 1 -  $\sum b$ , t (a, b)
3406   | some b => t (a, b)
3407
3408
3409
3410

```

```

3411 noncomputable def tens (x : X → ℝ≥0∞) (y :
3412   Y → ℝ≥0∞) : X × Y → ℝ≥0∞ := fun ab =>
3413   x ab.1 * y ab.2
3414
3415 abbrev W := Bool × Bool
3416
3417 def pent : Finset (W × W) :=
3418   {((true, true), (true, true)), ((true,
3419     false), (false, false)),
3420     ((false, true), (true, false)), ((false
3421       , true), (false, true)),
3422     ((false, false), (false, false))}
3423
3424 noncomputable def pentElt : W × W → ℝ≥0∞
3425   := fun pq => if pq ∈ pent then 2-1 else
3426     0
3427
3428 def covers (f g : Bool → Option Bool) (pq
3429   : W × W) : Prop :=
3430   f pq.1.1 = some pq.1.2 ∧ g pq.2.1 = some
3431     pq.2.2
3432
3433 def compat (p q : W) : Prop := p = q ∨ p.1
3434   ≠ q.1
3435
3436 def compat2 (s t : W × W) : Prop := compat
3437   s.1 t.1 ∧ compat s.2 t.2
3438
3439 def through (p q : W) : Bool → Option Bool
3440   := fun b => if b = p.1 then some p.2
3441     else some q.2
3442
3443 theorem pent_clique_le_two : ∀ B ∈ pent.
3444   powerset,
3445   (∀ s ∈ B, ∀ t ∈ B, compat2 s t) → B.
3446     card ≤ 2
3447
3448 theorem pent_indep_le_two : ∀ B ∈ pent.
3449   powerset,
3450   (∀ s ∈ B, ∀ t ∈ B, compat2 s t → s = t
3451     ) → B.card ≤ 2
3452
3453 structure FFirst where
3454   μ : Bool → ℝ≥0∞
3455   κ : Bool → Bool → Bool → ℝ≥0∞
3456   ρ : Bool → Bool → Bool → Bool → ℝ≥0∞
3457   hμ : μ true + μ false ≤ 1
3458   hκ : ∀ a b, κ a b true + κ a b false ≤ 1
3459   hρ : ∀ a b c d, ρ a b c d ≤ 1
3460
3461 structure GFirst where
3462   μ : Bool → ℝ≥0∞
3463   κ : Bool → Bool → Bool → ℝ≥0∞
3464   ρ : Bool → Bool → Bool → Bool → ℝ≥0∞
3465
3466   hμ : μ true + μ false ≤ 1
3467   hκ : ∀ c d, κ c d true + κ c d false ≤ 1
3468   hρ : ∀ c d a b, ρ c d a b ≤ 1
3469
3470 noncomputable def FFirst.den (S : FFirst)
3471   : W × W → ℝ≥0∞ := fun pq =>
3472   S.μ pq.1.1 * S.κ pq.1.1 pq.1.2 pq.2.1 *
3473   S.ρ pq.1.1 pq.1.2 pq.2.1 pq.2.2
3474
3475 noncomputable def GFirst.den (S : GFirst)
3476   : W × W → ℝ≥0∞ := fun pq =>
3477   S.μ pq.2.1 * S.κ pq.2.1 pq.2.2 pq.1.1 *
3478   S.ρ pq.2.1 pq.2.2 pq.1.1 pq.1.2
3479
3480 noncomputable def copyStrategy : FFirst
3481   where
3482     μ a
3483
3484 def idleG : GFirst where
3485   μ _
3486
3487 def idleF : FFirst where
3488   μ _
3489
3490 def tensDual : Set (W × W → ℝ≥0∞) :=
3491   {t | ∀ y ∈ lin Bool Bool, ∀ z ∈ lin Bool
3492     Bool, ∑ p, ∑ q, y p * t (p, q) * z q
3493     ≤ 1}
3494
3495 def sequential : Set (W × W → ℝ≥0∞) :=
3496   {t | ∃ (α β : ℝ≥0∞) (F : FFirst) (G :
3497     GFirst), α + β ≤ 1 ∧ t = α â F.den
3498     + β â G.den}
3499
3500 theorem seq_pent_le_two {t : W × W → ℝ≥0∞}
3501   (ht : t ∈ sequential) : ∑ pq ∈ pent, t
3502   pq ≤ 2
3503
3504 theorem seq_pent_tight : ∃ t ∈ sequential,
3505   ∑ pq ∈ pent, t pq = 2
3506
3507 theorem pentElt_pent_sum : ∑ pq ∈ pent,
3508   pentElt pq = 5 * 2-1
3509
3510 theorem tensDual_pent_le {t : W × W → ℝ≥0∞}
3511   (ht : t ∈ tensDual) :
3512   2 * ∑ pq ∈ pent, t pq ≤ 5
3513
3514 theorem sharp_bang_mixed_zero {z : ℕ × ℕ →
3515   ℝ≥0∞} (hz : z ∈ bang) (h01 : ∀ m, z m
3516   = 0 ∨ z m = 1)
3517   {i j : ℕ} (hi : 1 ≤ i) (hj : 1 ≤ j) :
3518   z (i, j) = 0
3519
3520

```

```

3521 def orthI {E : Type*} (S : Set (E → ℝ≥0∞))
3522   : Set (E → ℝ≥0∞) := {y | ∀ x ∈ S, ∑' e
3523     , x e * y e ≤ 1}
3524
3525 def gens : Set (ℕ × ℕ → ℝ≥0∞) := {u | ∃ x
3526   ∈ flat Bool, u = prom x}
3527
3528 noncomputable def half : Bool → ℝ≥0∞ :=
3529   fun _ => 2-1
3530
3531 noncomputable def prom (x : Bool → ℝ≥0∞) :
3532   ℕ × ℕ → ℝ≥0∞ := fun m => x true ^ m.1
3533   * x false ^ m.2
3534
3535 def bang : Set (ℕ × ℕ → ℝ≥0∞) := orthI (
3536   orthI gens)
3537
3538 theorem prom_half_not_mix : prom half ∈
3539   bang ∧
3540   prom half ∉ Mix {z | z ∈ bang ∧ ∀ m, z
3541     m = 0 ∨ z m = 1}
3542
3543 def excl (s t : (X × Y) × (X' × Y')) :
3544   Prop := s ≠ t ∧ cp2 s t
3545
3546 theorem tD_eq_QSTAB : tD X Y X' Y' = QSTAB
3547   (excl (X := X) (Y := Y) (X' := X') (Y'
3548     := Y'))
3549
3550 theorem recipe_eq_STAB :
3551   Mix {z | z ∈ tD X Y X' Y' ∧ ∀ u, z u =
3552     0 ∨ z u = 1}
3553   = STAB (excl (X := X) (Y := Y) (X'
3554     := X') (Y' := Y'))
3555
3556 def cyc {n : ℕ} (i j : ZMod n) : Prop := j
3557   = i + 1 ∨ i = j + 1
3558
3559 def IsCliqueG (C : Finset V) : Prop := ∀ u
3560   ∈ C, ∀ v ∈ C, u ≠ v → adj u v
3561
3562 def IsStableG (I : Finset V) : Prop := ∀ u
3563   ∈ I, ∀ v ∈ I, ¬ adj u v
3564
3565 noncomputable def ind (I : Finset V) : V →
3566   ℝ≥0∞ := fun v => if v ∈ I then 1 else
3567   0
3568
3569 structure Antihole (adj : V → V → Prop) (n
3570   : ℕ) where
3571   h : ZMod n → V
3572   inj : Function.Injective h
3573   adj_iff : ∀ i j, i ≠ j → (adj (h i) (h j)
3574     ) ↔ ¬ cyc i j
3575

```

```

3576 def QSTAB : Set (V → ℝ≥0∞) := {x | ∀ C,
3577   IsCliqueG adj C → ∑ v ∈ C, x v ≤ 1}
3578
3579 def STAB : Set (V → ℝ≥0∞) := Mix {x | ∃ I,
3580   IsStableG adj I ∧ x = ind I}
3581
3582 structure Hole (adj : V → V → Prop) (n : ℕ
3583   ) where
3584   h : ZMod n → V
3585   inj : Function.Injective h
3586   adj_iff : ∀ i j, i ≠ j → (adj (h i) (h j)
3587     ) ↔ cyc i j
3588
3589 theorem berge_of_stab_eq_qstab (heq : STAB
3590   adj = QSTAB adj) (k : ℕ) (hk : 2 ≤ k)
3591   [NeZero (2 * k + 1)] : IsEmpty (Hole
3592     adj (2 * k + 1)) ∧ IsEmpty (
3593     Antihole adj (2 * k + 1))
3594
3595 theorem berge_of_recipe_eq_pcs
3596   (heq : Mix {z | z ∈ tD X Y X' Y' ∧ ∀ u
3597     , z u = 0 ∨ z u = 1} = tD X Y X' Y'
3598     )
3599   (k : ℕ) (hk : 2 ≤ k) [NeZero (2 * k +
3600     1)] :
3601   IsEmpty (Hole (excl (X := X) (Y := Y)
3602     (X' := X') (Y' := Y')) (2 * k + 1))
3603   ∧
3604   IsEmpty (Antihole (excl (X := X) (Y
3605     := Y) (X' := X') (Y' := Y')) (2 *
3606     k + 1))
3607

```

B.11 Multiplicative-Additive Formulas

```

3611 inductive Fm where
3612   | base (n : ℕ)
3613   | neg (A : Fm)
3614   | tens (A B : Fm)
3615   | wth (A B : Fm)
3616
3617 def web : Fm → Type
3618   | base n => Fin n
3619   | neg A => web A
3620   | tens A B => web A × web B
3621   | wth A B => web A â web B
3622
3623 def P : (A : Fm) → Set (web A → ℝ≥0∞)
3624   | base n => flat (Fin n)
3625   | neg A => orth (P A)
3626   | tens A B => orth (orth {z | ∃ x ∈ P A,
3627     ∃ y ∈ P B, z = tvec x y})
3628

```

```

3631 | with A B => {z | (fun a => z (Sum.inl a
3632 |   )) ∈ P A ∧ (fun b => z (Sum.inr b)) ∈
3633 |   P B}
3634
3635 def coh : (A : Fm) → web A → web A → Prop
3636 | base _ => fun x y => x = y
3637 | neg A => fun x y => x = y ∨ ¬ coh A x
3638 |   y
3639 | tens A B => fun s t => coh A s.1 t.1 ∧
3640 |   coh B s.2 t.2
3641 | with A B => fun s t => match s, t with
3642 |   Sum.inl a, Sum.inl a' => coh A a a'
3643 |   Sum.inr b, Sum.inr b' => coh B b b'
3644 |   _, _ => True
3645
3646 theorem isPCS_P : ∀ A : Fm, IsPCS (P A)
3647
3648 def par (A B : Fm) : Fm := neg (tens (neg
3649 |   A) (neg B))
3650 def plus (A B : Fm) : Fm := neg (wth (neg
3651 |   A) (neg B))
3652 def lolli (A B : Fm) : Fm := neg (tens A (
3653 |   neg B))
3654
3655 def plus (A B : Fm) : Fm := neg (wth (neg
3656 |   A) (neg B))
3657 def lolli (A B : Fm) : Fm := neg (tens A (
3658 |   neg B))
3659
3660 def lolli (A B : Fm) : Fm := neg (tens A (
3661 |   neg B))
3662
3663 noncomputable def tvec {X Y : Type} (x : X
3664 |   → ℝ≥0∞) (y : Y → ℝ≥0∞) : X × Y → ℝ≥0∞
3665 |   :=
3666 |   fun ab => x ab.1 * y ab.2
3667
3668 noncomputable def svec {X Y : Type} (x : X
3669 |   → ℝ≥0∞) (y : Y → ℝ≥0∞) : X â Y → ℝ≥0
3670 |   ∞
3671 |   Sum.inl a => x a
3672 |   Sum.inr b => y b
3673
3674 noncomputable def pr (s t : E) : E → ℝ≥0∞
3675 |   := Pi.single s 1 + Pi.single t 1
3676
3677 theorem invariants : ∀ A : Fm,
3678 |   (∀ s : web A, Pi.single s 1 ∈ P A ∧ ∀
3679 |     x ∈ P A, x s ≤ 1) ∧
3680 |   (∀ s t : web A, s ≠ t → coh A s t → pr
3681 |     s t ∈ P A) ∧
3682 |   (∀ s t : web A, s ≠ t → ¬ coh A s t →
3683 |     ∀ x ∈ P A, x s + x t ≤ 1)
3684
3685

```

```

3686 theorem single_mem_P (A : Fm) (s : web A)
3687 |   : Pi.single s 1 ∈ P A
3688
3689 theorem pair_mem_P (A : Fm) {s t : web A}
3690 |   (hst : s ≠ t) (h : coh A s t) : pr s t
3691 |   ∈ P A
3692
3693 theorem sharp_isClique (A : Fm) {z : web A
3694 |   → ℝ≥0∞} (hz : z ∈ P A)
3695 |   (h01 : ∀ s, z s = 0 ∨ z s = 1) : ∀ s t
3696 |   , z s = 1 → z t = 1 → coh A s t
3697
3698 theorem recipe_subset_P (A : Fm) :
3699 |   Mix {z | z ∈ P A ∧ ∀ s, z s = 0 ∨ z s
3700 |     = 1} ⊆ P A
3701
3702 theorem recipe_sharp_iff (A : Fm) {x : web
3703 |   A → ℝ≥0∞} (hx : ∀ s, x s = 0 ∨ x s =
3704 |   1) :
3705 |   x ∈ Mix {z | z ∈ P A ∧ ∀ s, z s = 0 ∨
3706 |     z s = 1} ↔ x ∈ P A
3707
3708 theorem Mix_subset_orth {S T : Set (E → ℝ
3709 |   ≥0∞)} (h : S ⊆ orth T) : Mix S ⊆ orth T
3710
3711 theorem P_lolli_base (n m : ℕ) : P (lolli
3712 |   (base n) (base m)) = lin (Fin n) (Fin m
3713 |   )
3714
3715 def second (n m n' m' : ℕ) : Fm := neg (
3716 |   tens (lolli (base n) (base m)) (lolli (
3717 |     base n') (base m'))))
3718
3719 theorem P_second (n m n' m' : ℕ) :
3720 |   P (second n m n' m') = tD (Fin n) (Fin
3721 |     m) (Fin n') (Fin m')
3722
3723 def boolFin {k : ℕ} (hk : 2 ≤ k) : Bool →
3724 |   Fin k
3725
3726 theorem recipe_first_order (n m : ℕ) :
3727 |   P (lolli (base n) (base m)) =
3728 |   ConstructiveProb.Mix (Set.range
3729 |     (det : (Fin n → Option (Fin m)) →
3730 |       Fin n × Fin m → ℝ≥0∞))
3731
3732 theorem recipe_second_order {n m n' m' : ℕ
3733 |   } (hn : 0 < n) (hn' : 0 < n') :
3734 |   ConstructiveProb.Mix {z | z ∈ P (
3735 |     second n m n' m') ∧ ∀ u, z u = 0 ∨
3736 |     z u = 1}
3737 |   = sequentialG (Fin n) (Fin m) (Fin n
3738 |     ') (Fin m')
3739
3740

```

```

3741 theorem recipe_second_order_ssubset {n m n' : ℕ} (hn : 2 ≤ n) (hm : 2 ≤ m) (hn' : 2 ≤ n')
3742   (hm' : 2 ≤ m') :
3743   ConstructiveProb.Mix {z | z ∈ P (
3744     second n m n' m') ∧ ∀ u, z u = 0 ∨
3745     z u = 1}
3746   â P (second n m n' m')
3747
3748 def Î : W × W → web S2 := emb (boolFin
3749   le_rfl) (boolFin le_rfl) (boolFin
3750   le_rfl) (boolFin le_rfl)
3751
3752 abbrev S2 : Fm := second 2 2 2 2
3753
3754 abbrev TT : Fm := neg (tens S2 S2)
3755
3756 noncomputable def shannonF : Finset (web (
3757   tens S2 S2)) := shannon.image fun u =>
3758   (Î u.1, Î u.2)
3759
3760 theorem shannonF_clique : ∀ u ∈ shannonF,
3761   ∀ v ∈ shannonF, coh TT u v
3762
3763 noncomputable def pentS2 : web S2 → ℝ≥0
3764   :=
3765   pentE (boolFin le_rfl) (boolFin le_rfl)
3766   (boolFin le_rfl) (boolFin le_rfl)
3767
3768 theorem shannonF_not_mem : (fun u => if u
3769   ∈ shannonF then (1 : ℝ≥0) else 0) ∉ P
3770   TT
3771
3772
3773
3774
3775
3776

```

B.12 Quantum Event Logic

```

3777 def vec : Fin 18 → Fin 4 → ℤ := ![
3778   [-1, 1, 1, 1], ![0, 0, 0, 1], ![0, 0,
3779     1, 0], ![0, 0, 1, 1], ![0, 1, -1, 0],
3780     ![0, 1, 0, -1],
3781     ![0, 1, 0, 0], ![1, -1, -1, 1], ![1, -1,
3782       0, 0], ![1, -1, 1, -1], ![1, 0, -1,
3783         0], ![1, 0, 0, -1],
3784         ![1, 0, 0, 1], ![1, 0, 1, 0], ![1, 1,
3785           -1, 1], ![1, 1, 0, 0], ![1, 1, 1,
3786             -1], ![1, 1, 1, 1]]
3787
3788 def bas : Fin 9 → Fin 4 → Fin 18 := ![
3789   [1, 2, 15, 8], [1, 6, 13, 10], [9, 7,
3790     15, 3], [9, 17, 10, 5], [2, 6, 12,
3791       11],
3792       [7, 17, 11, 4], [14, 16, 8, 3], [14,
3793         0, 13, 5], [16, 0, 12, 4]]
3794
3795

```

```

3796 def dot (u v : Fin 4 → ℤ) : ℤ := ∑ k, u k
3797   * v k
3798
3799 theorem basis_orth : ∀ b : Fin 9, ∀ i j :
3800   Fin 4, i ≠ j → dot (vec (bas b i)) (vec
3801     (bas b j)) = 0
3802
3803 theorem occ_two : ∀ v : Fin 18,
3804   (univ.filter (fun p : Fin 9 × Fin 4 =>
3805     bas p.1 p.2 = v)).card = 2
3806
3807 theorem no_ks : ¬ ∃ f : Fin 18 → Bool, ∀ b
3808   , (univ.filter (fun i => f (bas b i))).
3809     card = 1
3810
3811 def certain : Set (Fin 18 → ℝ≥0) :=
3812   {s | IsSharpVec s ∧ ∀ b, ∑ i, s (bas b i)
3813     = 1}
3814
3815 theorem no_certain_state : certain = ∅
3816
3817 theorem Mix_certain_empty : Mix certain =
3818   ∅
3819
3820 theorem parseval (Î : Fin 4 → ℝ) (b : Fin
3821   9) :
3822   ∑ i, (∑ k, Î k * vec (bas b i) k) ^ 2
3823     / ∑ k, ((vec (bas b i) k : ℝ)) ^ 2
3824     = ∑ k, Î k ^ 2
3825
3826 def states : Set (Fin 18 → ℝ≥0) := {x | ∀
3827   b, ∑ i, x (bas b i) = 1}
3828
3829 noncomputable def born (Î : Fin 4 → ℝ) :
3830   Fin 18 → ℝ≥0 := fun v =>
3831   ENNReal.ofReal ((∑ k, Î k * vec v k) ^
3832     2 / ∑ k, ((vec v k : ℝ)) ^ 2)
3833
3834 theorem born_mem {Î : Fin 4 → ℝ} (hÎ : ∑
3835   k, Î k ^ 2 = 1) : born Î ∈ states
3836
3837 theorem born_not_mix {Î : Fin 4 → ℝ} (hÎ :
3838   ∑ k, Î k ^ 2 = 1) :
3839   born Î ∈ states ∧ born Î ∉ Mix
3840     certain
3841
3842
3843
3844
3845
3846
3847
3848
3849
3850

```

B.13 Finite Frames without Choice

```

3844 def below (a : α) : Finset α := Finset.
3845   univ.filter (λ· ≤ a)
3846
3847 def ind (j x : α) : ℕ := if j ≤ x then 1
3848   else 0
3849
3850

```

```

3851 def JoinPrime (j :  $\alpha$ ) : Prop :=  $\forall x y : \alpha$ ,
3852   j  $\leq$  x  $\sqcup$  y  $\rightarrow$  j  $\leq$  x  $\vee$  j  $\leq$  y
3853
3854 theorem exists_joinPrime_aux :  $\forall (n : \mathbb{N})$  (
3855   a b :  $\alpha$ ), (below a).card = n  $\rightarrow$   $\neg$  a  $\leq$  b
3856  $\rightarrow$ 
3857    $\exists j$ , j  $\leq$  a  $\wedge$   $\neg$  j  $\leq$  b  $\wedge$  JoinPrime j
3858
3859 theorem exists_sharp_separating_fin [
3860   BoundedOrder  $\alpha$ ] {a b :  $\alpha$ } (hab :  $\neg$  a  $\leq$ 
3861   b) :
3862    $\exists v : \alpha \rightarrow \mathbb{N}$ , ( $\forall x$ , v x = 0  $\vee$  v x = 1)
3863      $\wedge$  Monotone v  $\wedge$  v  $\perp$  = 0  $\wedge$  v  $\top$  = 1  $\wedge$ 
3864     ( $\forall x y$ , v x + v y = v (x  $\sqcup$  y) + v (x
3865        $\sqcap$  y))  $\wedge$  v a = 1  $\wedge$  v b = 0
3866
3867 theorem em_of_sharp_complement_fin [
3868   BoundedOrder  $\alpha$ ] (a c :  $\alpha$ )
3869   (h :  $\forall v : \alpha \rightarrow \mathbb{N}$ , ( $\forall x$ , v x = 0  $\vee$  v x
3870     = 1)  $\rightarrow$  Monotone v  $\rightarrow$  v  $\perp$  = 0  $\rightarrow$  v  $\top$  =
3871     1  $\rightarrow$ 
3872     ( $\forall x y$ , v x + v y = v (x  $\sqcup$  y) + v (x
3873        $\sqcap$  y))  $\rightarrow$  v a + v c = 1) :
3874   a  $\sqcup$  c =  $\top$ 
3875
3876
3877

```

B.14 Choice-Free Finite Results

```

3878 def wt (b : Bool) (m : Nat) : Nat := if b
3879   then m else 0
3880
3881 def mem : Op  $\rightarrow$  Bool  $\rightarrow$  Bool
3882 | bot, _ => false
3883 | halts, b => b
3884 | top, _ => true
3885
3886 def le (o o' : Op) : Bool := mem o true  $\leq$ 
3887   mem o' true && mem o false  $\leq$  mem o'
3888   false
3889
3890 def inf : Op  $\rightarrow$  Op  $\rightarrow$  Op
3891 | bot, _ => bot
3892 | _, bot => bot
3893 | halts, _ => halts
3894 | top, o => o
3895
3896 def val (wT wF : Nat) (o : Op) : Nat := wt
3897   (mem o true) wT + wt (mem o false) wF
3898
3899 def inClosure (x : Bool) : Bool := [bot,
3900   halts, top].all fun o => !mem o x ||
3901   mem o true
3902
3903 def frontier (x : Bool) : Bool :=
3904   inClosure x && !mem halts x
3905

```

```

inductive Op | bot | halts | top
deriving DecidableEq

def neg : Op  $\rightarrow$  Op
| bot => top
| halts => bot
| top => bot

theorem neg_spec :  $\forall o o' : Op$ , (inf o' o
  = bot)  $\leftrightarrow$  le o' (neg o) = true

theorem neg_halts : neg halts = bot

theorem frontier_iff :  $\forall x : Bool$ ,
  frontier x = true  $\leftrightarrow$  x = false

theorem slack_eq (wT wF : Nat) :
  val wT wF top = val wT wF halts + val
    wT wF (neg halts) + wt (frontier
      false) wF

def fs : (n : Nat)  $\rightarrow$  (Fin n  $\rightarrow$  Nat)  $\rightarrow$  Nat
| 0, _ => 0
| n + 1, f => f <0, by omega> + fs n (
  fun i => f <i.val + 1, by omega>)

def allFin : (n : Nat)  $\rightarrow$  (Fin n  $\rightarrow$  Bool)  $\rightarrow$ 
  Bool
| 0, _ => true
| n + 1, p => p <0, by omega> && allFin
  n (fun i => p <i.val + 1, by omega>)

def ext (ops : List (Fin n  $\rightarrow$  Bool)) (U :
  Fin n  $\rightarrow$  Bool) (x : Fin n) : Bool :=
  ops.any fun 0 => 0 x && allFin n (fun y
    => !(0 y && U y))

def bd (ops : List (Fin n  $\rightarrow$  Bool)) (U :
  Fin n  $\rightarrow$  Bool) (x : Fin n) : Bool :=
  !U x && !ext ops U x

theorem sub_ext {ops : List (Fin n  $\rightarrow$  Bool)}
  {U : Fin n  $\rightarrow$  Bool} {0 : Fin n  $\rightarrow$  Bool}
  (h0 : 0  $\in$  ops) (hd :  $\forall y$ , (!(0 y && U
    y)) = true) {x : Fin n} (hx : 0 x =
    true) :
  ext ops U x = true

theorem slack_eq_boundary_fin (ops : List
  (Fin n  $\rightarrow$  Bool)) (U : Fin n  $\rightarrow$  Bool) ( $\mu$  :
  Fin n  $\rightarrow$  Nat) :

```

```

3961      fs n  $\mu$  = fs n (fun x => wt (U x) ( $\mu$  x)
3962      ) + fs n (fun x => wt (ext ops U x)
3963      ( $\mu$  x)) +
3964      fs n (fun x => wt (bd ops U x) ( $\mu$  x)
3965      )
3966
3967 def isPre : List Bool → List Bool → Bool
3968 | [], _ => true
3969 | _ :: _, [] => false
3970 | a :: u, b :: v => a == b && isPre u v
3971
3972 def Incomp (u v : List Bool) : Prop :=
3973   isPre u v = false  $\wedge$  isPre v u = false
3974
3975 def PF : List (List Bool) → Prop
3976 | [] => True
3977 | w :: S => ( $\forall v \in S$ , Incomp w v)  $\wedge$  PF S
3978
3979 def tails (b : Bool) : List (List Bool) →
3980   List (List Bool)
3981 | [] => []
3982 | [] :: S => tails b S
3983 | (a :: t) :: S => if a = b then t ::
3984   tails b S else tails b S
3985
3986 def ksum (L : Nat) : List (List Bool) →
3987   Nat
3988 | [] => 0
3989 | w :: S => 2 ^ (L - w.length) + ksum L
3990   S
3991
3992 theorem kraft :  $\forall (L : Nat) (S : List ($ 
3993   List Bool)), PF S  $\rightarrow$  ( $\forall w \in S$ , w.length
3994    $\leq$  L)  $\rightarrow$ 
3995   ksum L S  $\leq$  2 ^ L
3996
3997 def Sat {n : Nat} (r : Row n) (w : Fin n  $\rightarrow$ 
3998   Int) : Prop :=
3999   0  $\leq$  dot n r.c w  $\wedge$  (r.s = true  $\rightarrow$  0 < dot
4000   n r.c w)
4001
4002 def comb {n : Nat} (p q : Int) (r1 r2 :
4003   Row n) : Row n :=
4004   (fun i => p * r1.c i + q * r2.c i, (
4005     decide (0 < p) && r1.s || (decide (0
4006     < q) && r2.s))
4007
4008 def zeroS (n : Nat) : Row n := (fun _ =>
4009   0, true)
4010
4011 def lastc (r : Row (n + 1)) : Int := r.c (
4012   Fin.last n)
4013
4014
4015
4016 def proj (r : Row (n + 1)) : Row n := (fun
4017   i => r.c i.castSucc, r.s)
4018
4019 def posR (rows : List (Row (n + 1))) :=
4020   rows.filter fun r => decide (0 < lastc
4021   r)
4022
4023 def negR (rows : List (Row (n + 1))) :=
4024   rows.filter fun r => decide (lastc r <
4025   0)
4026
4027 def zerR (rows : List (Row (n + 1))) :=
4028   rows.filter fun r => decide (lastc r =
4029   0)
4030
4031 def negR (rows : List (Row (n + 1))) :=
4032   rows.filter fun r => decide (lastc r <
4033   0)
4034
4035 def zerR (rows : List (Row (n + 1))) :=
4036   rows.filter fun r => decide (lastc r =
4037   0)
4038
4039 def pair (p q : Row (n + 1)) : Row n :=
4040   proj (comb (-lastc q) (lastc p) p q)
4041
4042 def step (rows : List (Row (n + 1))) :
4043   List (Row n) :=
4044   (zerR rows).map proj ++ (posR rows).
4045   flatMap fun p => (negR rows).map fun
4046   q => pair p q
4047
4048 def extd (D W : Int) (w' : Fin n  $\rightarrow$  Int) :
4049   Fin (n + 1)  $\rightarrow$  Int :=
4050   fun i => if h : i.val < n then D * w' <i
4051     .val, h> else W
4052
4053 def hrow ( $\mu$  : Fin N  $\rightarrow$  Int) ( $\hat{1}_{\frac{1}{2}} \alpha$  : Int) ( $\beta$ 
4054   : Fin m  $\rightarrow$  Int) : Fin (N + 1)  $\rightarrow$  Int :=
4055   fun i => if h : i.val < N then  $\mu$  <i.val,
4056     h> +  $\alpha$  + d * dot m  $\beta$  (S <i.val, h>)
4057   else  $\hat{1}_{\frac{1}{2}} - \alpha - \text{dot } m \beta$  y
4058
4059 def hullRows : List (Row (N + 1)) :=
4060   (List.finRange N).map (fun j => (hrow S
4061     y d (fun k => if k = j then 1 else 0)
4062     0 0 (fun _ => 0), false)) ++
4063   [(hrow S y d (fun _ => 0) 1 0 (fun _ =>
4064     0), true),
4065     (hrow S y d (fun _ => 0) 0 1 (fun _ =>
4066     0), false),
4067     (hrow S y d (fun _ => 0) 0 (-1) (fun _
4068     => 0), false)] ++
4069
4070

```

```

4071 (List.finRange m).flatMap (fun e =>
4072   [⟨hrow S y d (fun _ => 0) 0 0 (fun k
4073     => if k = e then 1 else 0), false⟩,
4074   ⟨hrow S y d (fun _ => 0) 0 0 (fun k
4075     => if k = e then -1 else 0), false
4076   ⟩])
4077
4078 structure Row (n : Nat) where
4079   c : Fin n → Int
4080   s : Bool
4081
4082 inductive Der {n : Nat} (rows : List (Row
4083   n)) : Row n → Prop
4084   | base {r : Row n} : r ∈ rows → Der rows
4085     r
4086   | comb {r1 r2 : Row n} (p q : Int) : 0 ≤
4087     p → 0 ≤ q → Der rows r1 → Der rows r
4088     2 →
4089     Der rows (comb p q r1 r2)
4090
4091 theorem fm : ∀ (n : Nat) (rows : List (Row
4092   n)), (∃ w, ∀ r ∈ rows, Sat r w) ∨ Der
4093   rows (zeroS n)
4094
4095 theorem complete (hd : 0 < d)
4096   (hlaw : ∀ (c : Fin m → Int) (k : Int),
4097     (∀ j, k ≤ dot m c (S j)) → k * d ≤
4098     dot m c y) :
4099   ∃ (lam : Fin N → Int) (t : Int), 0 < t
4100     ∧ (∀ j, 0 ≤ lam j) ∧
4101     dot N (fun _ => 1) lam = t ∧ ∀ e, d
4102     * dot N lam (fun j => S j e) = t
4103     * y e
4104
4105 theorem sound (hd : 0 < d) (lam : Fin N →
4106   Int) (t : Int) (ht : 0 < t) (hl : ∀ j,
4107   0 ≤ lam j)
4108   (hsum : dot N (fun _ => 1) lam = t) (
4109     hx : ∀ e, d * dot N lam (fun j => S
4110     j e) = t * y e)
4111   (c : Fin m → Int) (k : Int) (hc : ∀ j,
4112     k ≤ dot m c (S j)) : k * d ≤ dot m
4113     c y
4114
4115 def union {A B : Nat → Nat → Bool} (a :
4116   EOD A) (b : EOD B) : EOD (fun m m' => A
4117     m m' || B m m') where
4118   N
4119
4120 def inter {A B : Nat → Nat → Bool} (a :
4121   EOD A) (b : EOD B) : EOD (fun m m' => A
4122     m m' && B m m') where
4123   N
4124
4125

```

```

def compl {A : Nat → Nat → Bool} (a : EOD
  A) : EOD (fun m m' => !A m m') where
  N
def val (b : Bool) : Nat := if b then 1
  else 0
structure EC (S : Nat → Bool) where
  N : Nat
  v : Bool
  spec : ∀ m, N ≤ m → S m = v
def rect {S T : Nat → Bool} (s : EC S) (t
  : EC T) : EOD (fun m m' => S m && T m')
  where
  N
def tri : EOD (fun m m' => decide (m < m'))
  ) where
  N
structure EOD (A : Nat → Nat → Bool) where
  N : Nat
  lt : Bool
  eq : Bool
  gt : Bool
  spec : ∀ m m', N ≤ m → N ≤ m' →
    A m m' = if m < m' then lt else if m =
    m' then eq else gt
def L {A : Nat → Nat → Bool} (w : EOD A) :
  Bool := w.lt
/-- The iterated limit with `m → ∞` first.
-/
def R {A : Nat → Nat → Bool} (w : EOD A) :
  Bool := w.gt
def R {A : Nat → Nat → Bool} (w : EOD A) :
  Bool := w.gt
theorem L_modular {A B} (a : EOD A) (b :
  EOD B) :
  val (L a) + val (L b) = val (L (union
    a b)) + val (L (inter a b))
theorem rect_L {S T} (s : EC S) (t : EC T)
  : L (rect s t) = (s.v && t.v)
theorem rect_R {S T} (s : EC S) (t : EC T)
  : R (rect s t) = (s.v && t.v)
theorem products_differ : L tri ≠ R tri

```

B.15 Valued Frames and the Valuation Monoid

```

noncomputable def ValF :
  inductive Gen : Multiset P → Multiset P →
    Prop
    | modular (a b : P) : Gen ({a} + {b}) ({
      a  $\sqcup$  b} + {a  $\sqcap$  b})
    | bot : Gen { $\perp$ }  $\emptyset$ 

def con : AddCon (Multiset P) := addConGen
  (Gen P)

abbrev V := (con P).Quotient

def  $\iota$  (a : P) : V P := (con P).mk' {a}

structure Modular (f : P → M) : Prop where
  bot : f  $\perp$  = 0
  modular :  $\forall$  a b, f a + f b = f (a  $\sqcup$  b) +
    f (a  $\sqcap$  b)

def lift (f : P → M) (hf : Modular f) : V
  P  $\rightarrow$  M

theorem lift_ $\iota$  (f : P → M) (hf : Modular f)
  (a : P) : lift f hf ( $\iota$  a) = f a

theorem hom_ext {h h' : V P  $\rightarrow$  M} (H :  $\forall$  a
  , h ( $\iota$  a) = h' ( $\iota$  a)) : h = h'

theorem modular_of_hom (h : V P  $\rightarrow$  M) :
  Modular (fun a => h ( $\iota$  a))

noncomputable def lvaluation_equiv {P :
  Type*} [Lattice P] [BoundedOrder P] :
  LValuation P  $\simeq$ 
  {h : V P  $\rightarrow$   $\mathbb{R}_{\geq 0\infty}$  // h ( $\iota$   $\top$ ) = 1  $\wedge$ 
    Monotone fun a => h ( $\iota$  a)}

theorem exists_flag (x : V P) :  $\exists$  l : List
  P, l.Pairwise ( $\hat{A} \cdot \leq \hat{A} \cdot$ )  $\wedge$  x = (l.map  $\iota$ 
  ).sum

```

B.16 A Language for the Second-Order Type

```

inductive PEnd
  | halt
  | diverge
  | choose (k : Nat) (w : Fin k  $\rightarrow$   $\mathbb{R}_{\geq 0\infty}$ ) (
    hw :  $\sum$  i, w i  $\leq$  1) (p : Fin k  $\rightarrow$  PEnd)

inductive POne (C D : Type)
  | diverge

```

```

  | choose (k : Nat) (w : Fin k  $\rightarrow$   $\mathbb{R}_{\geq 0\infty}$ ) (
    hw :  $\sum$  i, w i  $\leq$  1) (p : Fin k  $\rightarrow$  POne
    C D)
  | call (c : C) (k : D  $\rightarrow$  PEnd)

inductive P0 (X Y X' Y' : Type)
  | diverge
  | choose (k : Nat) (w : Fin k  $\rightarrow$   $\mathbb{R}_{\geq 0\infty}$ ) (
    hw :  $\sum$  i, w i  $\leq$  1) (p : Fin k  $\rightarrow$  P0 X
    Y X' Y')
  | callF (a : X) (k : Y  $\rightarrow$  POne X' Y')
  | callG (c : X') (k : Y'  $\rightarrow$  POne X Y)

noncomputable def PEnd.den : PEnd  $\rightarrow$   $\mathbb{R}_{\geq 0\infty}$ 
  | .halt => 1
  | .diverge => 0
  | .choose _ w _ p =>  $\sum$  i, w i * (p i).
    den

noncomputable def POne.den {C D : Type} [
  DecidableEq C] : POne C D  $\rightarrow$  C  $\times$  D  $\rightarrow$   $\mathbb{R}_{\geq 0\infty}$ 
  | .diverge => fun _ => 0
  | .choose _ w _ p => fun cd =>  $\sum$  i, w i
    * (p i).den cd
  | .call c k => fun cd => if cd.1 = c
    then (k cd.2).den else 0

noncomputable def P0.den : P0 X Y X' Y'  $\rightarrow$ 
  (X  $\times$  Y)  $\times$  (X'  $\times$  Y')  $\rightarrow$   $\mathbb{R}_{\geq 0\infty}$ 
  | .diverge => fun _ => 0
  | .choose _ w _ p => fun u =>  $\sum$  i, w i *
    (p i).den u
  | .callF a k => fun u => if u.1.1 = a
    then (k u.1.2).den u.2 else 0
  | .callG c k => fun u => if u.2.1 = c
    then (k u.2.2).den u.1 else 0

theorem P0.sound :  $\forall$  p : P0 X Y X' Y', p.
  den  $\in$  sequentialG X Y X' Y'

theorem P0.den_mem_recipe [Nonempty X] [
  Nonempty X'] (p : P0 X Y X' Y') :
  p.den  $\in$  Mix {z | z  $\in$  tD X Y X' Y'  $\wedge$   $\forall$ 
    u, z u = 0  $\vee$  z u = 1}

def PEnd.ofBool (b : Bool) : PEnd := if b
  then .halt else .diverge

theorem P0.sharp_definable [Nonempty X] [
  Nonempty X'] {z : (X  $\times$  Y)  $\times$  (X'  $\times$  Y')  $\rightarrow$ 
   $\mathbb{R}_{\geq 0\infty}$ }

```

```

4291      (hz : z ∈ tD X Y X' Y') (h01 : ∀ u, z
4292        u = 0 ∨ z u = 1) : ∃ p : P0 X Y X'
4293        Y', p.den = z
4294

```

B.17 The Boolean Case

```

4295 noncomputable def comp (m : Bool → ℝ≥0∞) :
4296   Option Bool → ℝ≥0∞
4297   | none => 1 - (m true + m false)
4298   | some b => m b
4299
4300 noncomputable def bern (r : ℝ≥0∞) : Bool →
4301   ℝ≥0∞
4302   | true => r
4303   | false => 1 - r
4304
4305 abbrev Choice := (Unit → Option Bool) × (
4306   Bool × Bool → Option Bool) × (Bool ×
4307   Bool × Bool × Bool → Bool)
4308
4309 noncomputable def detCore (σ : Choice) (a
4310   b c d : Bool) : ℝ≥0∞ :=
4311   (if σ.1 () = some a then 1 else 0) * (if
4312     σ.2.1 (a, b) = some c then 1 else 0)
4313   *
4314   (if σ.2.2 (a, b, c, d) = true then 1
4315     else 0)
4316
4317 noncomputable def detF (σ : Choice) :
4318   FFirst where
4319     μ a
4320
4321 noncomputable def detG (σ : Choice) :
4322   GFirst where
4323     μ c
4324
4325 def sharpSeq : Set (W × W → ℝ≥0∞) := {z |
4326   z ∈ sequential ∧ ∀ pq, z pq = 0 ∨ z pq
4327   = 1}
4328
4329 structure Beh where
4330   μ : Bool → ℝ≥0∞
4331   κ : Bool → Bool → Bool → ℝ≥0∞
4332   ρ : Bool → Bool → Bool → Bool → ℝ≥0∞
4333   hκ : ∀ a b, κ a b true + κ a b false ≤ 1
4334   hρ : ∀ a b c d, ρ a b c d ≤ 1
4335
4336 noncomputable def Beh.val (B : Beh) (a b c
4337   d : Bool) : ℝ≥0∞ := B.μ a * B.κ a b c
4338   * B.ρ a b c d
4339
4340 noncomputable def Beh.mass (B : Beh) : ℝ≥0
4341   ∞ := B.μ true + B.μ false
4342

```

```

4343 def seqU : Set (W × W → ℝ≥0∞) :=
4344   {t | ∃ BF BG : Beh, BF.mass + BG.mass ≤
4345     1 ∧
4346     t = fun pq => BF.val pq.1.1 pq.1.2 pq
4347       .2.1 pq.2.2 + BG.val pq.2.1 pq.2.2
4348       pq.1.1 pq.1.2}
4349
4350 theorem FFirst.den_mem_Mix (F : FFirst) :
4351   F.den ∈ Mix sharpSeq
4352
4353 theorem sequential_subset_Mix : sequential
4354   ⊆ Mix sharpSeq
4355
4356 theorem Mix_sequential : Mix sequential =
4357   sequential
4358
4359 theorem recipe_second_order_eq_sequential'
4360   :
4361   Mix {z | z ∈ tensDual ∧ ∀ pq, z pq = 0
4362     ∨ z pq = 1} = sequential
4363
4364 abbrev W2 := (W × W) × (W × W)
4365
4366 noncomputable def tens2 (t t' : W × W → ℝ
4367   ≥0∞) : W2 → ℝ≥0∞ := fun u => t u.1 * t'
4368   u.2
4369
4370 def gensTT : Set (W2 → ℝ≥0∞) := {u | ∃ t ∈
4371   tensDual, ∃ t' ∈ tensDual, u = tens2 t
4372   t'}
4373
4374 def PTT : Set (W2 → ℝ≥0∞) := orth (orth
4375   gensTT)
4376
4377 def PTTdual : Set (W2 → ℝ≥0∞) := orth
4378   gensTT
4379
4380 def PTTdual : Set (W2 → ℝ≥0∞) := orth
4381   gensTT
4382
4383 def cohTT : W2 → W2 → Prop := tensC
4384   secondC secondC
4385
4386 def cohTTdual : W2 → W2 → Prop := dualC
4387   cohTT
4388
4389 def cohTTdual : W2 → W2 → Prop := dualC
4390   cohTT
4391
4392 def shannon : Finset W2 :=
4393   {(((true, true), (true, true)), ((true,
4394     true), (true, true))),
4395     (((false, true), (false, true)), ((
4396       false, true), (true, false))),
4397     (((false, true), (true, false)), ((
4398       false, false), (false, false))),
4399     (((false, false), (false, false))),
4400

```

```

4401      (((true, false), (false, false)), ((
4402        false, true), (false, true))),
4403      (((false, false), (false, false)), ((
4404        true, false), (false, false))))}
4405
4406  theorem shannon_clique : IsClique
4407    cohTTdual (Â· ∈ shannon)
4408
4409  theorem shannon_not_mem :
4410    (fun u => if u ∈ shannon then (1 : ℝ≥0
4411      ∞) else 0) ∉ PTTdual
4412
4413  theorem recipe_eq_denotations [Nonempty X]
4414    [Nonempty X'] :
4415    Mix {z | z ∈ tD X Y X' Y' ∧ ∀ u, z u =
4416      0 ∨ z u = 1} = Set.range (P0.den (
4417      X := X) (Y := Y)
4418      (X' := X') (Y' := Y'))
4419
4420
4421

```

C The Aczél Development

This appendix sketches the from-scratch construction behind the product-rule half of Theorem 30. The target shape is Aczél’s associativity theorem [Aczél 1966]: a suitably regular associative $F : \mathbb{R} \rightarrow \mathbb{R} \rightarrow \mathbb{R}$ regroups to multiplication, $G(F x y) = G(x) \cdot G(y)$ for all x, y and a strictly monotone G . `mathlib` has no associativity representation, no Hölder-style embedding of ordered semigroups on intervals, and no Cauchy functional equation under monotonicity (only under continuity).

Hypotheses. A Scale bundles: associativity of F , *divisibility* $\forall c \exists x, F x x = c$, positivity on the cone $x < F x c$, joint order-preservation, joint continuity, a unit u , and identity at $-\infty$, meaning $F x z \rightarrow x$ as $z \rightarrow -\infty$. Order-preservation is required by the additivity proof below. Divisibility yields the 2^n -th roots used in the dyadic construction below. Without it the generator is not definable. It is assumed directly in the bundle, though its interval form follows from the intermediate value theorem (`exists_diag_eq`). Non-vacuity: *log-sum-exp*, $F x y = \log(e^x + e^y)$ with $u = 0$, satisfies every field of the bundle (`logSumExpScale`), with intended generator globally `exp`.

Construction. Write $a^{(n)}$ for n right-associated copies of a under F (`Fpow`). Associativity alone makes $n \mapsto a^{(n)}$ a semigroup morphism from $(\mathbb{N}, +)$. Commutativity is not assumed, and follows instead as a corollary of the generator. Unboundedness of the iterates is derived: were they bounded, monotone convergence would give a fixed point $F L a = L$, contradicting positivity, and unboundedness yields the Archimedean property. Divisibility yields a tower of 2^n -th roots $u \succ u^{1/2} \succ u^{1/4} \succ \dots$ (`droot`), where 2^l copies of the finer root compose under F to one copy of the coarser root. Counting the number of copies of the n -th root not

exceeding b (`dcount`, well defined by the Archimedean property) gives dyadic approximants $g_n(b) = \text{dcount}(b, n)/2^n$. The refinement inequality $2N + 1 \leq N' \leq 2N + 2$, where N and N' are the counts at consecutive refinement levels n and $n + 1$, makes (g_n) monotone and Cauchy, and the generator is its limit, $g(b) = \bigsqcup_n g_n(b)$.

Results (all sorry-free, standard axioms). On the cone $\{x \mid u \leq x\}$ we prove three things. Normalization: $g(u) = 1$. Additivity: $g(F x y) = g(x) + g(y)$, since $\text{dcount } x + \text{dcount } y + 1 \leq \text{dcount}(F x y) \leq \text{dcount } x + \text{dcount } y + 3$ and the 2^{-n} error term vanishes. And strict monotonicity, proved without any continuity of g : a root inserted between $x < y$ contributes a full extra block of 2^{n-M} copies, M being the refinement level of the inserted root. Exponentiating gives the multiplicative form, a strictly monotone G with $G(F x y) = G(x) G(y)$ on the cone (`exists_mul_generator`), the conclusion of Aczél’s theorem restricted to the cone (`aczelStatement_cone`). The reorientation $\bar{G} = \exp(-g)$ gives the $[0, 1]$ -valued form used in Section 6.

Remaining gap. The constructed g is discontinuous at the unit (it is 0 below u), so matching the classical statement verbatim needs the off-cone extension (a group completion) and continuity of the extension. This is open. It is not vacuous: the log-sum-exp example admits a global continuous generator (`hasOrderedGenerator_logSumExp`). Of Hölder’s classical hypotheses, the Archimedean property is proved, and is what enables the embedding into $\mathbb{R}$. The interval form of divisibility follows from the intermediate value theorem. Only the embedding’s global regularity remains.

4511	4566
4512	4567
4513	4568
4514	4569
4515	4570
4516	4571
4517	4572
4518	4573
4519	4574
4520	4575
4521	4576
4522	4577
4523	4578
4524	4579
4525	4580
4526	4581
4527	4582
4528	4583
4529	4584
4530	4585
4531	4586
4532	4587
4533	4588
4534	4589
4535	4590
4536	4591
4537	4592
4538	4593
4539	4594
4540	4595
4541	4596
4542	4597
4543	4598
4544	4599
4545	4600
4546	4601
4547	4602
4548	4603
4549	4604
4550	4605
4551	4606
4552	4607
4553	4608
4554	4609
4555	4610
4556	4611
4557	4612
4558	4613
4559	4614
4560	4615
4561	4616
4562	4617
4563	4618
4564	4619
4565	4620

4621	4676
4622	4677
4623	4678
4624	4679
4625	4680
4626	4681
4627	4682
4628	4683
4629	4684
4630	4685
4631	4686
4632	4687
4633	4688
4634	4689
4635	4690
4636	4691
4637	4692
4638	4693
4639	4694
4640	4695
4641	4696
4642	4697
4643	4698
4644	4699
4645	4700
4646	4701
4647	4702
4648	4703
4649	4704
4650	4705
4651	4706
4652	4707
4653	4708
4654	4709
4655	4710
4656	4711
4657	4712
4658	4713
4659	4714
4660	4715
4661	4716
4662	4717
4663	4718
4664	4719
4665	4720
4666	4721
4667	4722
4668	4723
4669	4724
4670	4725
4671	4726
4672	4727
4673	4728
4674	4729
4675	4730